ThinkRook



Flask Web Development Bootcamp | Hands-On Learning

- By Thinknyx Technologies



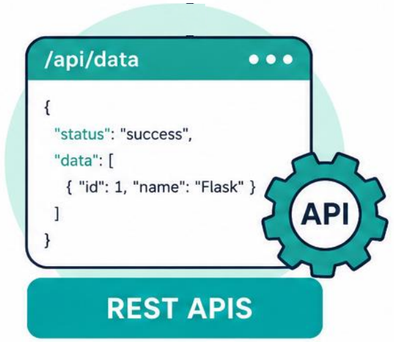
Python Web Framework

Simplicity

Flexibility

Ease of Learning

ThinkRook
By Thinknyx Technologies LLP







Flask



LIGHTWEIGHT | POWERFUL

MODERN WEB DEVELOPMENT FOUNDATION

ThinkRook
By Thinknyx Technologies LLP



Flask

Fundamentals of Web Development

Building secure & production-ready applications

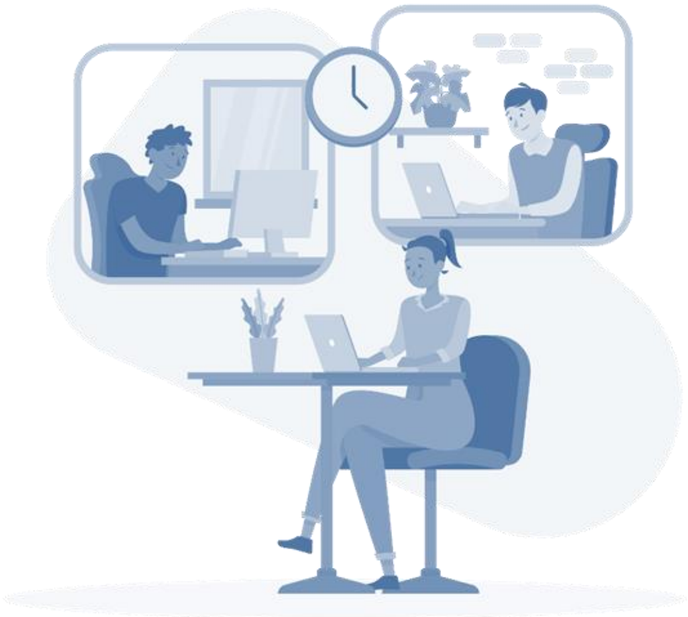


Real-world Demonstrations & Practical Use Cases

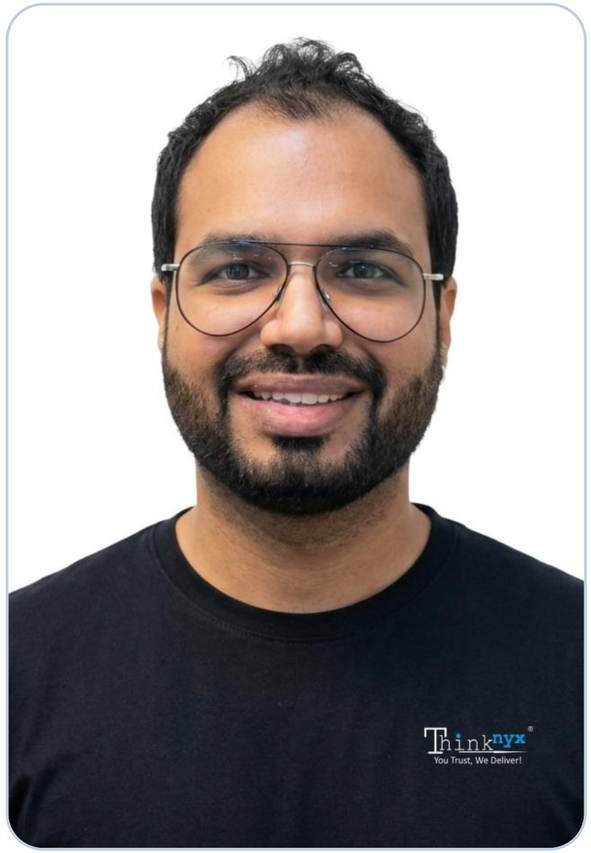
- ❑ Concepts
- ❑ Demonstrations
- ❑ Use Cases
- ❑ Capstone Project

ThinkRook
By Thinknyx Technologies LLP

Thinknyx®



Thinknyx Team



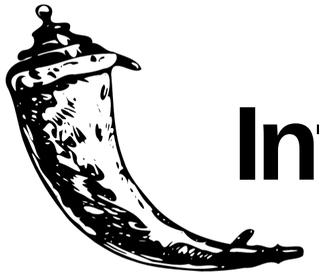
Yogesh Raheja

ThinkRook
By Thinknyx Technologies LLP



- Introduction to Web Applications & Flask
- Flask Setup & First Application
- Routing, Requests & Responses
- Jinja2 Templating & Form Handling
- Database Integration with SQLAlchemy
- Authentication & Authorization
- Building RESTful APIs
- Debugging, Error Handling & Logging
- Blueprints & Flask Security Best Practices
- Testing & Production-Ready Deployments

ThinkRook
By Thinknyx Technologies LLP



Introduction to W. eb Applications

Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP



Introduction to Web Applications

Section Overview

- What is a web application
- How it works in real life

ThinkRook
By Thinknyx Technologies LLP

Introduction to Web Applications

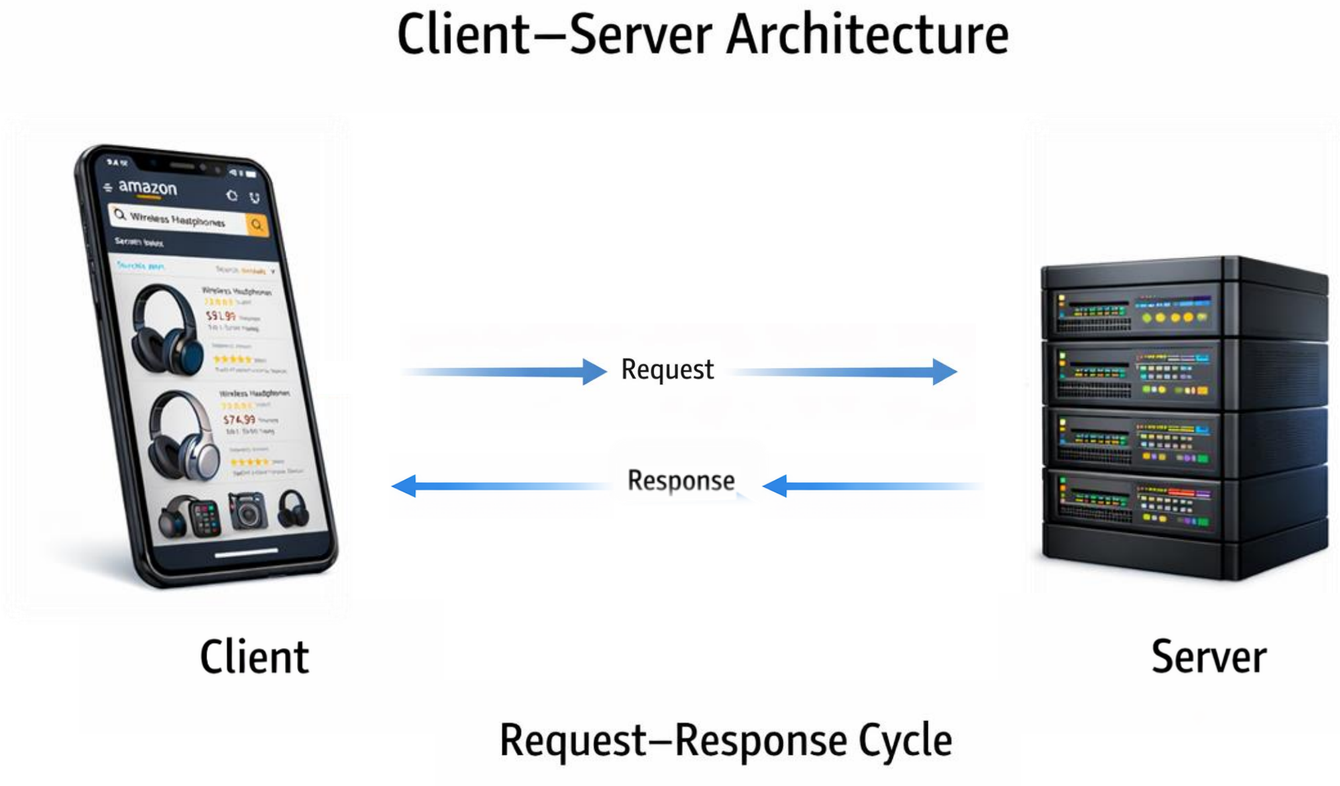


- **Web application**
  - Runs in a browser or app
  - Communicates over the internet
  - Processes user requests
  - Returns dynamic responses

Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

How a Web Application Works

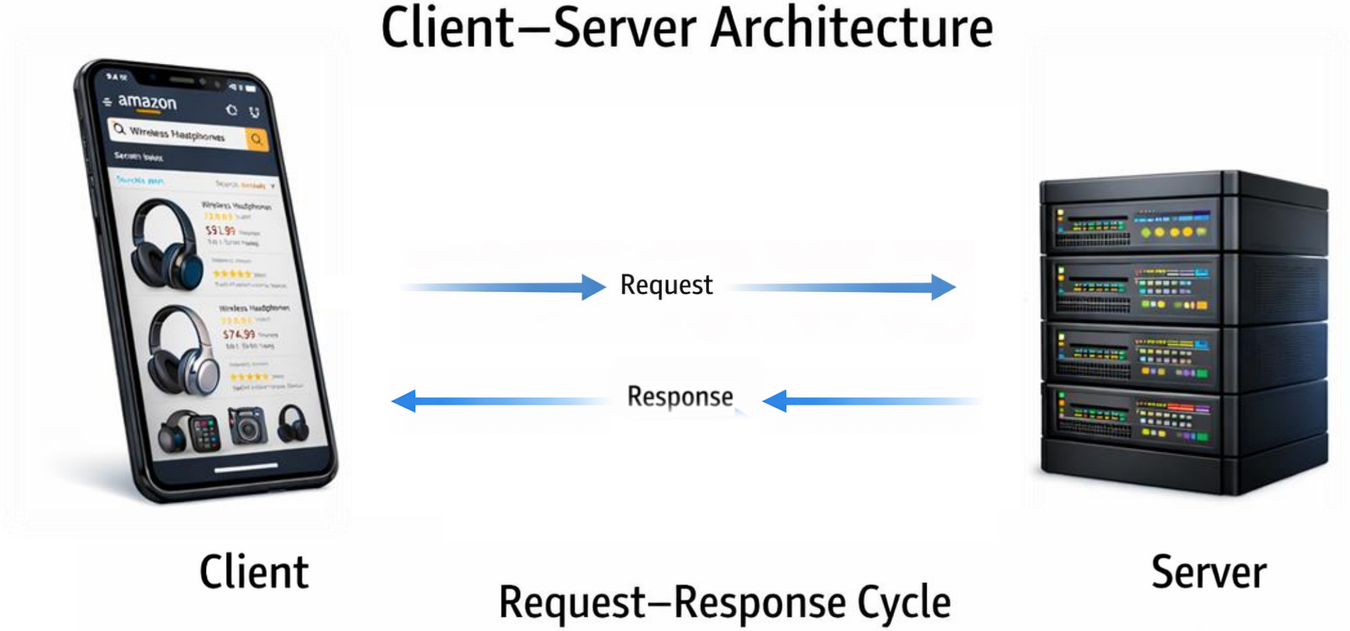


Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

Roles Inside a Web Application

What the user sees and does



How the system thinks & responds

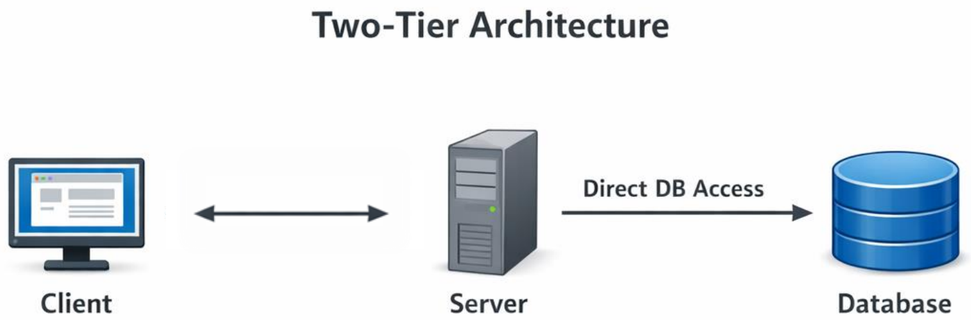
- User Interaction
  - Collects input
  - Sends requests
  - Displays responses

- Processing
  - Executes business logic
  - Interacts with data
  - Ensures security & consistency

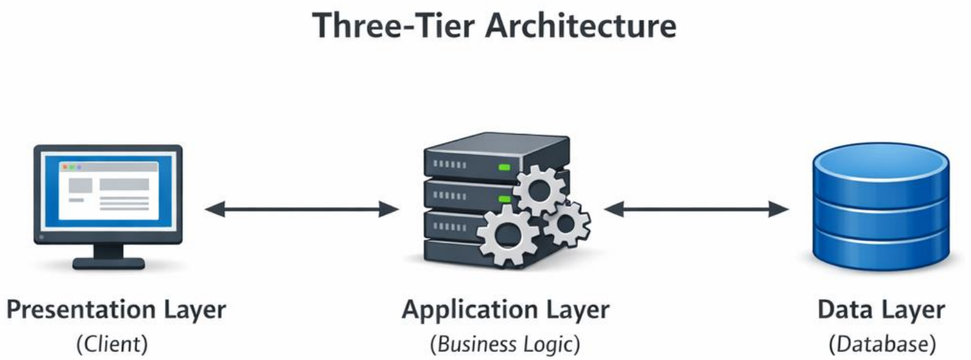
ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Architecture of Web Applications



This approach is simple but not suitable for large-scale systems



This separation makes the system easier to maintain, scale, and secure

ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

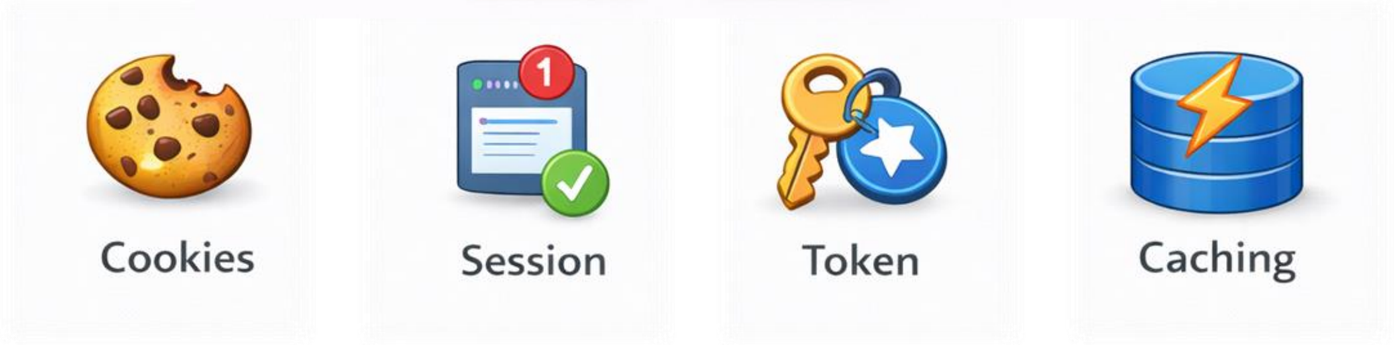
State Management in Web Applications



By default: **Stateless**



Stateful



Maintains continuity across multiple requests



ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Core Components of a Web Application



User Interface & Interaction
Displays Products, Search, & Buttons



Data Storage
Stores Products, Users & Orders



Application Programming Interfaces
Connects Frontend & Backend



Processing & Business Logic
Handles Requests & Data Processing



Authentication
Verify Identity

Authorization
Control Access

ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Supporting Systems in Web Applications

Version Control



Manage Code Changes
Track Changes & Collaborate with Git

Testing and Optimization



Improve Quality & Performance
Identify Issues & Enhance Scalability

Deployment and Hosting



Deployment and Hosting
Deliver Application to Servers or Cloud

ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Supporting Systems in Web Applications



Encrypt User Connections
Ensure Secure Data Transfer via TLS

- ✓ HTTPS = secure version of HTTP
- ✓ HTTP uses request–response model
- ✓ HTTP sends data in plain text
- ✓ Plain text data can be intercepted
- ✓ HTTPS uses TLS for encryption
- ✓ Data is sent in encrypted form
- ✓ Prevents interception and tampering
- ✓ Requires digital certificate
- ✓ Certificate issued by Certificate Authority
- ✓ Verifies server identity
- ✓ Ensures secure communication
- ✓ HTTPS is essential for secure deployment

ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP



What are Web Frameworks and Their Type

Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

What are Web Frameworks and Their Type

A **web application** is built using multiple components, and each component relies on different technologies working together









ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

What are Web Frameworks and Their Type



Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

What are Web Frameworks and Their Type

Building a Web Application from Scratch



HTTP requests



Routing system



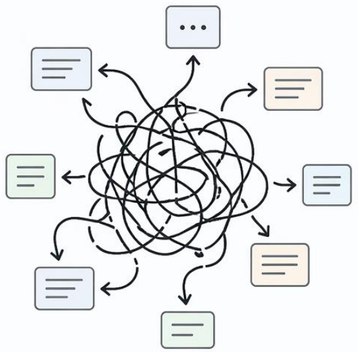
Authentication logic



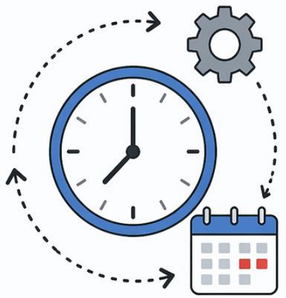
Security vulnerabilities



Database queries



Complex



Time-consuming



Difficult to maintain

ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

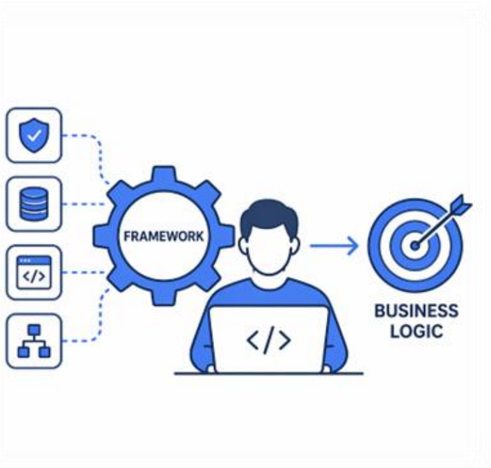
What are Web Frameworks and Their Type

Web Frameworks

A **web framework** is a structured foundation that provides **ready-made tools, libraries, and best practices** to build web applications efficiently







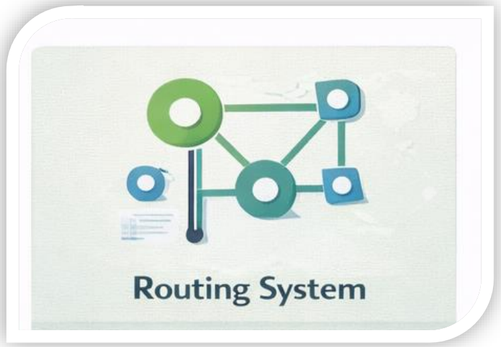


Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

What are Web Frameworks and Their Type

➢ Core Responsibilities of a Web Framework



Maps URLs to specific functions



Manages HTTP methods



Generates dynamic HTML using backend data



Allows working with databases using objects



Executes logic before or after request processing



Handles Input validation, CSRF protection, XSS filtering

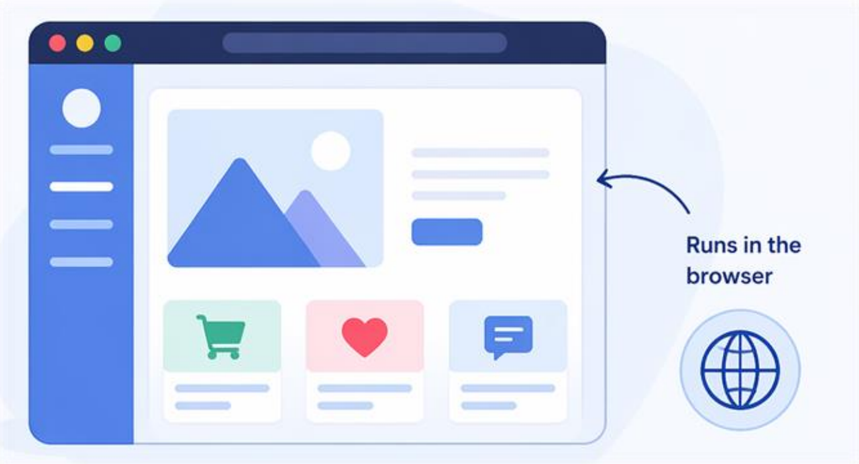
Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

What are Web Frameworks and Their Type

- **Types of Frameworks**

Frontend Framework













ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

What are Web Frameworks and Their Type

- **Types of Frameworks**

Backend Frameworks



Flask spring express

Business Logic

Database Operations

Authentication

API Development

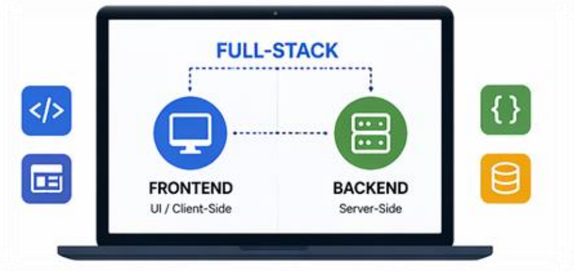
ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

What are Web Frameworks and Their Type

- **Types of Frameworks**

Full-Stack Frameworks



Django

Ruby on Rails

Laravel

ORM

Authentication

Templating

Admin Panels

ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

What are Web Frameworks and Their Type

➢ Types of Frameworks

Microframeworks

```
from flask import Flask, jsonify

app = Flask(__name__)

@app.route('/api/hello')
def hello():
    return jsonify({'message': 'Hello
    World!'})
```

Flask

Sinatra

Minimal Built-in Features

High Flexibility

Easy to Extend

APIs

Small Services

Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

What are Web Frameworks and Their Type

- **Types of Frameworks**

MVC-Based Frameworks





ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

What are Web Frameworks and Their Type

How should we choose right framework?

- ✓ Project complexity
- ✓ Performance requirements
- ✓ Team expertise
- ✓ Ecosystem and community support
- ✓ Scalability needs



A web framework is not just a coding tool; it is an architectural backbone for your application









ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Thank you



ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

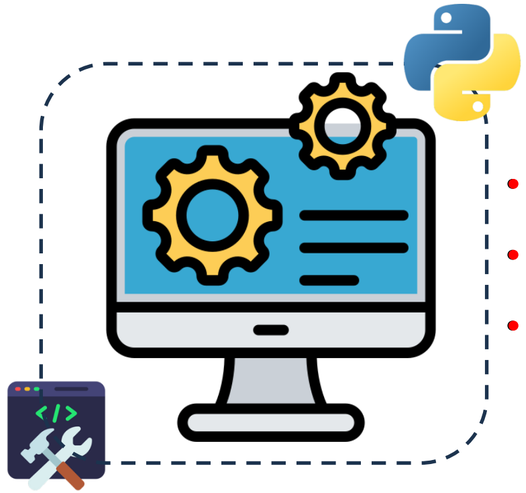


What is Flask?

Copyright © Thinknyx Technologies LLP

What is Flask?





- Handling user requests
- Processing orders
- Sending responses quickly



Flask is a lightweight web framework in Python that helps you build web applications and APIs quickly, without dealing with low-level complexities

Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

Why Flask?

Simple to start | Lightweight | Flexible | Easy to extend

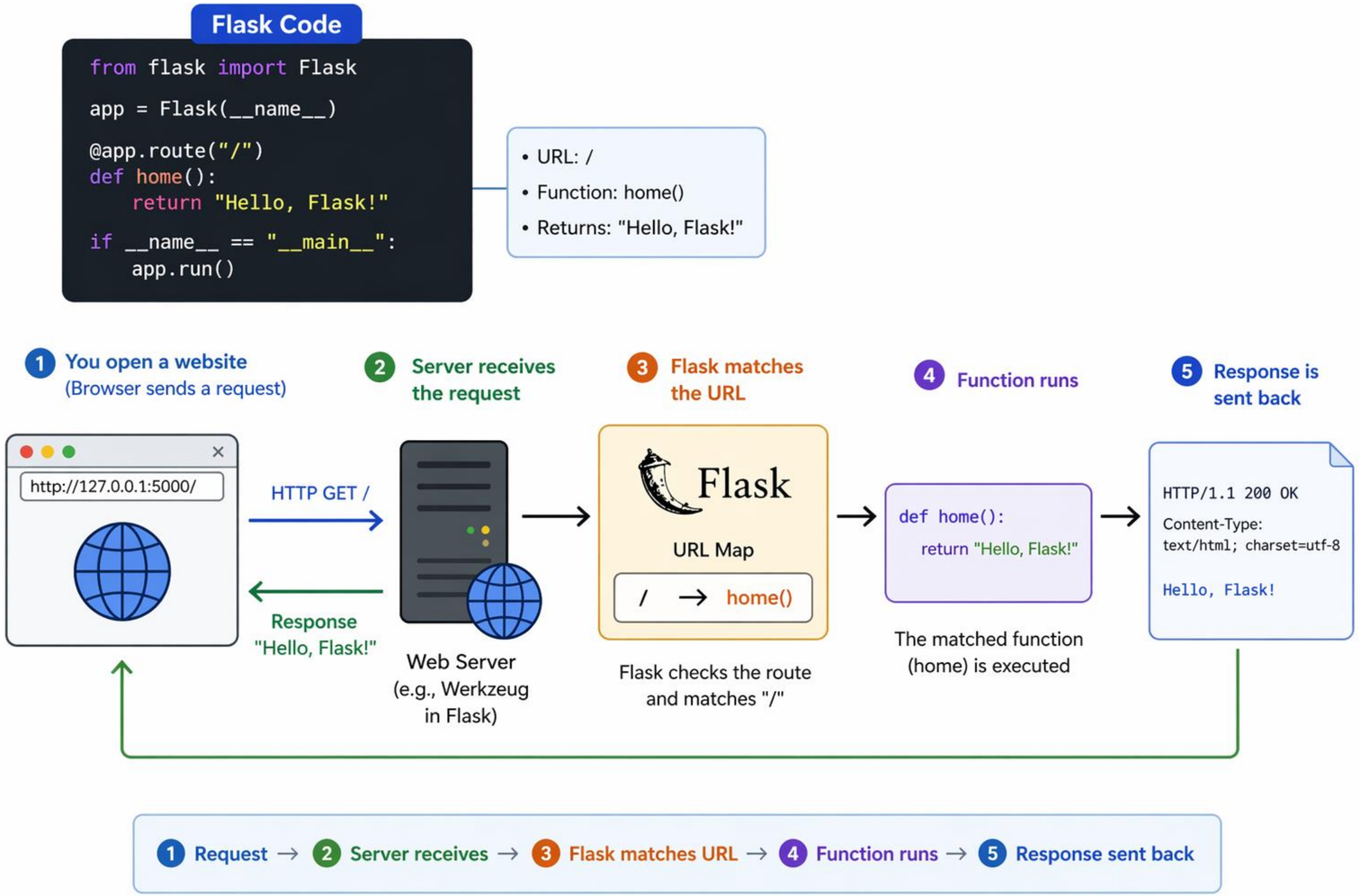
- Example

```python
from flask import Flask
app = Flask(__name__)
@app.route("/")
def home():
    return "Hello, Flask!"
if __name__ == "__main__":
    app.run()
```

ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Working of Flask



ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Working of Flask

- **Werkzeug** → Handles requests and responses
- **Jinja2** → Renders HTML templates

- **Flask is commonly used for:**



REST APIs



Microservices



AI/ML model deployment



Dashboards

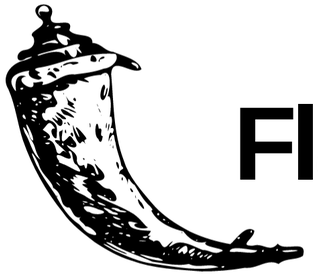


Backend for mobile apps



Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP



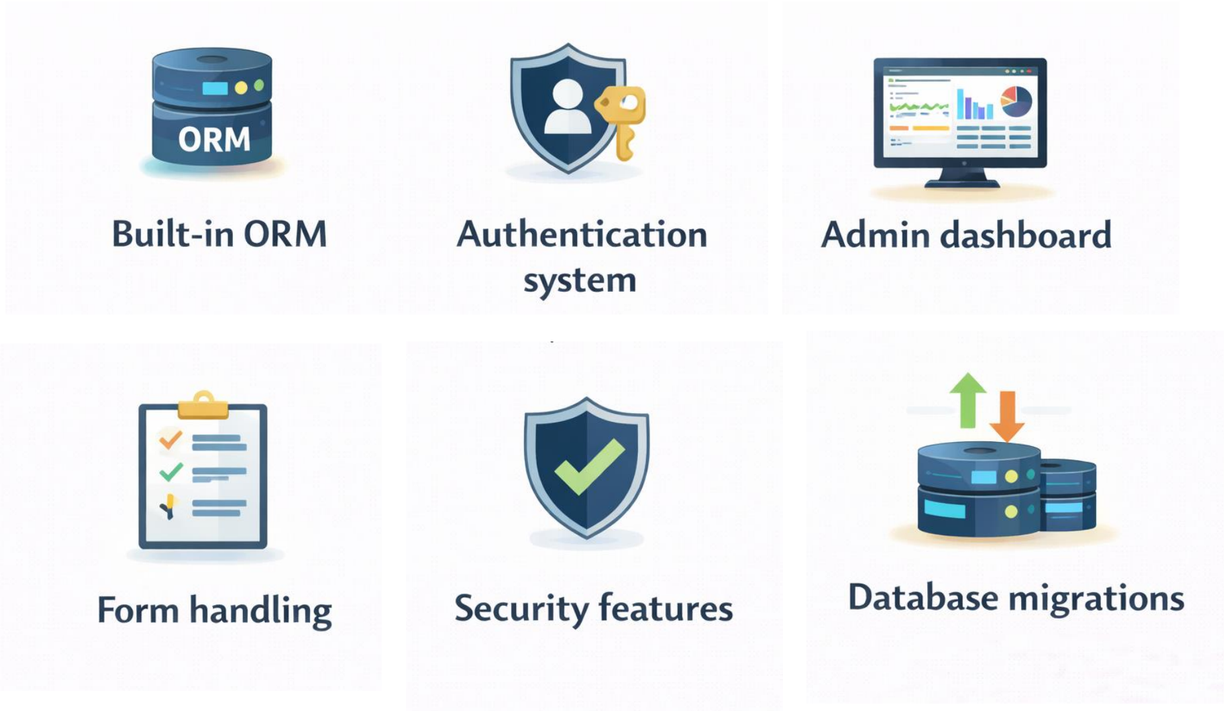
Flask vs Django

Copyright © Thinknyx Technologies LLP

Flask vs Django

- **Django**

Django is a full-stack, batteries-included web framework



Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

Flask vs Django

- **Django**



Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

Flask vs Django

- **Flask**

Flask is a **micro** web framework

Minimal in core functionality

Routing **Request handling** **Basic templating**

If you want authentication?



Flask-Login

If you want migrations?



Flask-Migrate

- Freedom
- Doesn't Force structure
- Design your own architecture

ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Flask vs Django

Django	Flask
Structured, predefined, feature-rich, best for large applications	Lightweight, flexible, minimal; best for APIs and microservices
Used for building E-commerce platforms, enterprise portals, admin-heavy systems, and large SaaS applications	Used for building REST APIs, AI/ML model backends, microservices, and lightweight SaaS products
Helps you develop faster because most features are already built in	Easy to get started with, ideal for beginners to quickly grasp core concepts
Full restaurant kitchen with all equipment ready	Modular cooking station where you bring only what you need

Flask framework

ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP



Demo | Setting up Python and VS Code

Copyright © Thinknyx Technologies LLP



Demo | Installing Flask

Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP



Demo | Creating First Flask Application

Copyright © Thinknyx Technologies LLP



Demo | Flask Project Structure

Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

Flask Project Structure



Flask app

- Files become messy
- Code gets harder to manage
- Bugs increase

Flask gives the freedom but with that comes responsibility







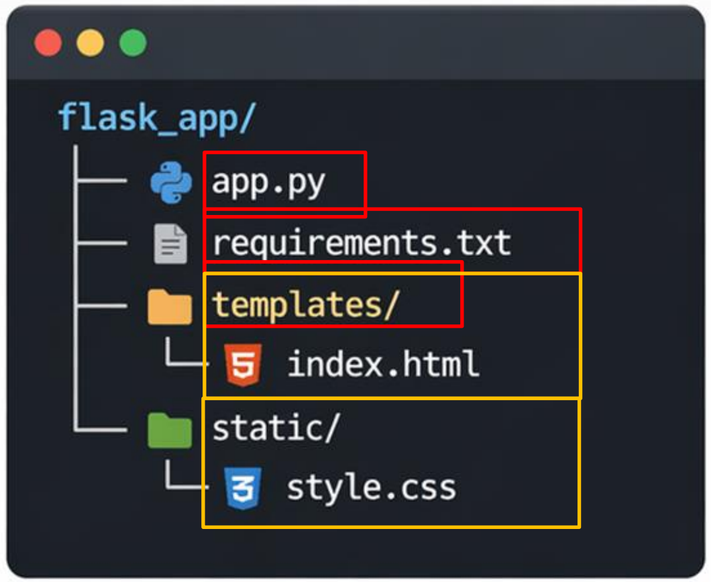


ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Flask Project Structure

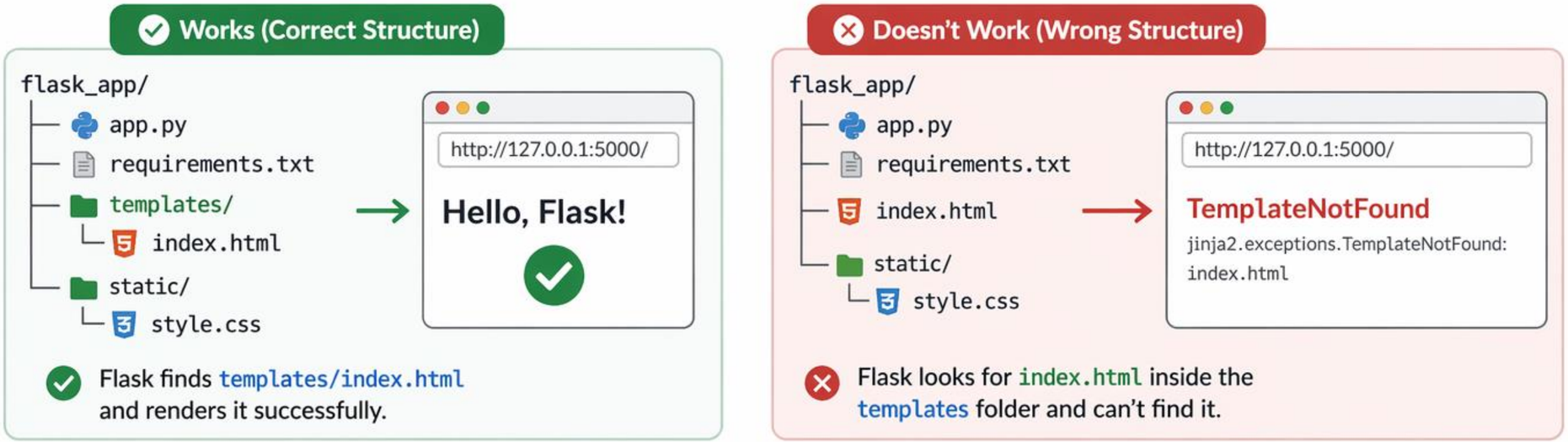
Basic structure of flask application



Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

Flask Project Structure



ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Thank you



ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP



Routing in Flask

Copyright © Thinknyx Technologies LLP



Routing in Flask

Section Overview

- Introduction to Basic Routing in Flask
- Understand Dynamic Routing
- Explore HTTP Methods
- Handle Multiple Routes

ThinkRook
By Thinknyx Technologies LLP



Demo | Basic Routing

Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

Basic Routing



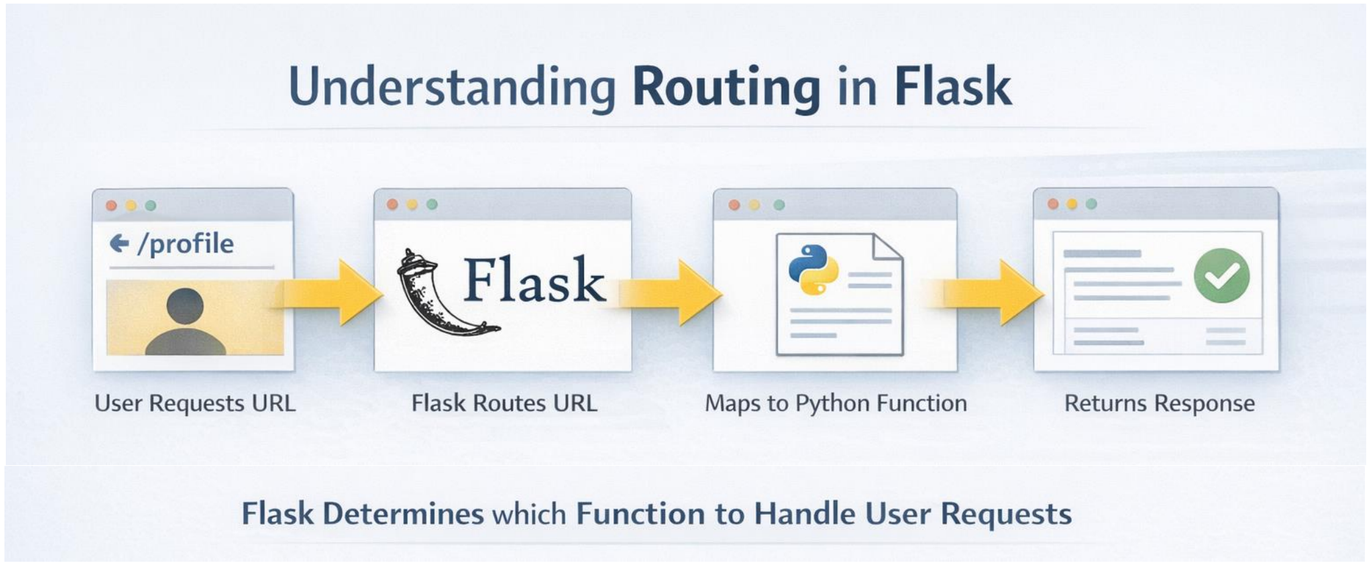
Routing



ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Basic Routing

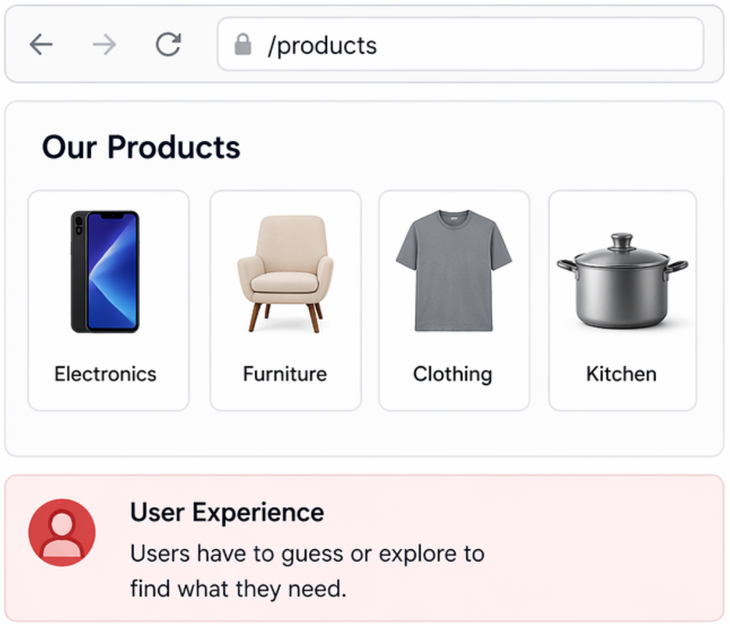


ThinkRook
By Thinknyx Technologies LLP

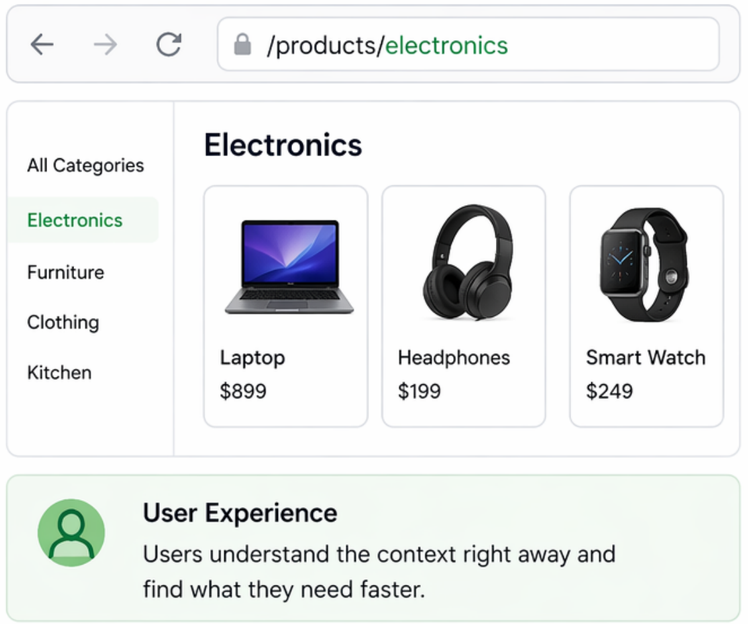
Copyright © Thinknyx Technologies LLP

Basic Routing

Generic Route



Specific Route



Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP



Demo | Dynamic URL Routing

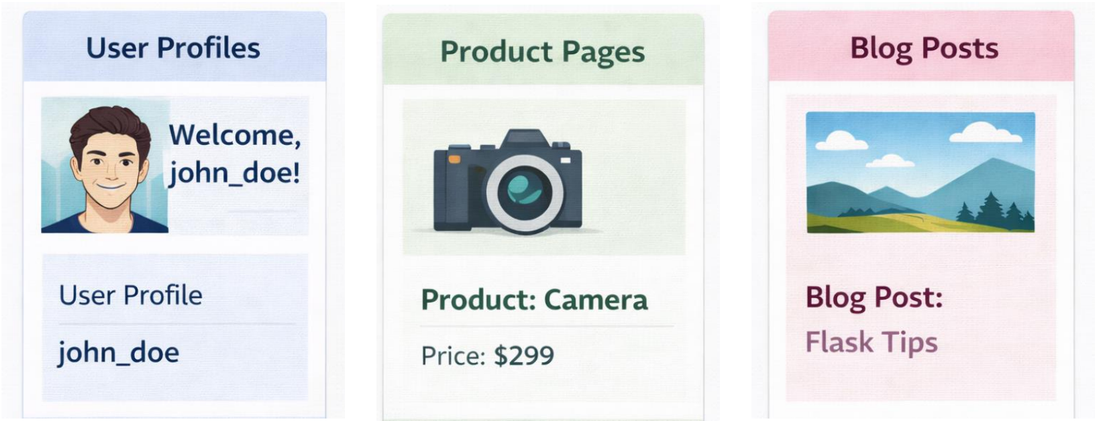
Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

Dynamic URL Routing

Dynamic routing in Flask allows you to create URLs that can accept **variable values** instead of being fixed

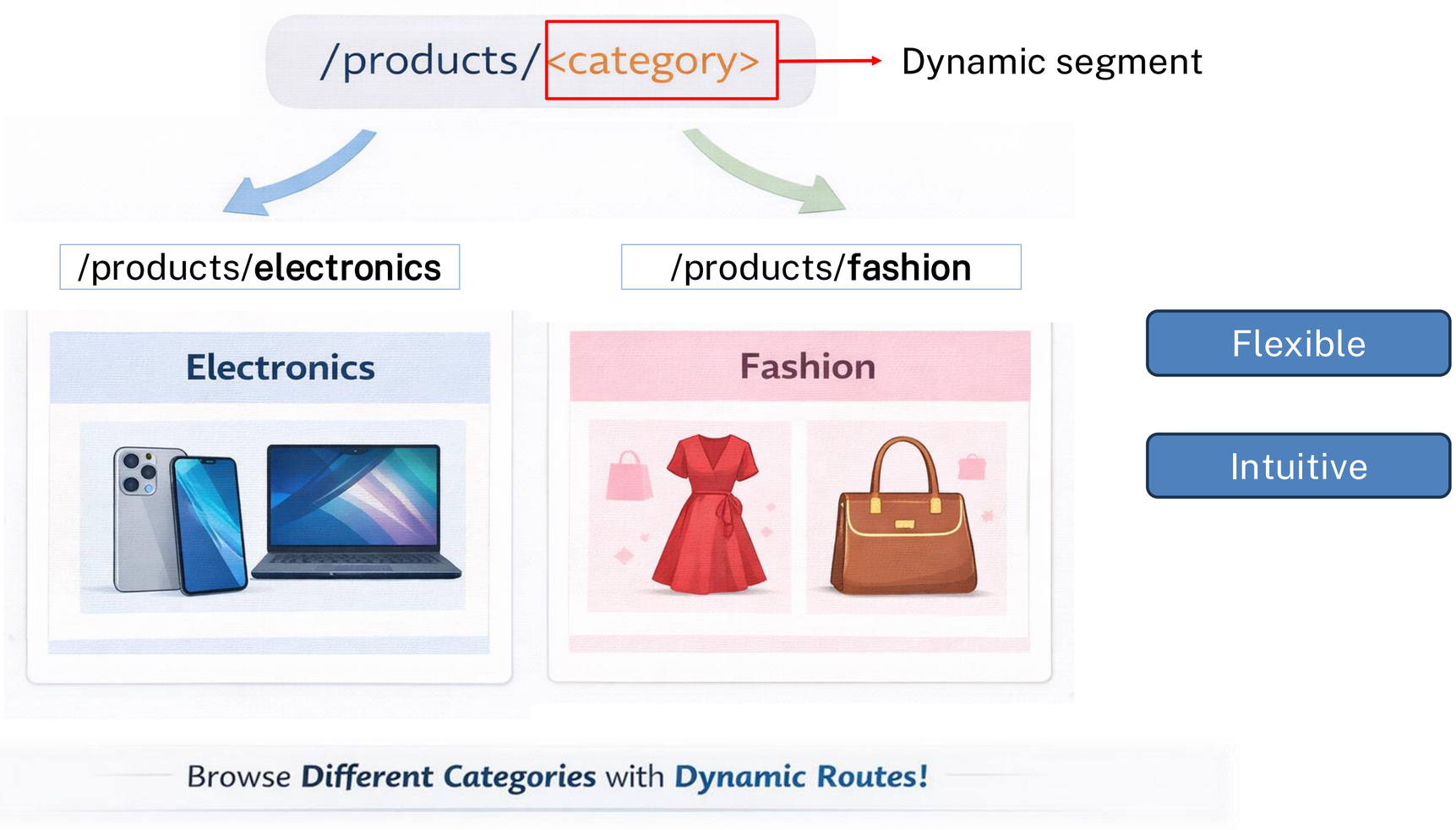
Essential when building real-world applications where data changes dynamically



Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

Dynamic URL Routing



ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

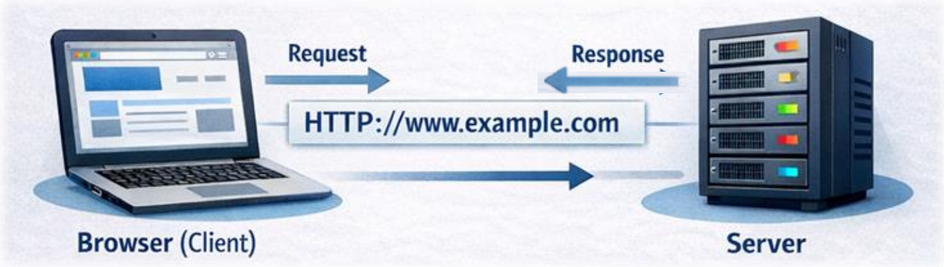


Demo | HTTP Methods (GET, POST)

Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

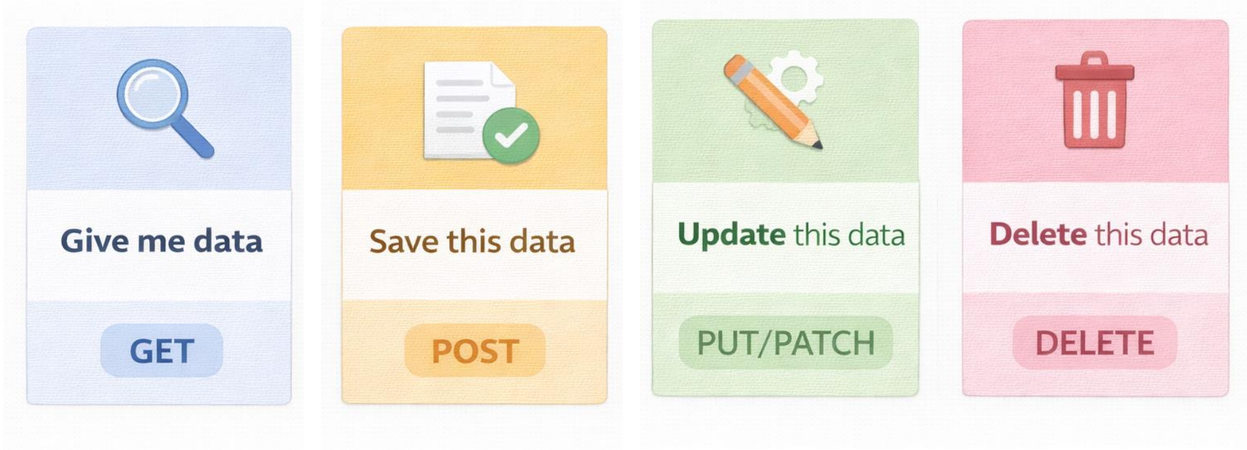
HTTP Methods



HTTP verbs

HTTP (HyperText Transfer Protocol) governs the actions between your browser (client) and the server

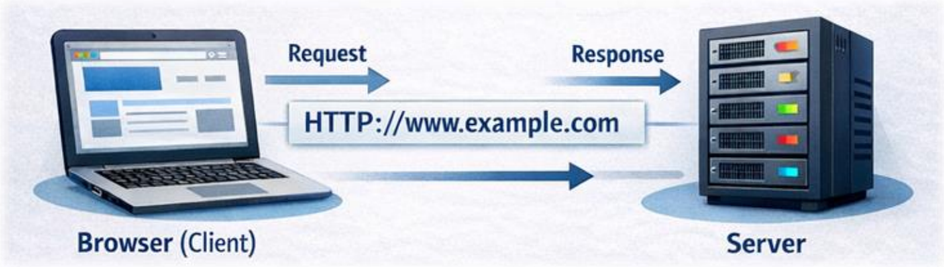
Essential in designing RESTful APIs and modern web applications



ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

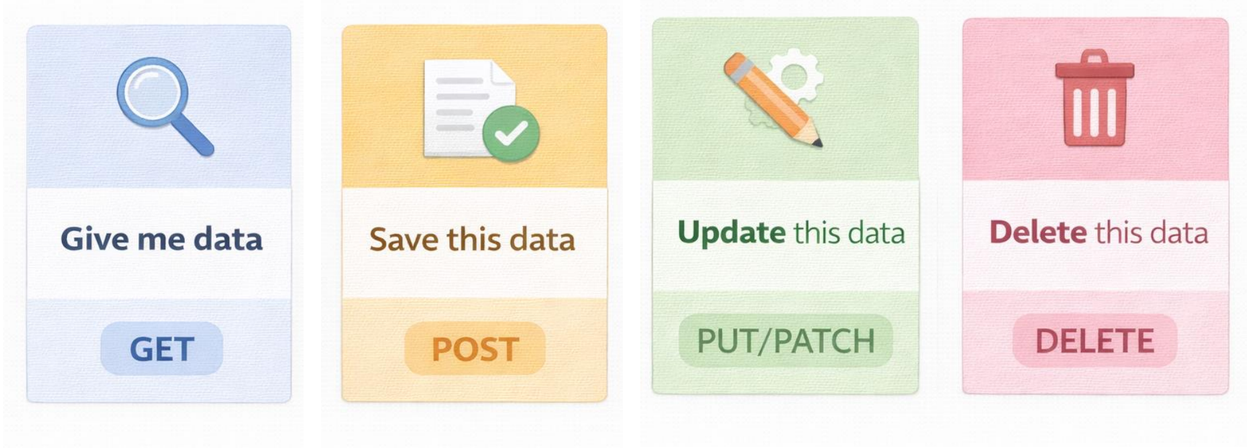
HTTP Methods



HTTP verbs

HTTP (HyperText Transfer Protocol) governs the actions between your browser (client) and the server

Essential in designing RESTful APIs and modern web applications



ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

HTTP Methods

GET

GET requests data from a server without changing it.
Data is sent in the URL as query parameters.

GET /products?id=1234&category=books

- **Safe and idempotent**
- E.g, fetching user profiles, product listings.



POST

POST sends data to the server **to create a new resource.**
Data is sent in the request body.

POST /user

- **Not idempotent**
- E.g, user registration, submitting forms.

```
{ " username": "john_doe",
  " password": "securepass" }
}...
```



PUT

PUT updates an existing resource **fully**, replacing it with new data.

PUT /user/1234

- **Idempotent**

E.g, replacing all user details.

```
{ " username": "john_doe",
  " email: "john@example.com" }
```



PATCH

PATCH partially updates a resource's fields.

PATCH /user/1234

- **Idempotent**

E.g. Updating just an email or password.

```
{ "email": "john.doe@example.com" }
```



DELETE

DELETE removes a resource from the server.

- **DELETE**: /user/1234

E.g. deleting accounts**, removing posts.**



ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

HTTP Methods

- **Why HTTP methods matters?**

Foundation of **REST APIs**



Improve **Clarity** & **Consistency**
In Backend Design



Help Frontend & Backend
Systems Communicate Efficiently



Enforce Best Practices
In Web Architecture

ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP



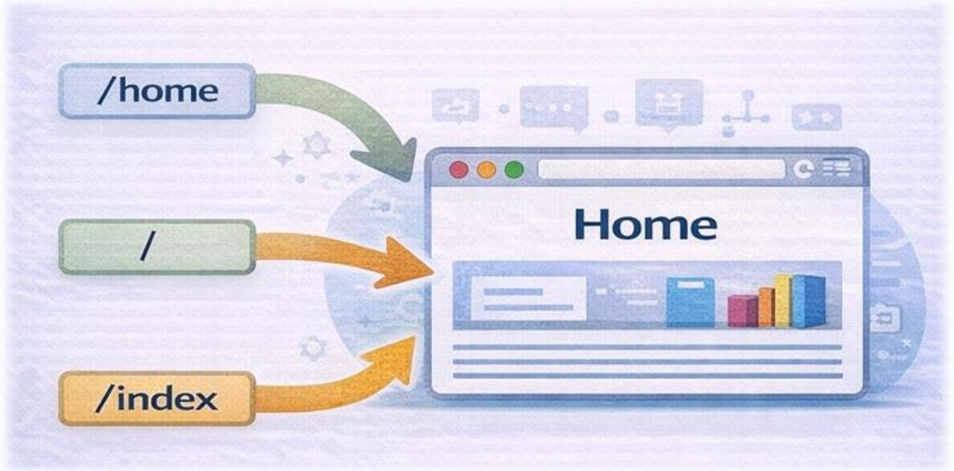
Demo | Handling Multiple Routes

Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

Handling Multiple Routes

Redirect multiple routes on the same page



ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Thank you



ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP



Request and Response Handling

Copyright © Thinknyx Technologies LLP



Request and Response Handling

Section Overview

- Introduction to **Request Object** in Flask
- Understanding **Form Data Handling** in Flask
- Working with **Query Parameters**
- Handling **JSON Data** in Flask Applications
- Implementing **Redirects** in Flask
- Creating **Custom Responses**

ThinkRook
By Thinknyx Technologies LLP



Demo | Request Object and Form Data Handling

Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

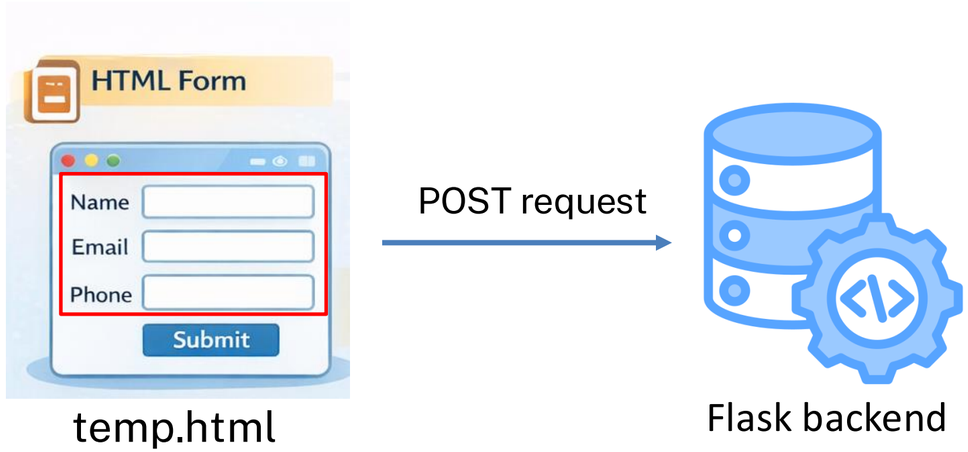
Request Object and Form Data Handling



Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

Request Object and Form Data Handling



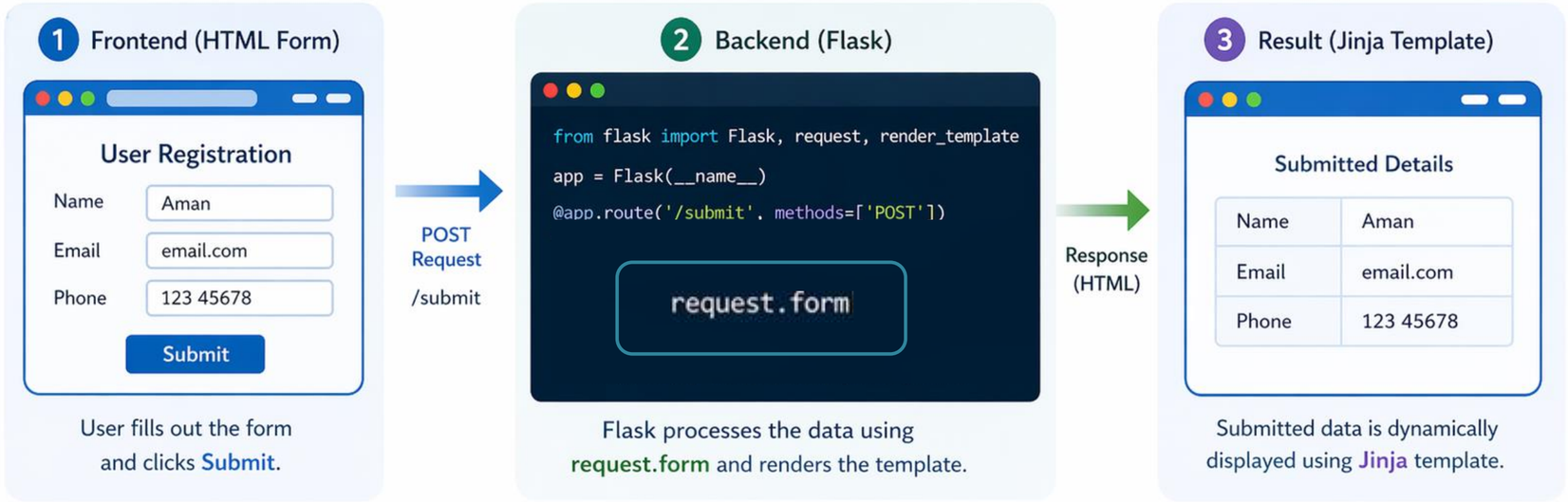
temp.html

- In **app.py**, Flask will capture this form data using **request.form** and passes it to another template named **result_data.html**
- Dynamically displayed in a table using **Jinja templating** using loops

ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Request Object and Form Data Handling



Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP



Demo | Query Parameters

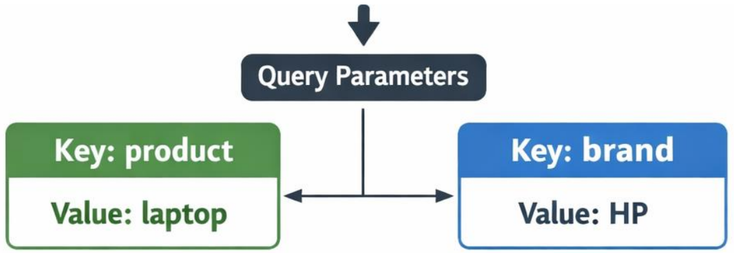
Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

Query Parameters



http://www.ecomm.com/search?product=laptop&brand=HP

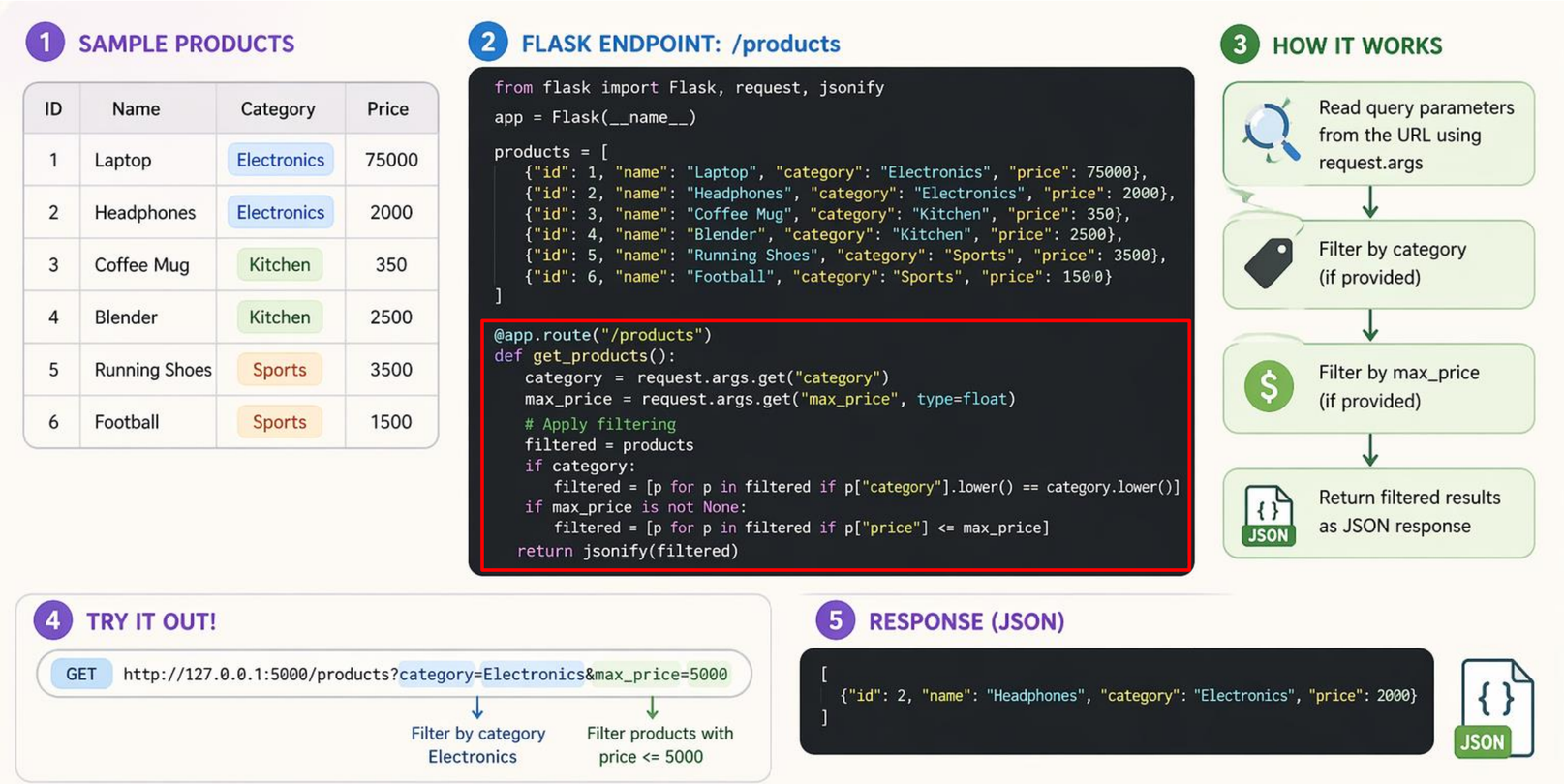


Flask provides the **request.args** object from request module to access query parameters

ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Query Parameters



Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP



Demo | JSON Data Handling

Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

JSON Data Handling

JSON: JavaScript Object Notation is a lightweight data format used to **store and exchange data** between systems especially between a server and a web application



ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

JSON Data Handling

Example

```
{
"name": "Alex",
"age": 25,
"isIntern": false,
"skills": ["Python", "JavaScript"]
}
```

Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

JSON Data Handling

1 User Input

User enters a message in the input box and clicks Send.

2 JavaScript (fetch)

JavaScript uses fetch() to send a POST request with JSON data to the /process endpoint.

3 Flask Backend

Flask receives the JSON data, processes the message (uppercase + metadata), and creates a JSON response.

4 JSON Response

Flask sends the JSON response back to the frontend.

5 Display Output

JavaScript receives, parses the response, and displays it on the webpage.



/process
Content-Type: application/json

```
{
  "message": "hello flask!"
}
```

```python
from flask import Flask, request, jsonify
from datetime import datetime

app = Flask(__name__)

@app.route('/process', methods=['POST'])
def process():
    data = request.get_json()
    message = data.get('message', '')
```

Response from Server:

```
{
  "status": "success",
  "original_message": "hello flask!",
  "processed_message": "HELLO FLASK!",
  "length": 11,
  "timestamp": "2024-05-20T12:30:45.123Z"
}
```



ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

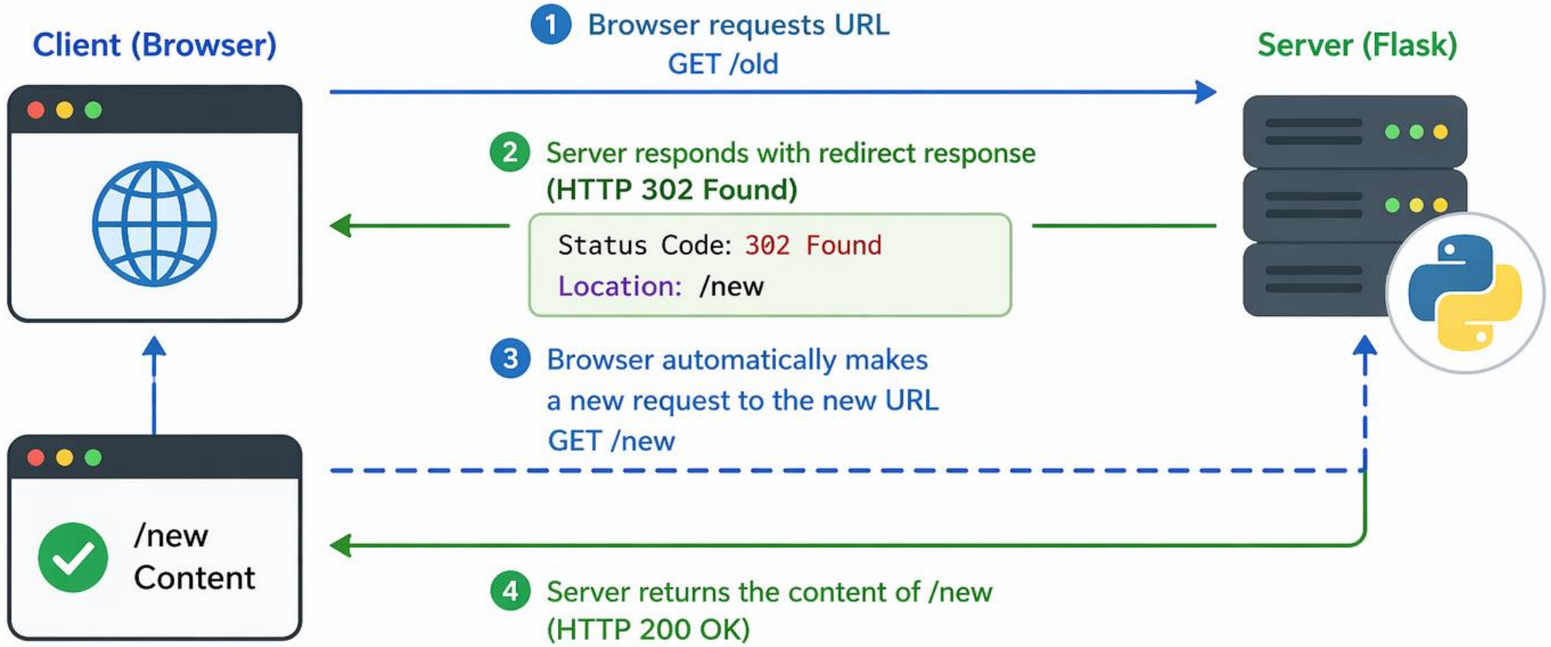


Demo | Redirects

Copyright © Thinknyx Technologies LLP

Redirects

A redirect is a mechanism that sends the client from one URL to another



ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Redirects

Syntax:

Route is passed in redirect function

```
return redirect('/login')
```

URLs: url_for() function to dynamically generate

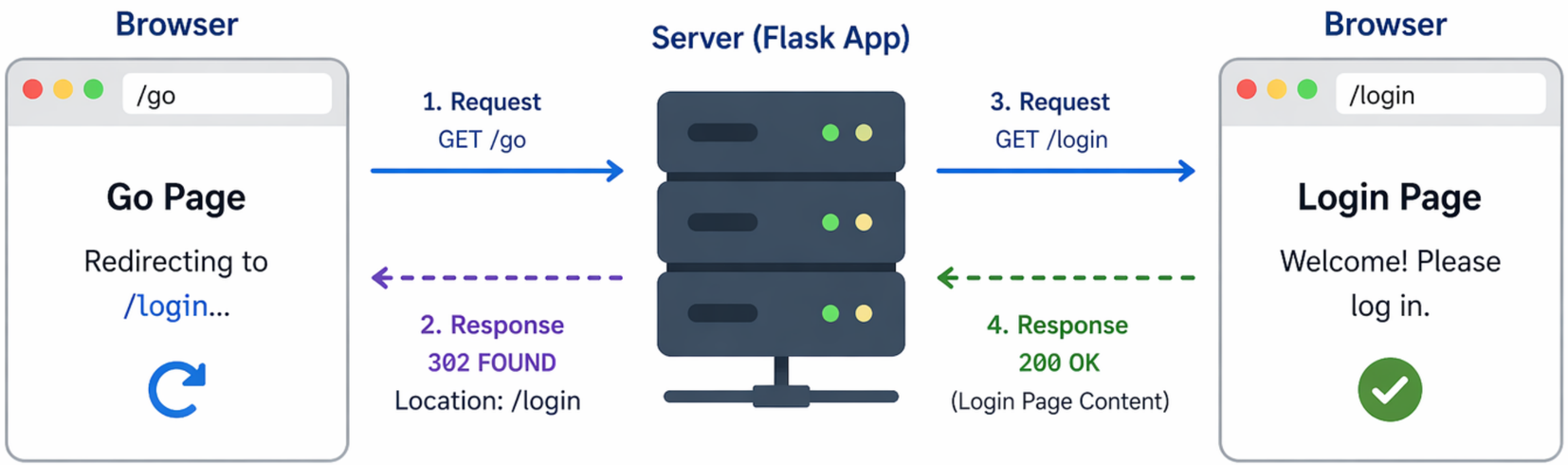
```
return redirect(url_for('login'))
```

This avoids breaking links if route names change and code becomes more maintainable and scalable.

ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Redirects



The server first responds with a 302 status code and the new URL.
The browser automatically makes a new request to that URL and displays the page.

ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP



Demo | Custom Responses

Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

Custom Responses

Manually constructing and controlling the HTTP response sent to the client

Instead of relying on Flask's default behavior

Default behavior includes returning a simple string or JSON automatically

- **By default:**

```
return "Hello World"
```

→

Status Code	200 (OK)
Content-Type	text/html
Body	"Hello World"

```
HTTP/1.1 200 OK
Content-Type: text/html

Hello World
```

- **Using custom responses:**

Response Body
The content you want to send to the client.

Status Code
Like 200, 404 or 500 to indicate the result of the request.

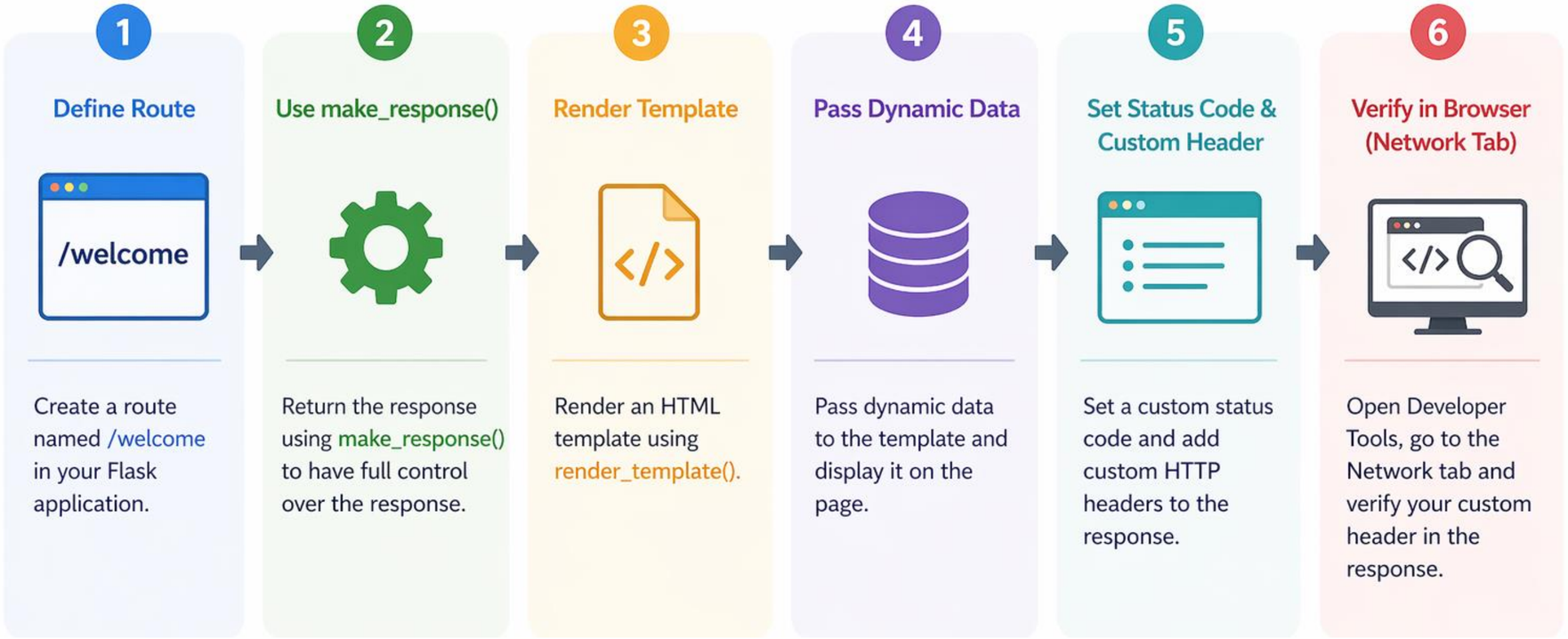
Headers
Metadata like content type, cookies, etc.

This gives you **full control** over how the server communicates with the client.

ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Custom Responses



ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Thank you



ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP



Demo | Use Case: Task Manager API

Copyright © Thinknyx Technologies LLP

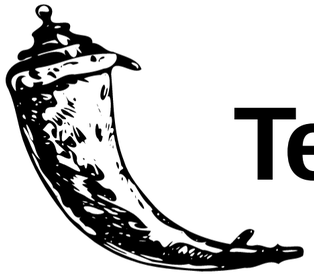
ThinkRook
By Thinknyx Technologies LLP

Thank you



ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP



Templates (jinja2)

Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

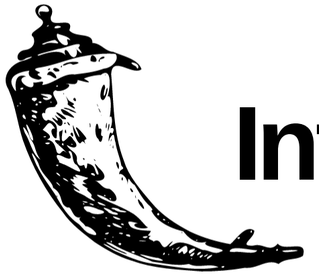


Templates (jinja2)

Section Overview

- Introduction to Jinja Templates
- Understanding Template Rendering and Inheritance
- Using Built-in Jinja Filters
- Control Structures in Jinja - Loops and Conditional Statements

ThinkRook
By Thinknyx Technologies LLP



Introduction to Jinja2

Copyright © Thinknyx Technologies LLP

Introduction to Jinja2



Template Rendering



Jinja

- Default templating engine

ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Introduction to Jinja2



Jinja

Jinja2 version (Version 2)

Modern and designer-friendly templating engine for Python

- Jinja2 allows embedding Python-like expressions directly into HTML files
- Flask integrates Jinja2 by default
- Dynamic content can be rendered without any additional setup

ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

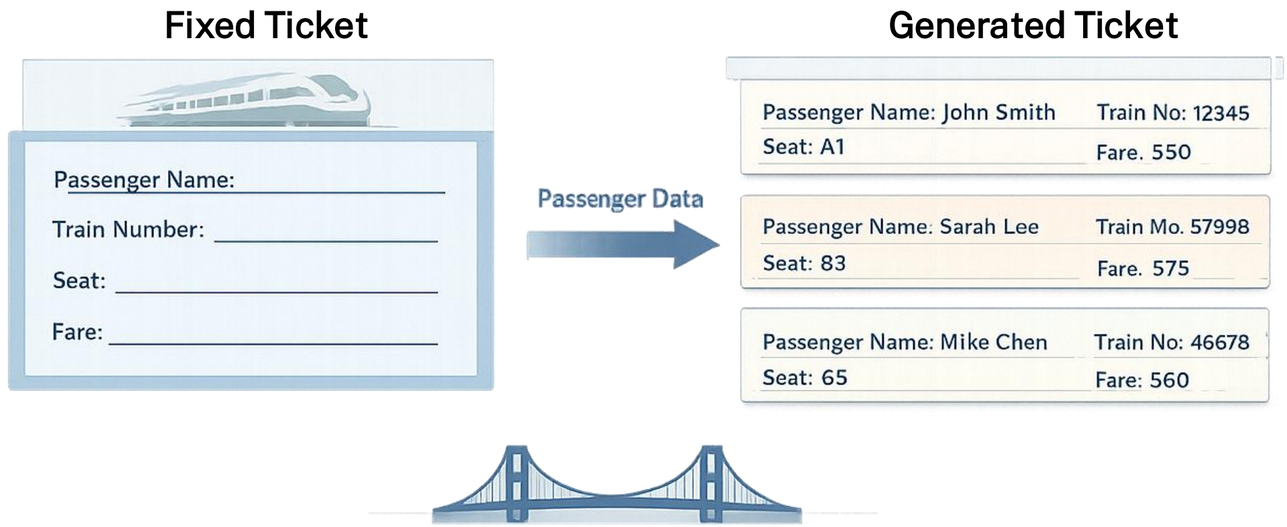
Introduction to Jinja2

- Jinja (Jinja2) is a modern and designer-friendly templating engine for Python
- The “2” in Jinja2 refers to its major version (Version 2)
- Version 2 introduced significant improvements over the original Jinja
- Jinja2 allows embedding Python-like expressions directly into HTML files
- Flask integrates Jinja2 by default
- Dynamic content can be rendered without any additional setup

Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

Introduction to Jinja2



Jinja Bridge Static Template & Dynamic Data



ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Why Jinja2?

Manually construct HTML strings inside your Python code

Messy

Unmaintainable

- Separate business logic from presentation
- Reuse HTML components (like headers and footers)
- Dynamically display data
- Keep your code clean and scalable

ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Why jinja2?

Inside your HTML file, you can access variables using **{{ }}** and pass name variable to **render_template** function

app.py

```
@app.route('/’)
def home():
emp_name = "Kanishk"
return render_template('index.html', name=emp_name)
```

html

```
<h2>Hello, {{ name }}</h2>
```



- Core part of Flask development
- Build dynamic, Maintainable & Scalable web applications
- Separate backend logic from frontend design

ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Why jinja2?

if condition

for loop

Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

If Condition

if condition We check a condition and move forward depending on the condition outcome

app.py

```
return render_template("index.html", is_logged_in=True, username="Kanishk")
```

html

```
{% if is_logged_in %}
<h2>Welcome, {{ username }}</h2>
{% else %}
<h2>Please log in</h2>
{% endif %}
```

ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

For Loop

for loop — We iterate over items to print them

app.py

```
food_items = ["Pizza", "Burger", "Pasta"]
return render_template("menu.html", items=food_items)
```

html

```
<ul>
  {% for item in items %}
    <li>{{ item }}</li>
  {% endfor %}
</ul>
```

ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP



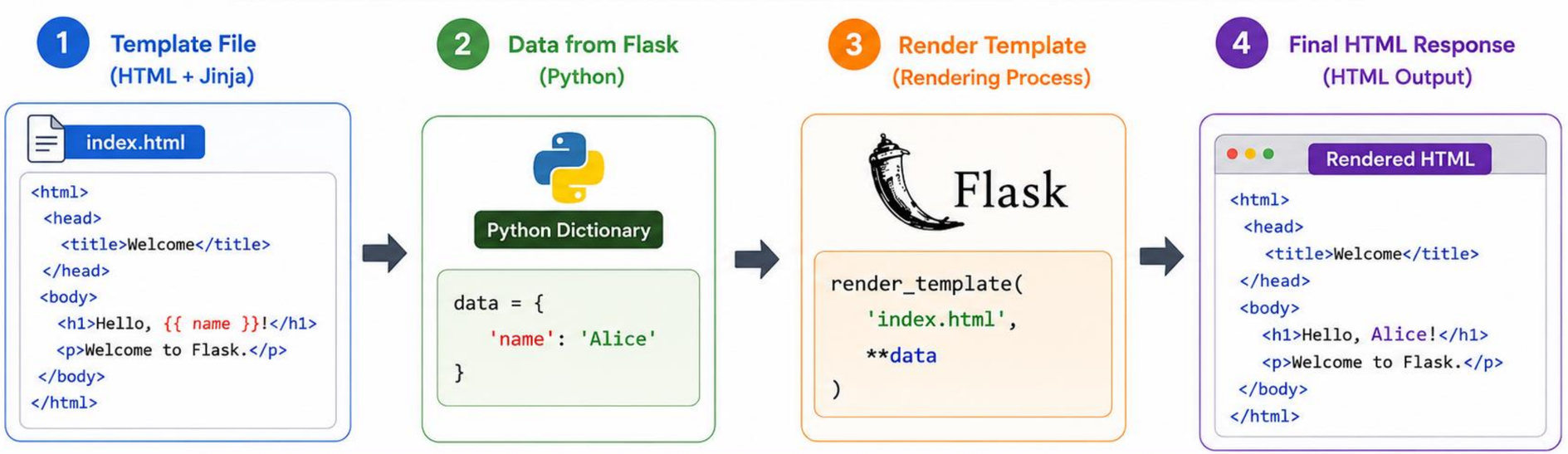
Demo | Template Rendering and Template Inheritance

ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Template Rendering and Template Inheritance

➢ Template rendering



render_template()

Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

Template Rendering and Template Inheritance

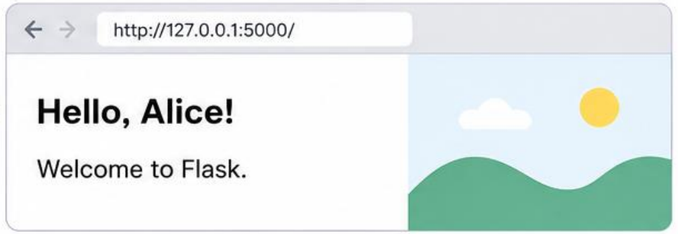
➢ Template rendering

Template rendering is the process of taking a template file that is HTML + Jinja syntax, injecting dynamic data from Flask that is Python, and producing a final HTML response




```python
from flask import Flask, render_template
app = Flask(__name__)

@app.route('/')
def home():
    return render_template('index.html', name='Alice')
```

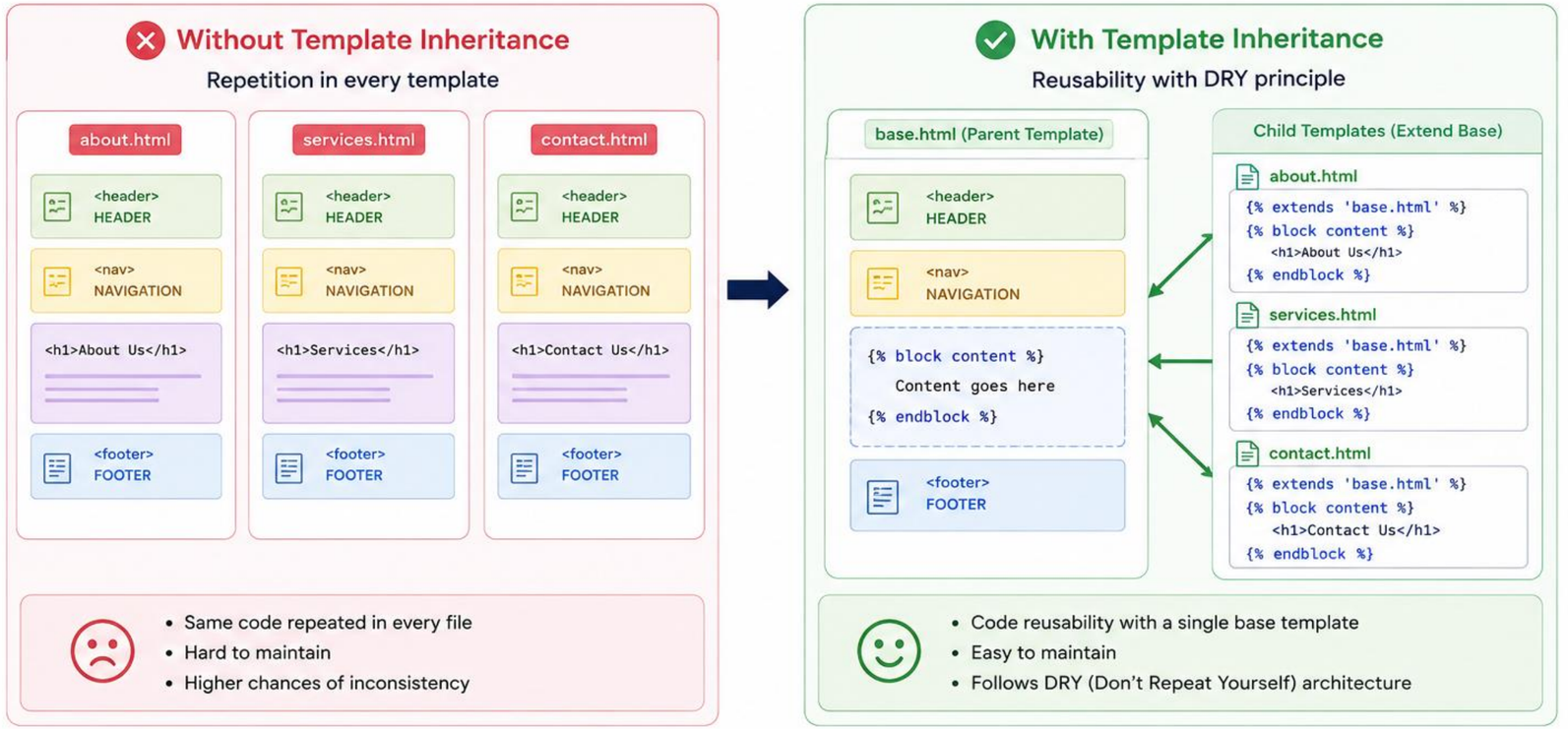


ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Template Rendering and Template Inheritance

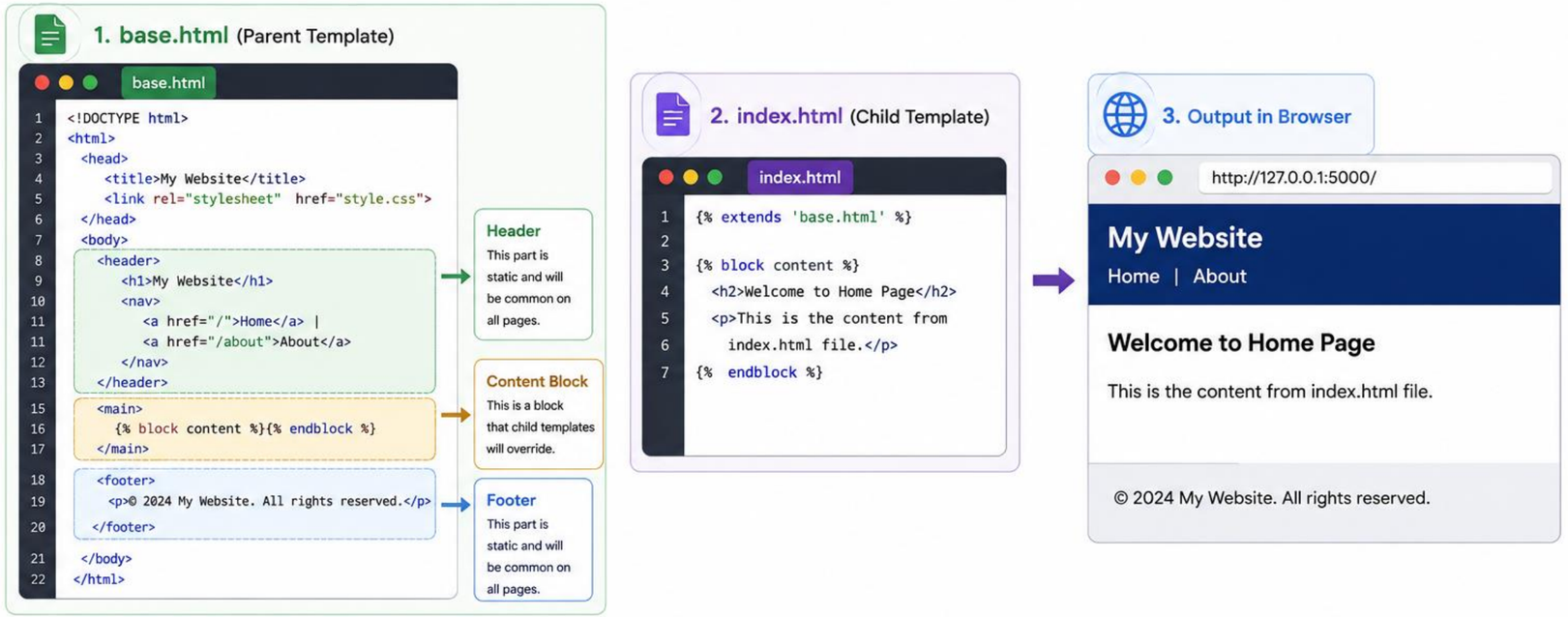
➢ Template inheritance



ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Template Rendering and Template Inheritance



Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP



Demo | Built-in Jinja filters

Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

Built-in Jinja filters

upper

upper function is used to convert every alphabet to upper case letter

length

length is used to find the length

Copyright © Thinknyx Technologies LLP

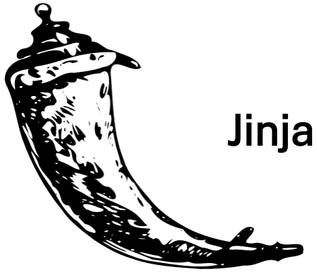
ThinkRook
By Thinknyx Technologies LLP



Control Structures in Flask

Copyright © Thinknyx Technologies LLP

Control Structures in Flask



Jinja acts as the templating engine that bridges your backend logic and frontend presentation



Jinja allows to introduce dynamic behavior using control structures, primarily loops and conditionals

Intelligent

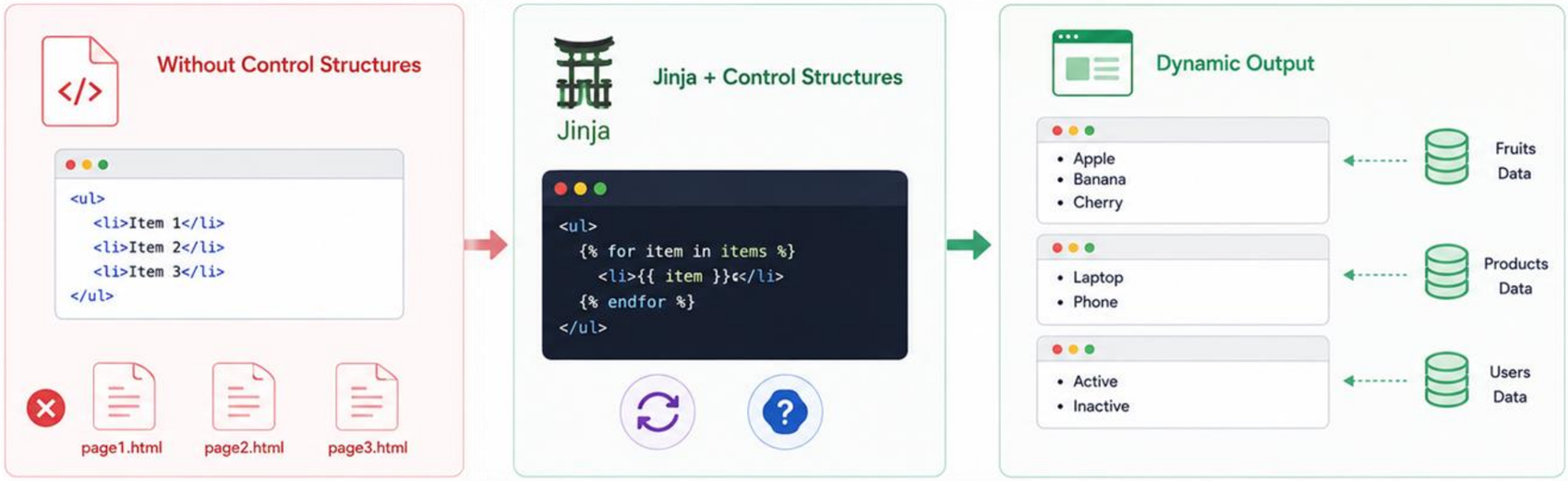
Reusable

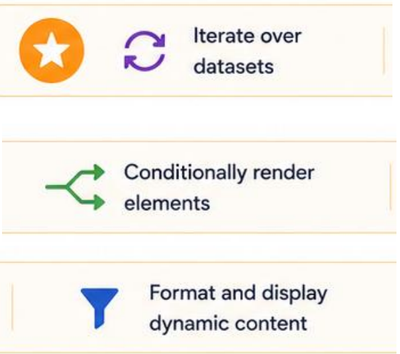
Data-Driven

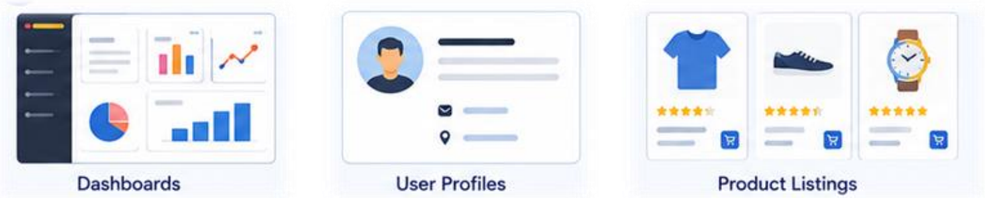
ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Why does it matter?







ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Why does it matter?

➢ Conditions

Conditionals allow you to render content based on specific conditions
similar to **if-else** in Python

- **Syntax**

```
{% if condition %}
    <!-- content -->
{% endif %}
```

- **Example**

```
 {% if is_logged_in %}
<h2>Welcome, {{ username }}</h2>
{% else %}
<h2>Please log in</h2>
{% endif %}
```

elif

```
{% if user.role == "admin" %}
<p>Admin Panel Access</p>
{% elif user.role == "editor" %}
<p>Editor Dashboard</p>
{% else %}
<p>Viewer Access</p>
{% endif %}
```

- Jinja conditionals follow Python-like logic but are enclosed within **{% %}** blocks
- Output variables still use double curly braces **{{ }}**

ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Why does it matter?

➢ Loops

Loops are used to iterate over collections like lists, dictionaries, or query results

- **Syntax**

```
{% for item in items %}
<!-- content -->
{% endfor %}
```

- **Display our data in a table**

```
<ul>
{% for product in products %}
<li>{{ product.name }} - ₹{{
product.price }}</li>
{% endfor %}
</ul>
```

Output

Laptop - ₹50000

Chair - ₹12000

Smart Phone - ₹25000

Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

Control Structures in Flask

Loop metadata Jinja provides a special loop object inside loops

- **Example**

```
{% for user in users %}
<p>{{ loop.index }}. {{ user.name
}}</p>
{% endfor %}
```

Output

1. Alex
2. Nathen
3. James

loop.index0

loop.first

loop.last



ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP



Demo | Control Structures (Loops & Conditionals)

Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

Thank you



ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP



Forms Handling

Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP



Forms Handling

Section Overview

- Understanding HTML Forms in Flask
- Preventing CSRF
- Web Forms with Flask-WTF Extension

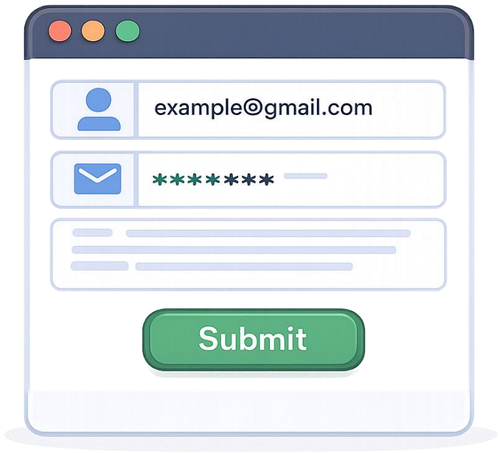
ThinkRook
By Thinknyx Technologies LLP



HTML Forms in Flask

Copyright © Thinknyx Technologies LLP

HTML Forms in Flask

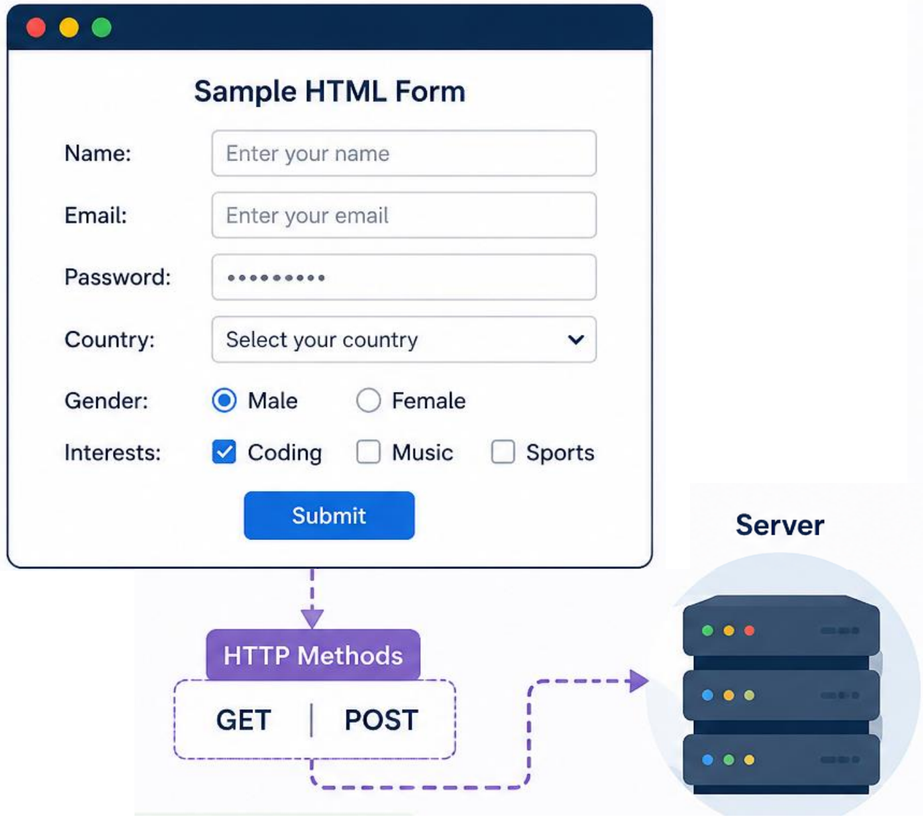


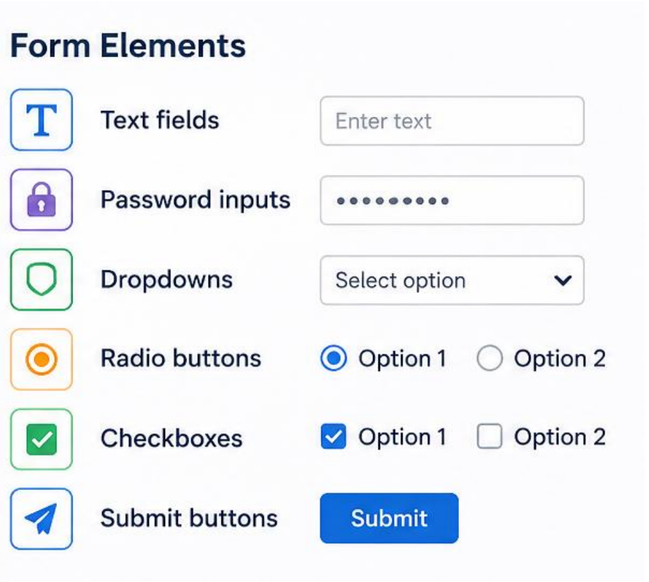
- Fundamental mechanism for building interactive web applications
- Act as the bridge between the client (browser) and the server (Flask application)
- Allow users to send data
- The backend can process, validate, and store the data

ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

HTML Forms in Flask






```
<form action="/submit" method="POST">
    <input type="text" name="username">
    <input type="submit">
</form>
```

The form sends this data to a server using **HTTP methods**.

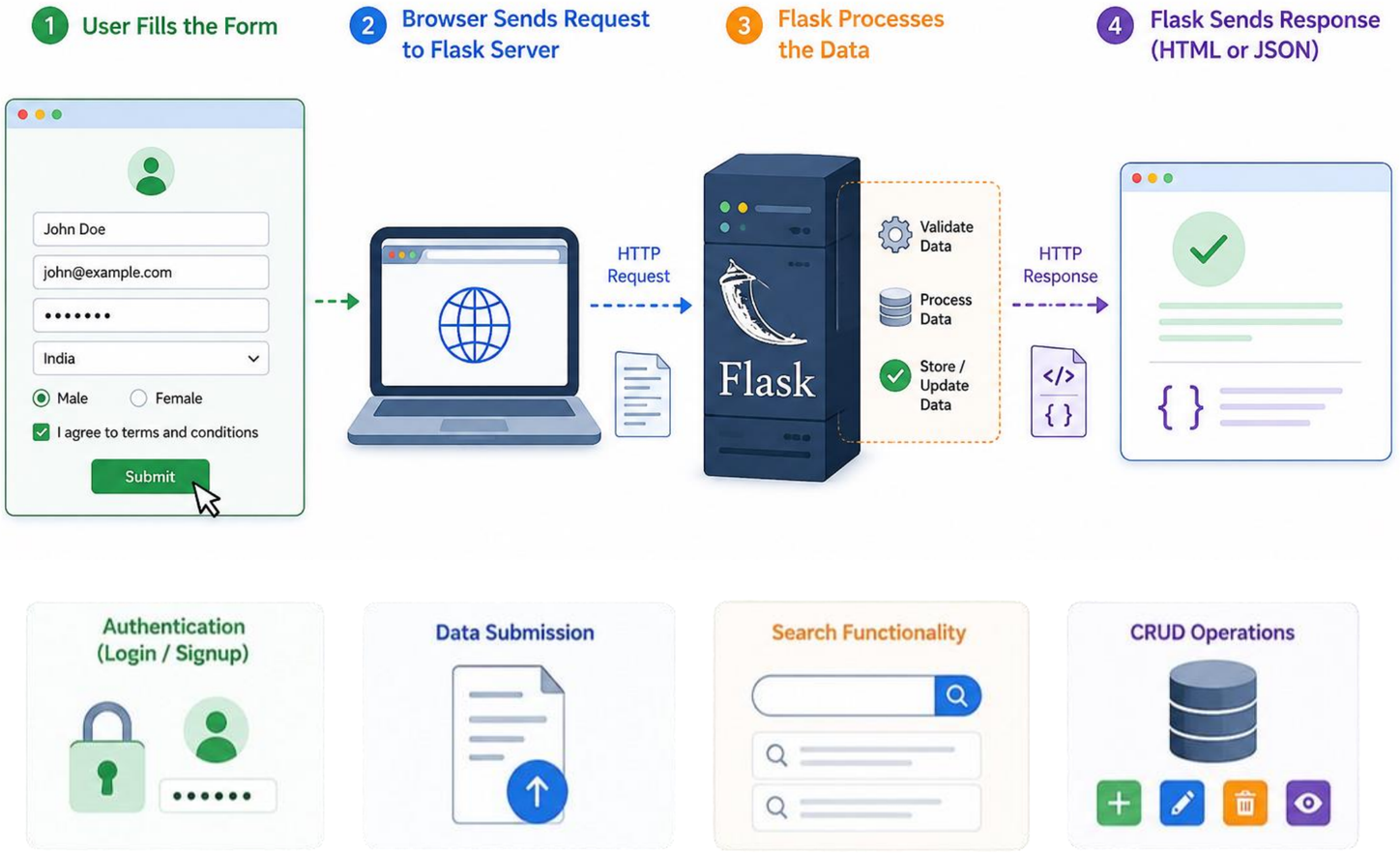
Forms follow the standard request-response cycle.

ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

HTML Forms in Flask

Forms follow the standard **request-response** cycle



Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

HTML Forms in Flask

Flask provides a request object to access incoming form data

- **Import request**

```
from flask import request
```

- **Access the data**

```
username = request.form.get("username")
```

ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP



Demo | HTML Forms in Flask

Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

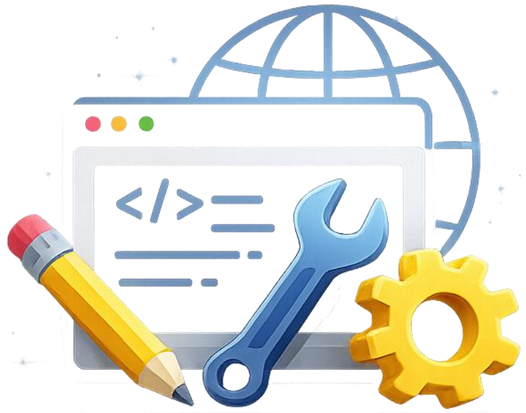


CSRF Protection

Copyright © Thinknyx Technologies LLP

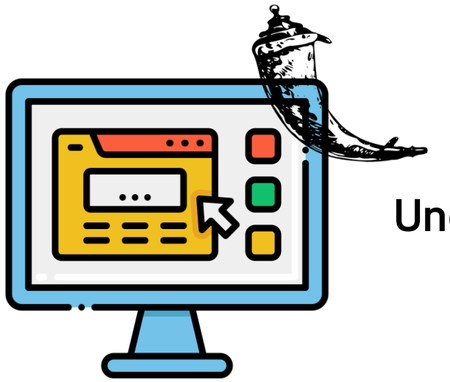
ThinkRook
By Thinknyx Technologies LLP

CSRF Protection



- Security is not optional
- It’s foundational

CSRF
(Cross-Site Request Forgery)



Understanding and **implementing**
CSRF protection is critical

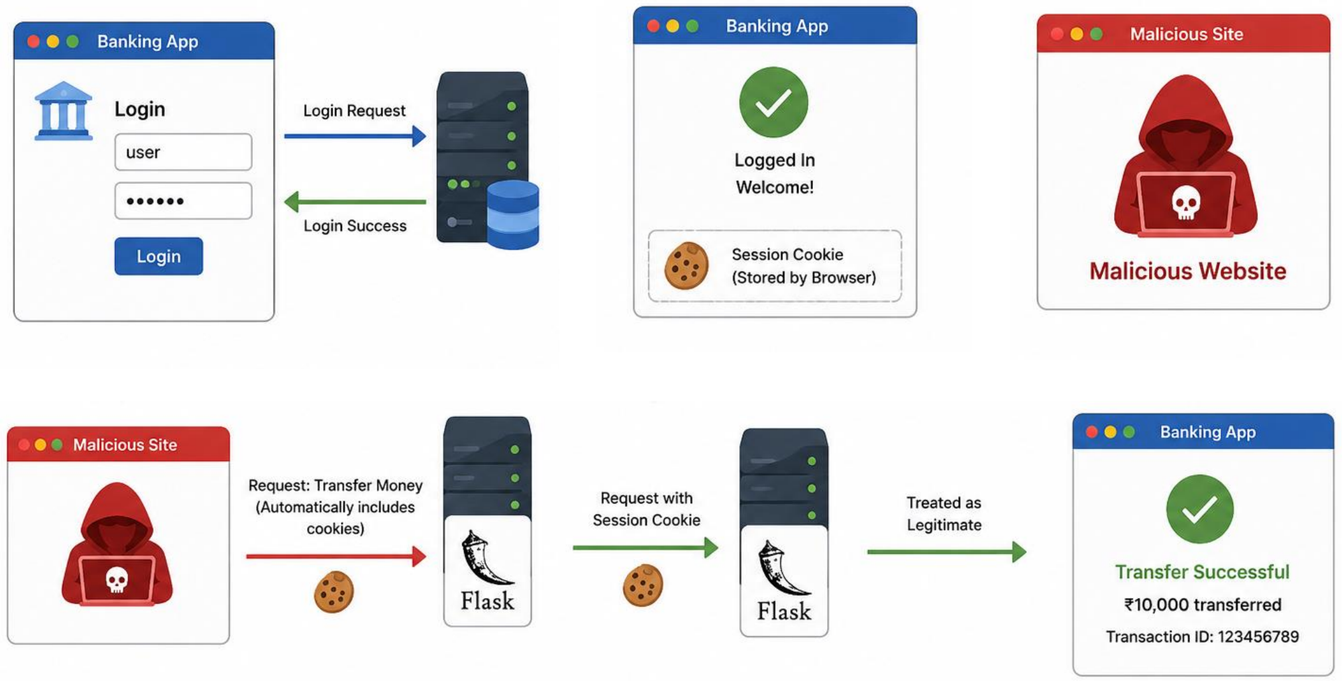
- ✓ Dealing with Forms
- ✓ Authentication
- ✓ User-specific actions

ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

CSRF Protection

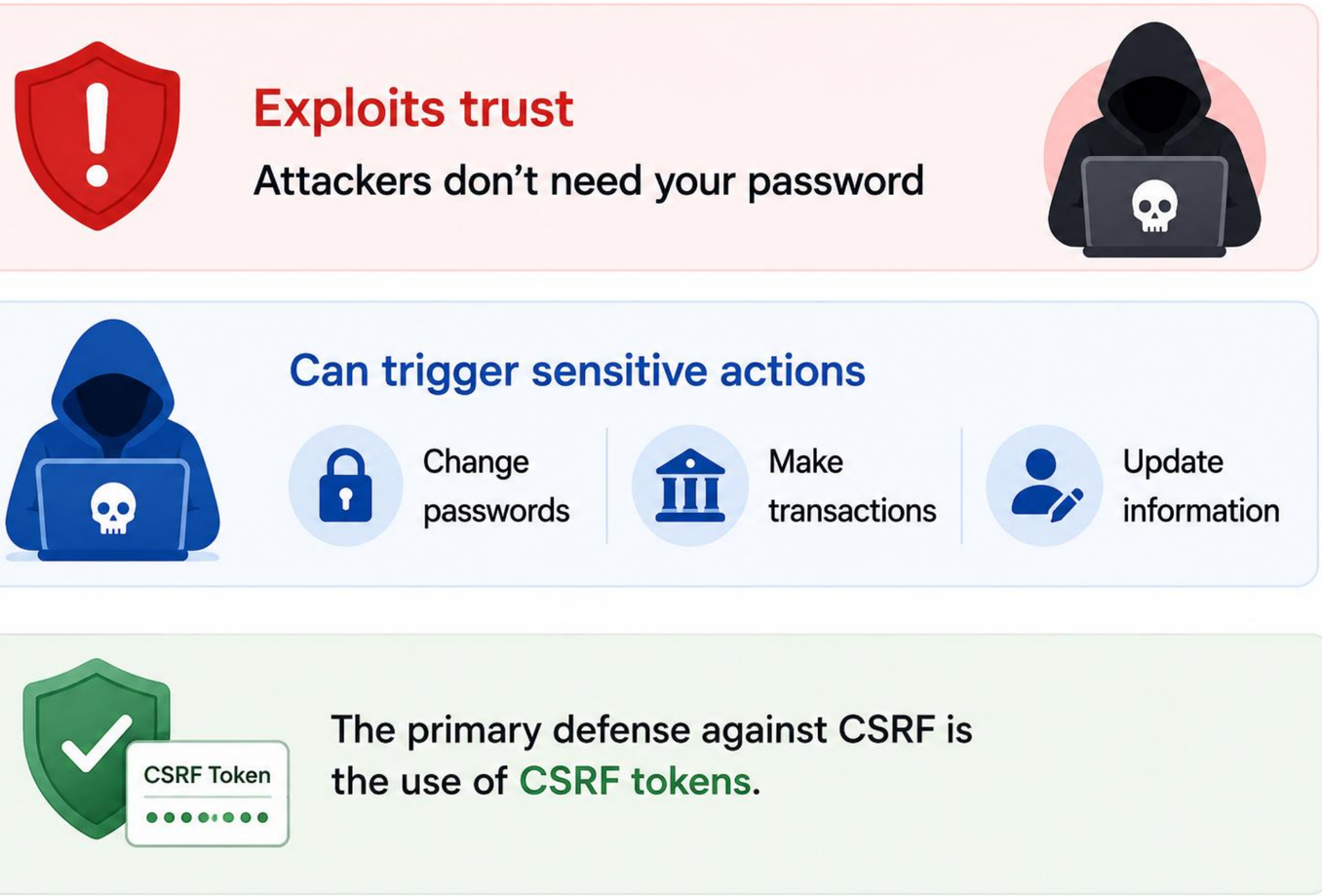
CSRF is an attack where a malicious website tricks a user's browser into sending unauthorized requests to another website where the user is already authenticated



Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

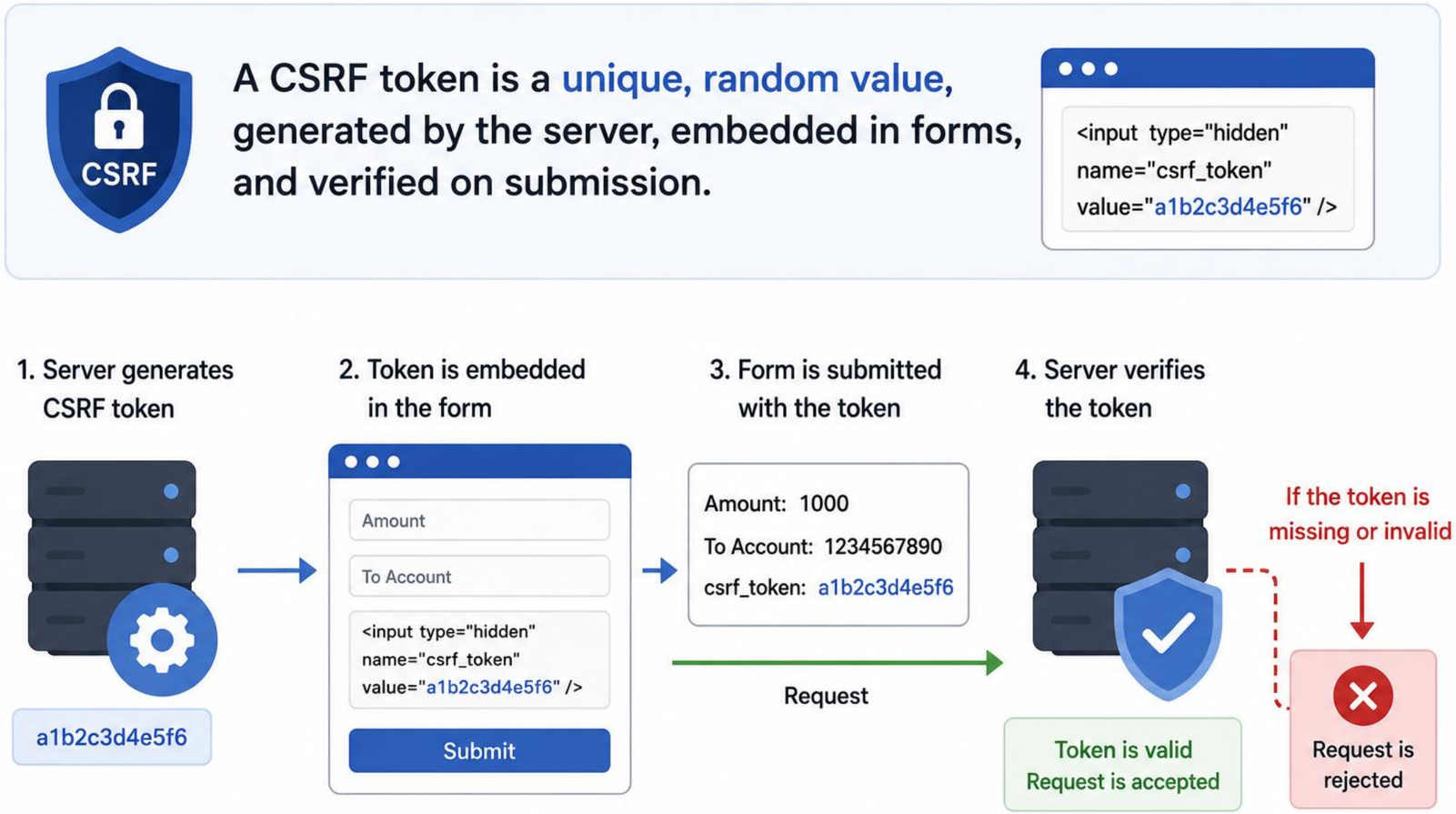
CSRF Protection



ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

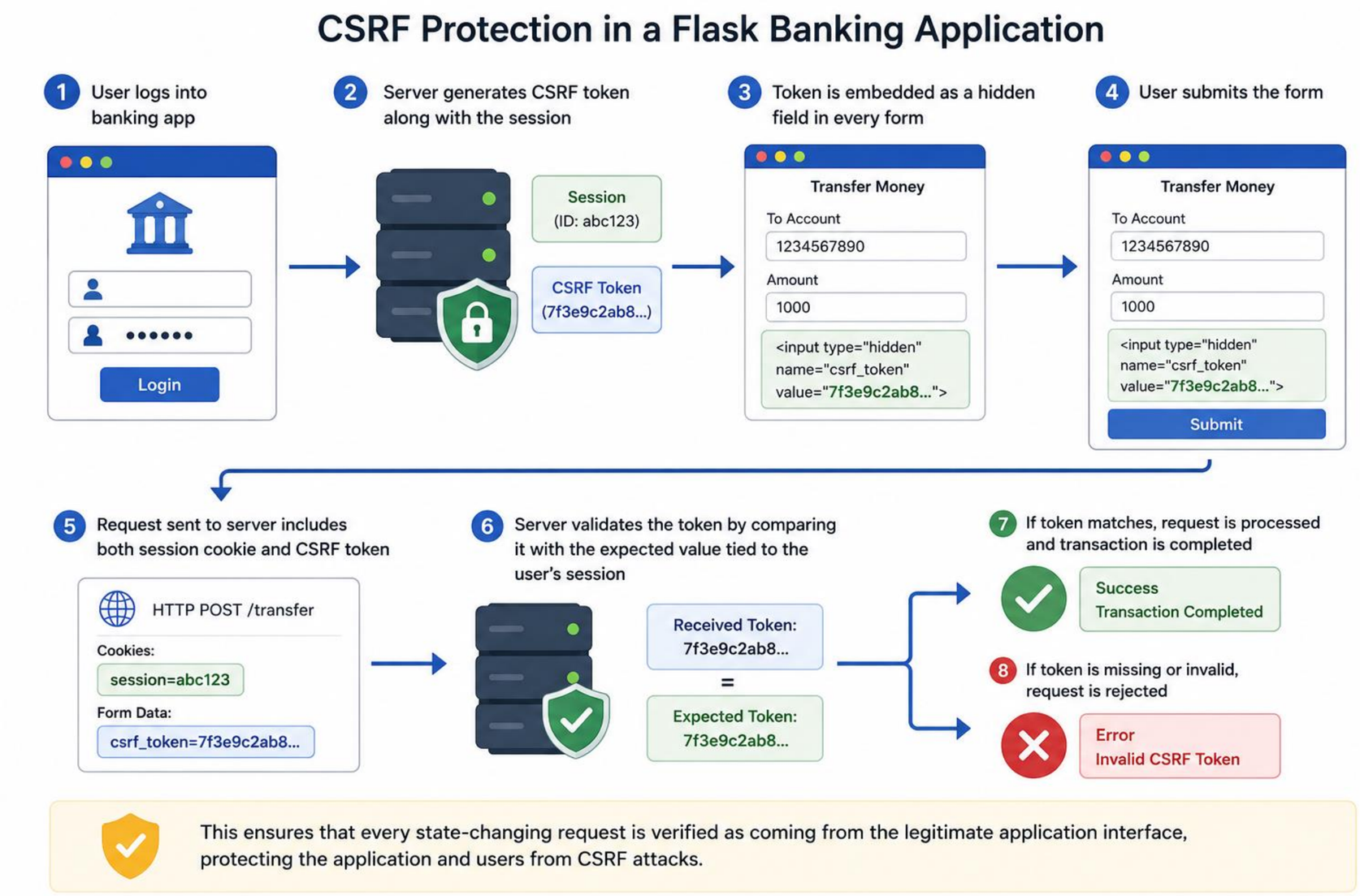
What is CSRF token?



Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

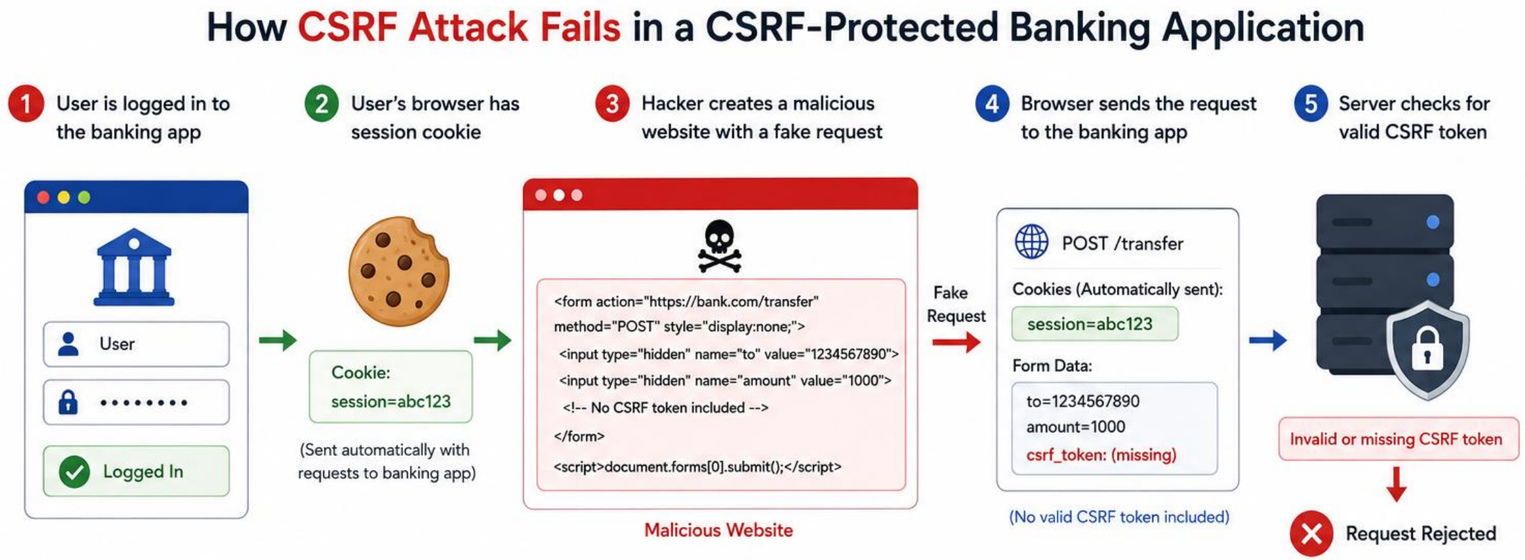
What is CSRF token?



Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

What is CSRF token?



This is why CSRF tokens are effective—they add an **extra layer of verification** that attackers cannot replicate, even if they can trick the browser into sending requests.

Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

Flask-WTF

- Flask does not provide built-in CSRF protection by default

flask-wtf

- Secret key
- Call CSRFProtect function

- **app.py**

```
app.config['SECRET_KEY'] = 'your_secret_key_here'
csrf = CSRFProtect(app)
```

Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP



Demo | Web Forms with Flask-WTF Extension

ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Web Forms with Flask-WTF Extension



- Handling Forms **securely** and **efficiently** is a critical requirement

Flask-WTF Extension

Creation | Validation | Protection

WTForms

Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

Web Forms with Flask-WTF Extension

Flask-WTF

Provides seamless integration of WTForms with Flask



Without Flask-WTF

You manually parse form data using request.form, write validation logic, and implement CSRF protection.

- Manual Parsing
- Manual Validation
- Manual CSRF Protection

```
from flask import request
from flask import render_template

@app.route('/submit', methods=['POST'])
def submit():
    name = request.form.get('name')
    email = request.form.get('email')
    # Manual validation
    if not name or not email:
        return "Invalid input"
    # Manual CSRF check (if implemented)
    # Process the data
    return render_template('success.html')
```

VS

With Flask-WTF

You have cleaner and structured code, built-in validators, automatic CSRF protection, and better scalability for large application.

```
from flask import render_template
from flask_wtf import FlaskForm
from wtforms import StringField, EmailField, SubmitField
from wtforms.validators import DataRequired, Email

class MyForm(FlaskForm):
    name = StringField('Name', validators=[DataRequired()])
    email = EmailField('Email', validators=[DataRequired(), Email()])
    submit = SubmitField('Submit')

@app.route('/submit', methods=['GET', 'POST'])
def submit():
    form = MyForm()
    if form.validate_on_submit():
        # Process the data
        return render_template('success.html')
    return render_template('form.html', form=form)
```

- Cleaner Code
- Built-in Validators
- Automatic CSRF Protection
- Better Scalability

ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Thank you



ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP



Database Integration

Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

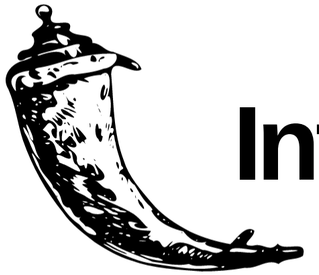


Database Integration

Section Overview

- Introduction to Databases
- Understanding SQLAlchemy
- Integrating SQLite with Flask
- Performing CRUD Operations
- Migrating Databases Using Flask-Migrate

ThinkRook
By Thinknyx Technologies LLP



Introduction to Databases

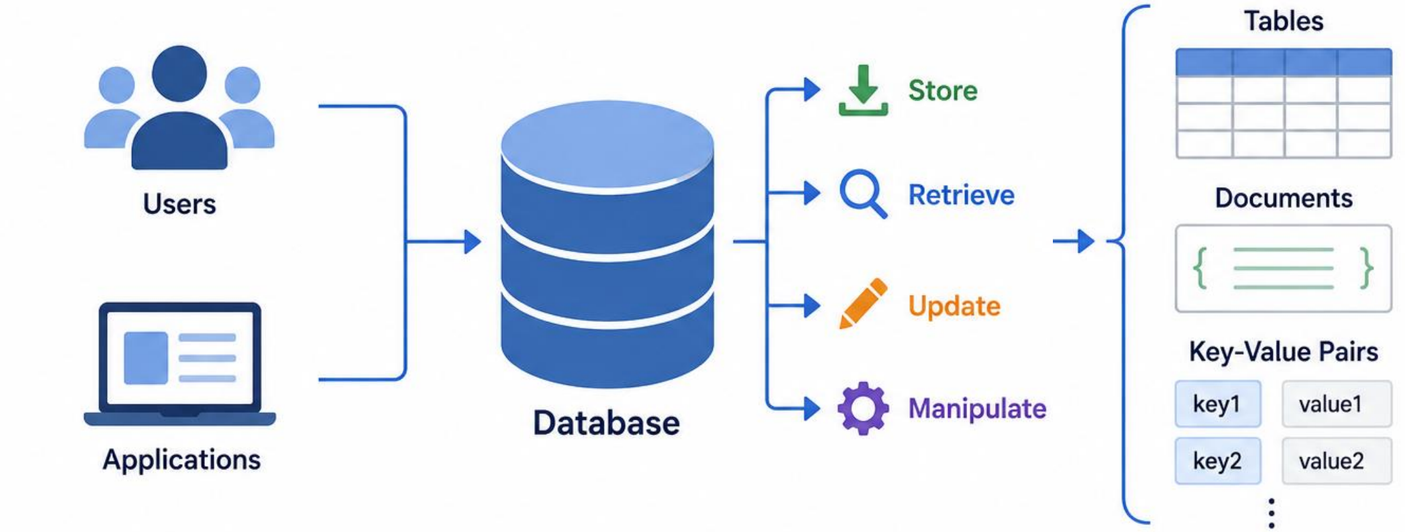
Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

Introduction to Databases



A database is a collection of data that is **stored**, **managed**, and **accessed** in an organized way



Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

Introduction to Databases

Before databases, data was stored in **flat files**

➢ Limitations

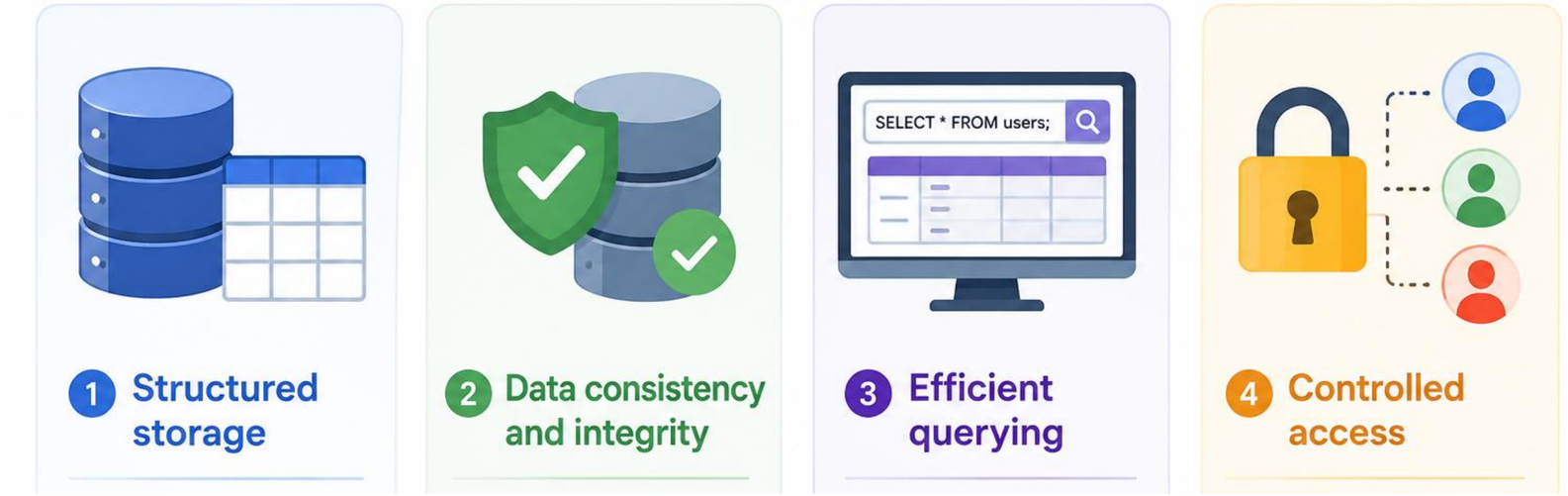
- Data redundancy (Same data repeated multiple times)
- Inconsistency (Different versions of the same data)
- Difficulty in searching and updating
- Poor security and access control

ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Introduction to Databases

Databases **solve these problems**



Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

Introduction to Databases

➢ Key Components in Database

1 Data

It is the actual information stored, such as **User details, Transactions, or Logs.**

User details

User ID	Name	Email
101	Alice	alice@example.com
102	Bob	bob@example.com
103	Charlie	charlie@example.com

Transactions

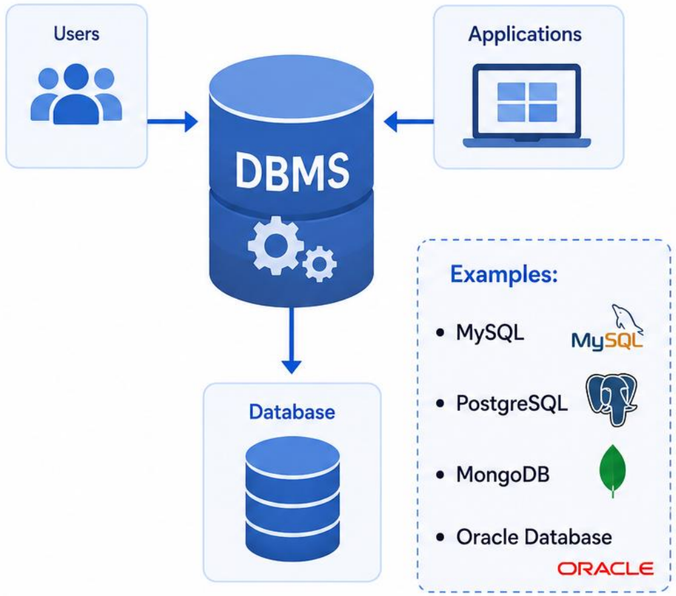
Txn ID	User ID	Amount	Status
T001	101	$250.00	Success
T002	102	$120.00	Success
T003	103	$80.00	Pending

Logs

Log ID	Timestamp	Event
L001	2025-05-18 10:15:23	User Login
L002	2025-05-18 10:16:45	Payment Success
L003	2025-05-18 10:17:10	Password Changed

2 DBMS (Database Management System)

It is a software that interacts with users and applications to **manage the database.**



Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

Introduction to Databases

- **Key Components in Database**

Relational Databases

Non-relational Databases

Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

Introduction to Databases

➢ Key Components in Database

Relational Databases



In Relational Databases, data is stored in tables that are called relations.



It uses Structured Query Language also known as SQL.



It maintains relationships using keys.

Example System

MySQL

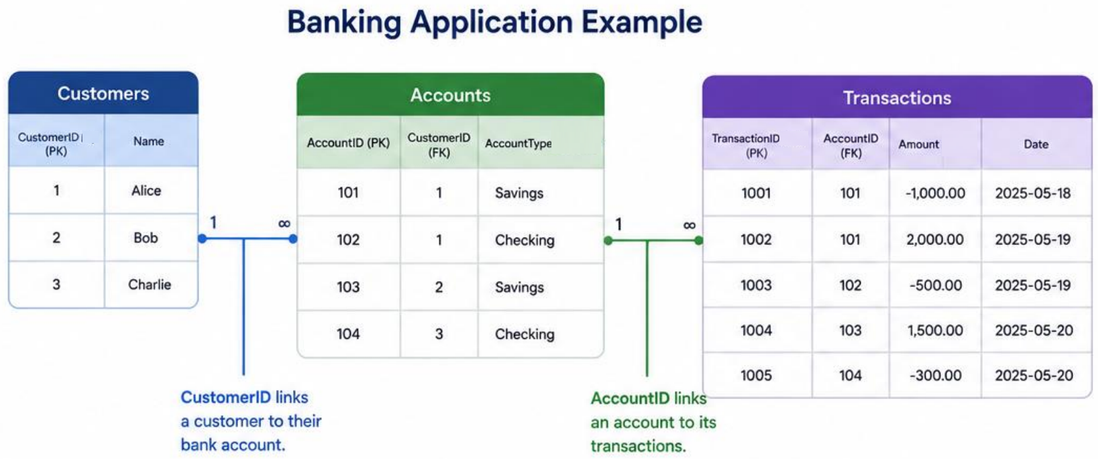
ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Introduction to Databases

➢ Key Components in Database

Relational Databases







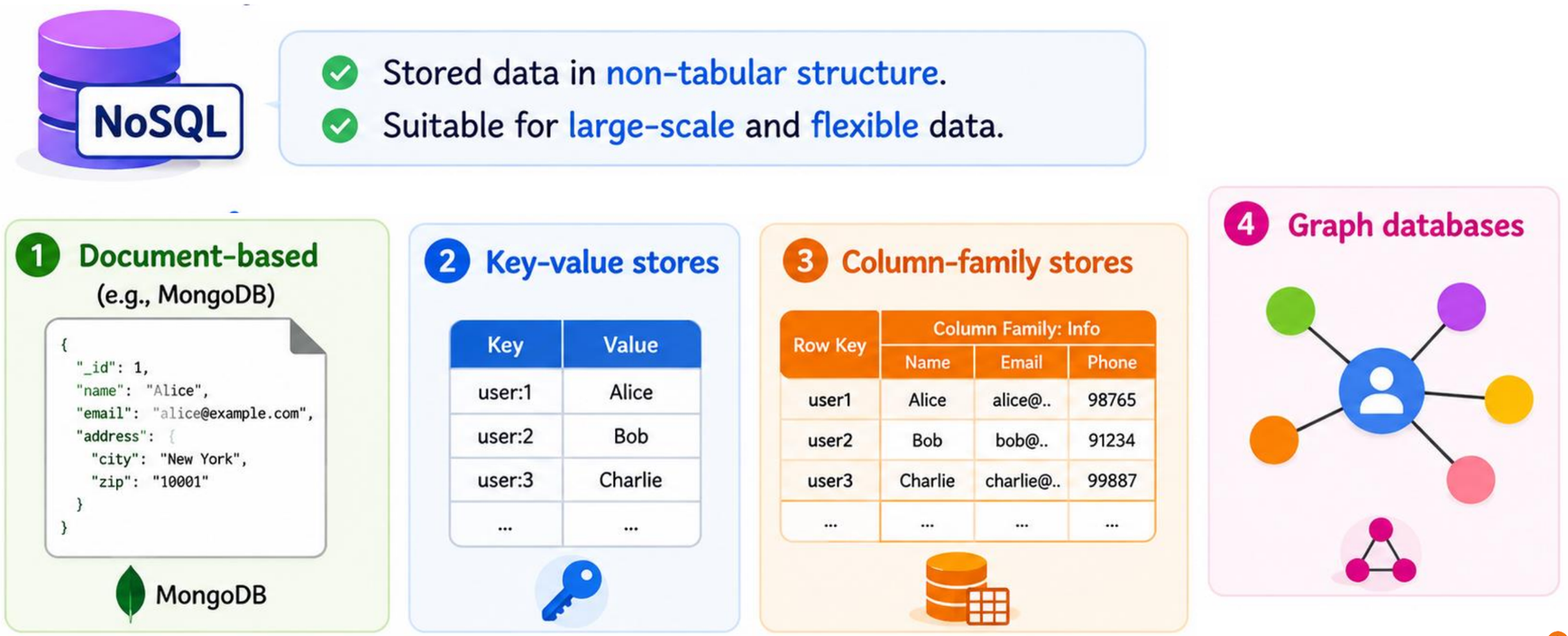
Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

Introduction to Databases

➢ Key Components in Database

Non-relational Databases



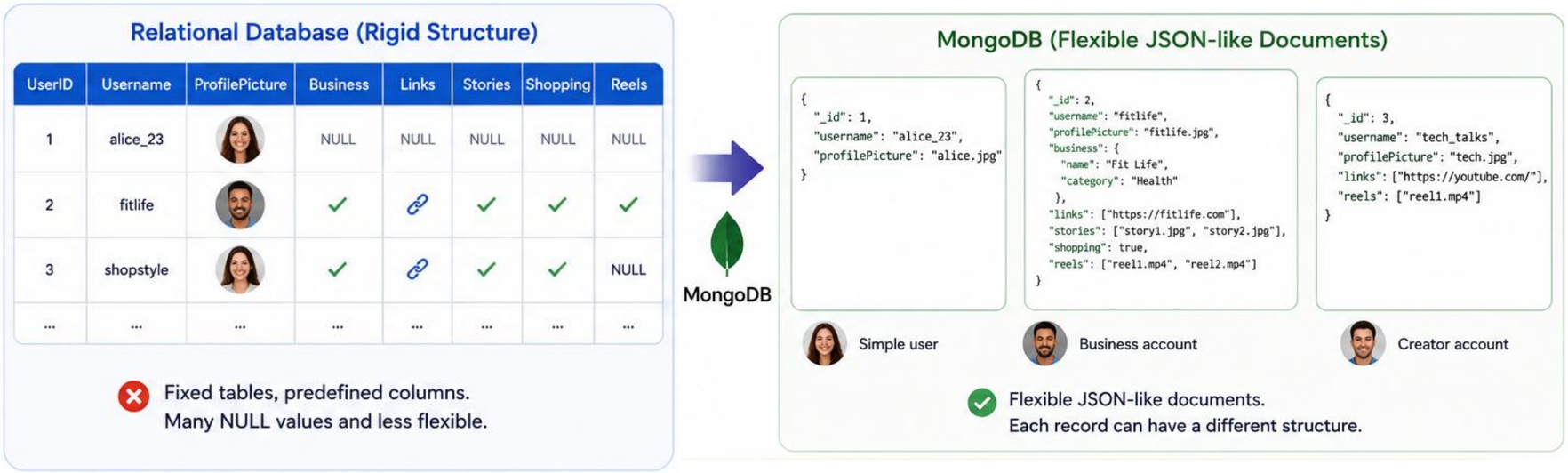
ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Introduction to Databases

➢ Key Components in Database

Non-relational Databases



Social Media Platforms

Chat Applications

IoT Systems

AI-based Applications

Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

Introduction to Databases

Relational Databases

1 SCHEMA – The Blueprint

A schema is the blueprint or design of a database. It defines how data is stored, how tables relate to each other, and the rules they follow.



2 TABLES – Represent Entities

In a relational database, data is organized into tables.
Each table represents a specific entity.



3 COLUMNS – Define Data Types

Columns define the type of data we are storing in a table.

Users Table			
id (Integer)	name (String)	email (String)	password (String)

4 ROWS (RECORDS) – Individual Entries

Each row (or record) represents one complete entry.
One row in the Users table represents one user.

Users Table			
id	name	email	password
1	Alice	alice@mail.com	hashed_pw1
2	Bob	bob@mail.com	hashed_pw2
3	Charlie	charlie@mail.com	hashed_pw3

5 PRIMARY KEY – Unique Identifier

A primary key is a unique identifier for each row in a table.

The id column is often used as the primary key.

Users Table			
id (PK)	name	email	password
1	Alice	alice@mail.com	hashed_pw1
2	Bob	bob@mail.com	hashed_pw2
3	Charlie	charlie@mail.com	hashed_pw3

6 FOREIGN KEY – Create Relationships

Foreign keys are used to create relationships between tables.

They help connect data across different tables.

Users Table	
id (PK)	name
1	Alice
2	Bob
3	Charlie

Orders Table			
id (PK)	user_id (FK)	product	order_date
101	1	Laptop	2024-05-10
102	2	Phone	2024-05-11
103	1	Headphones	2024-05-12

7 RELATIONSHIPS – Linking Data

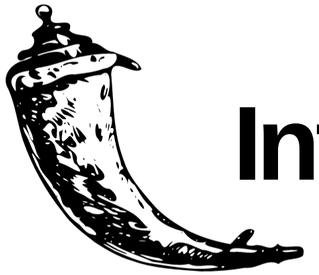
Using foreign keys, we can link and retrieve related data from multiple tables.



One User can have many Orders.

ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP



Introduction to SQLAlchemy

Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

Introduction to SQLAlchemy

SQLAlchemy is a powerful Python library used for working with databases





Without SQLAlchemy (Raw SQL)

```
SELECT id, name, email
FROM users
WHERE age > 18
ORDER BY name;
```

id	name	email
1	Alice	alice@mail.com
2	Bob	bob@mail.com
3	Charlie	charlie@mail.com

With SQLAlchemy (Pythonic Way)

```
from sqlalchemy import Column, Integer, String
from sqlalchemy.orm import declarative_base
Base = declarative_base()
class User(Base):
    __tablename__ = "users"
    id = Column(Integer, primary_key=True)
    name = Column(String  )
```

nkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Introduction to SQLAlchemy



- Repetitive
- Error-Prone
- Hard to Maintain

SQLAlchemy

- Providing abstraction over SQL
- Database portability
- Cleaner
- Maintainable code

ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Introduction to SQLAlchemy



SQLAlchemy Core	ORM (Object Relational Mapper)
Low-Level	High-Level

Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

Introduction to SQLAlchemy

SQLAlchemy Core

- Works closer to raw SQL queries
- Uses Python expressions to generate SQL
- Provides better control over query construction

```
from sqlalchemy import create_engine, text
engine = create_engine("sqlite:///example.db")
with engine.connect() as conn:
    result = conn.execute(text("SELECT * FROM users"))
```

Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

Introduction to SQLAlchemy

ORM (Object Relational Mapper)

- Maps Python classes to database tables
- Each object represents a table row
- Widely used in real-world applications

```
from sqlalchemy import Column, Integer, String
from sqlalchemy.orm import declarative_base
Base = declarative_base()
class User(Base):
    __tablename__ = "users"
    id = Column(Integer, primary_key=True)
    name = Column(String   )
```

ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Introduction to SQLAlchemy

➢ Key Concepts

Engine	Entry point to the database that manages connections
Session	Used for CRUD operations and acts as a staging area before committing changes
Models	Python classes mapped to database tables

ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Introduction to SQLAlchemy

➢ SQLAlchemy's Advantages

- ✓ Database agnostic – works with multiple databases
- ✓ Reduces the need for raw SQL queries
- ✓ Supports complex and advanced queries
- ✓ Scalable for large applications
- ✓ Integrates seamlessly with Flask using Flask-SQLAlchemy

ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP



Demo | Using SQLite with Flask

Copyright © Thinknyx Technologies LLP

Using SQLite with Flask

➢ What is SQLite

SQLite is a **lightweight, file-based relational database** that comes built into Python.
This means we **do not need to separately download or install a database server** like MySQL or PostgreSQL.



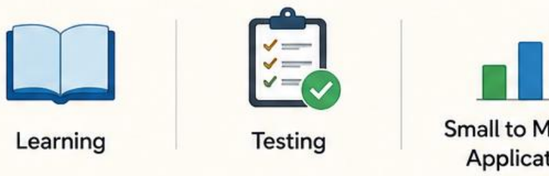
Instead of **running as a separate service**, SQLite stores all the database data **inside a single file** on our system.



Flask will communicate with this database file using **SQLAlchemy ORM**.



Because SQLite is **simple and lightweight**, it is commonly used for **learning, testing**, and **small to medium Flask applications**.



Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

Using SQLite with Flask



ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP



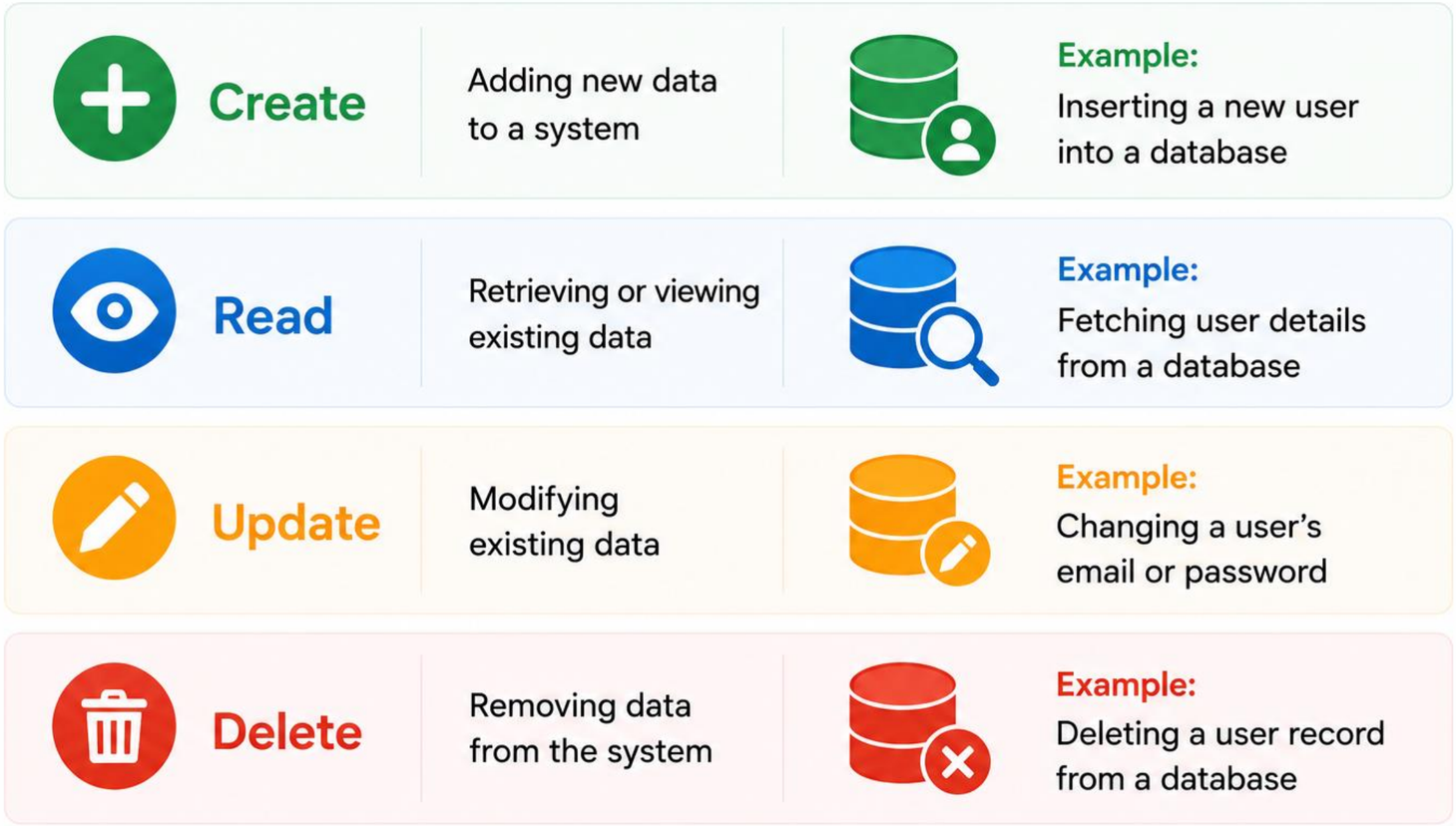
Demo | CRUD Operations

Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

CRUD Operations

CRUD operations are the **four fundamental** actions performed on data in any application, especially in databases and web development



ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

CRUD Operations

CRUD operations map closely to **HTTP methods** used in web applications

CRUD Operation		HTTP Method
Create	Adding new data to the system	POST
Read	Retrieving or viewing existing data	GET
Update	Modifying existing data	PUT / PATCH
Delete	Removing data from the system	DELETE

Overall, CRUD forms the **backbone** of any **data-driven application**, whether it's a simple form handler or a full-scale REST API.

Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP



Demo | Database Migrations with Flask-Migrate

ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Database Migrations with Flask-Migrate

- Structured way to manage database schema changes over time
- Prevents loss of existing data during updates
- Helps handle changes like:
  - Adding new columns
  - Modifying table structures
  - Renaming fields
  - Creating or deleting tables
- Avoids manual database editing (reduces errors)
- Tracks, applies, and versions schema changes systematically



ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

How migrations work?

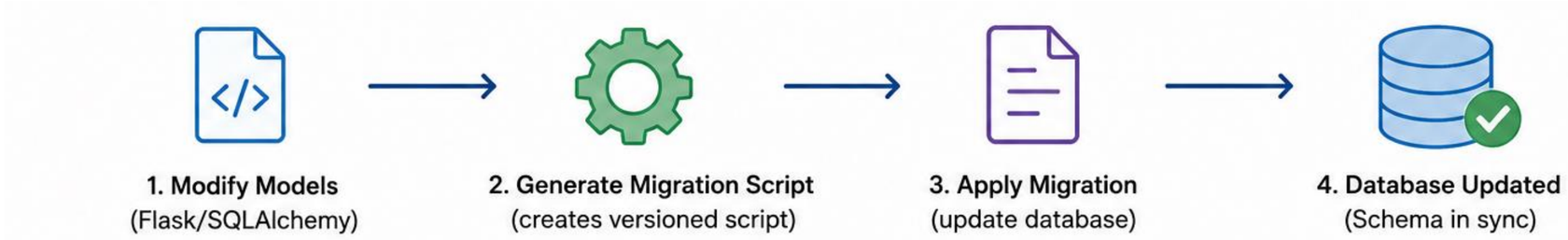
A migration system keeps a history of changes as versioned scripts

Upgrade

that is what changes
to apply (e.g., add a column)

Downgrade

that is how to undo
the change or rollback



ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Database Migrations with Flask-Migrate

1 Initial Migration

First, we will be migrating our db where we will see that no changes are detected.

```
$ flask db migrate -m "initial migration"
No changes detected
```

users

id	name	email
1	Alice	alice@example.com
2	Bob	bob@example.com



2 Add bio Column

Then we will add a bio column in our db and then again migrate.

```
$ flask db migrate -m "add bio column"
Detected changes in models:
+  add column users.bio
```

users

id	name	email	bio
1	Alice	alice@example.com	Hello!
2	Bob	bob@example.com	Dev



3 Upgrade (Apply Migration)

Apply the migration to update the database.

```
$ flask db upgrade
Done.
```

users

id	name	email	bio
1	Alice	alice@example.com	Hello!
2	Bob	bob@example.com	Dev



4 Downgrade (Rollback)

We will downgrade to the previous version to see that the bio column no longer exists.

```
$ flask db downgrade
Done.
```

users

id	name	email
1	Alice	alice@example.com
2	Bob	bob@example.com



ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Thank you



ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP



Authentication and Authorization

Copyright © Thinknyx Technologies LLP



Authentication and Authorization

Section Overview

- Implement registration, login, and logout in Flask
- Secure passwords using hashing
- Manage user sessions
- Understand Role-based access control (RBAC)

ThinkRook
By Thinknyx Technologies LLP



Demo | User Registration, Login, Logout and Password Hashing

ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP



Demo | Session Management

Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

Session Management



Session Management

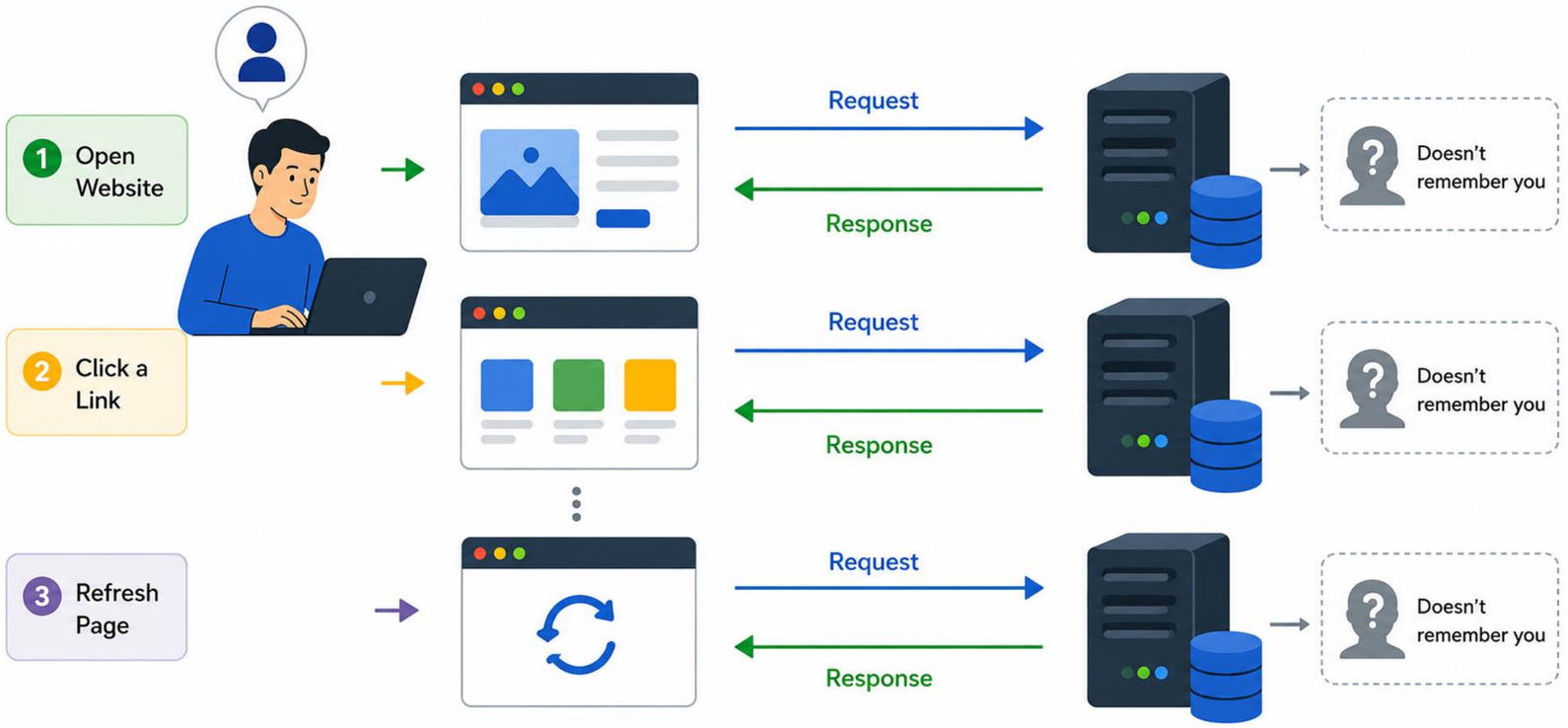
Maintaining a user’s state (data and identity)

across multiple requests

Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

Session Management

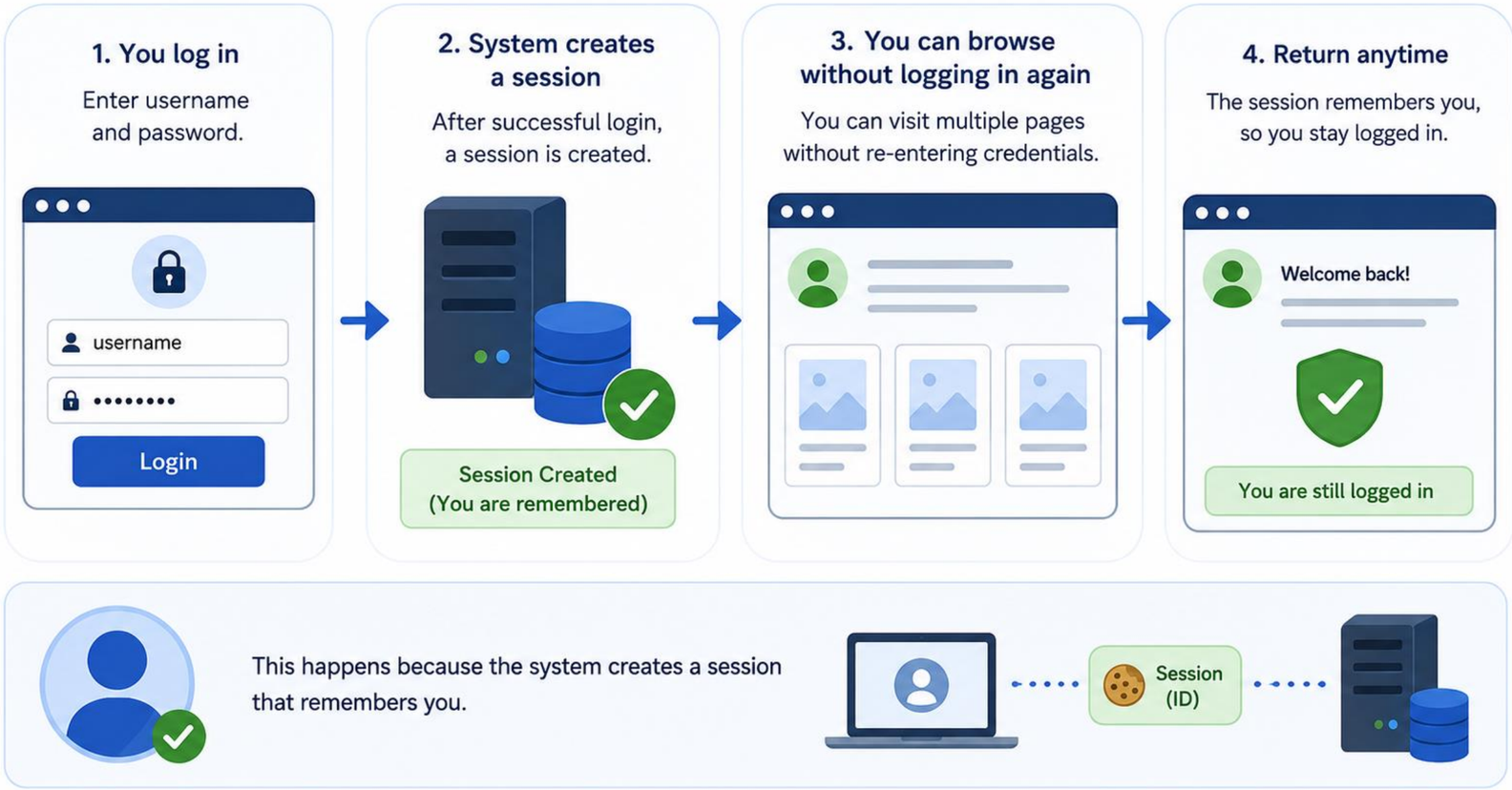


Session management tracks user activity and identity during a visit

Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

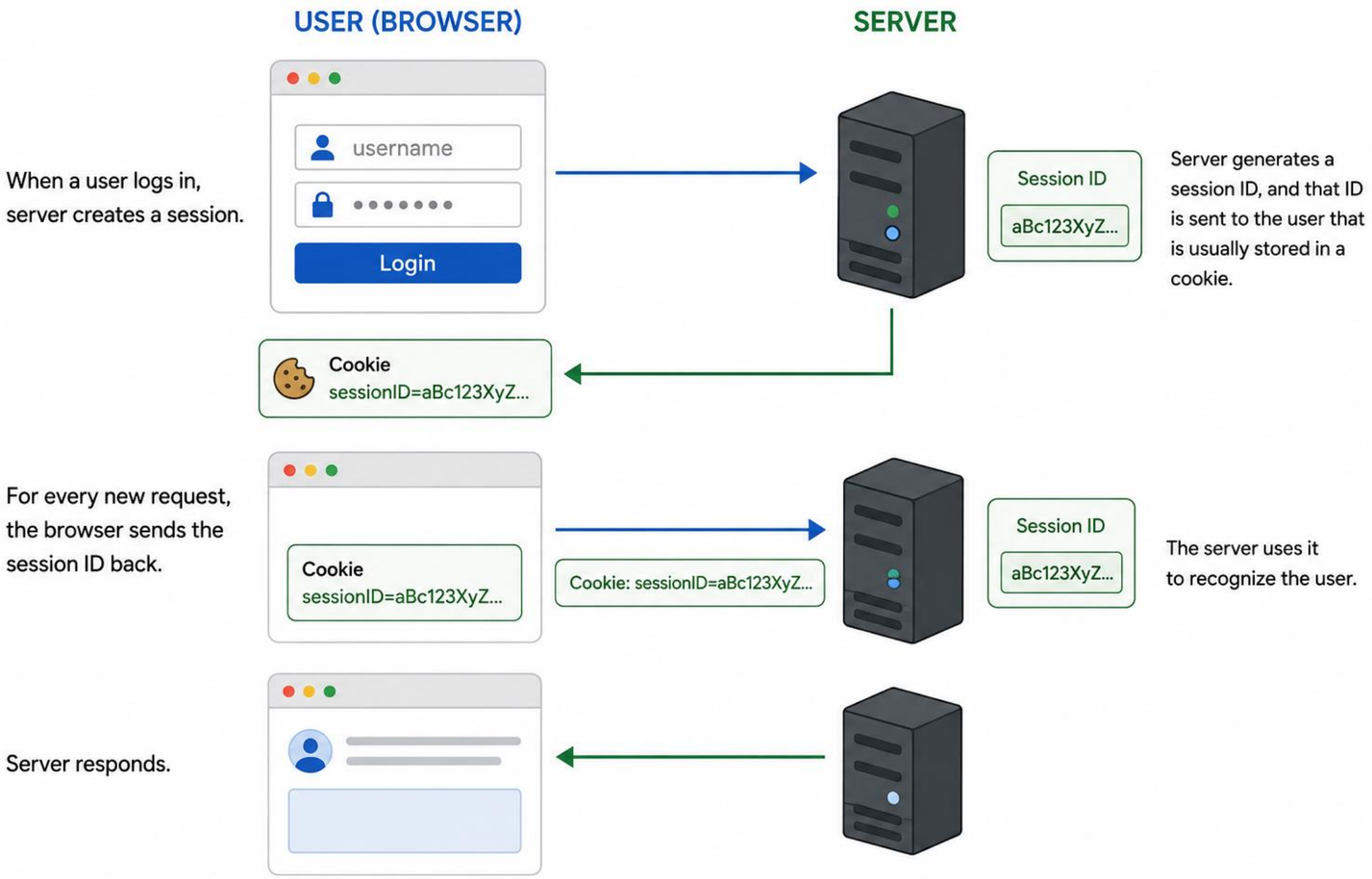
Session Management



Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

How does it works?



ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP



Demo | Role-Based Access Control

Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

Role-Based Access Control



Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

Thank you



ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

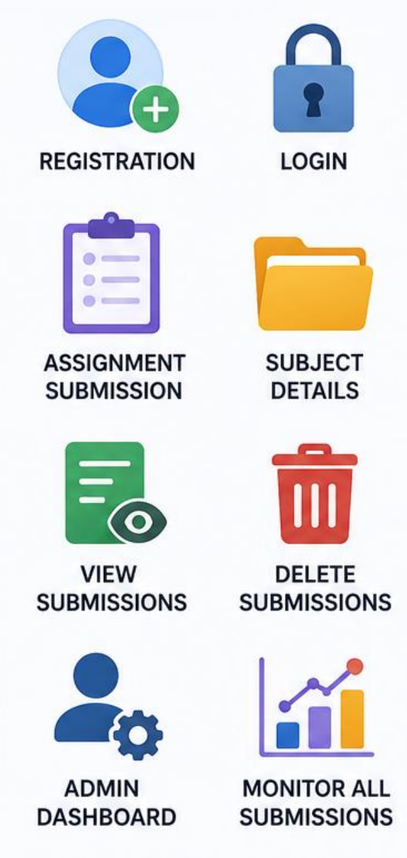


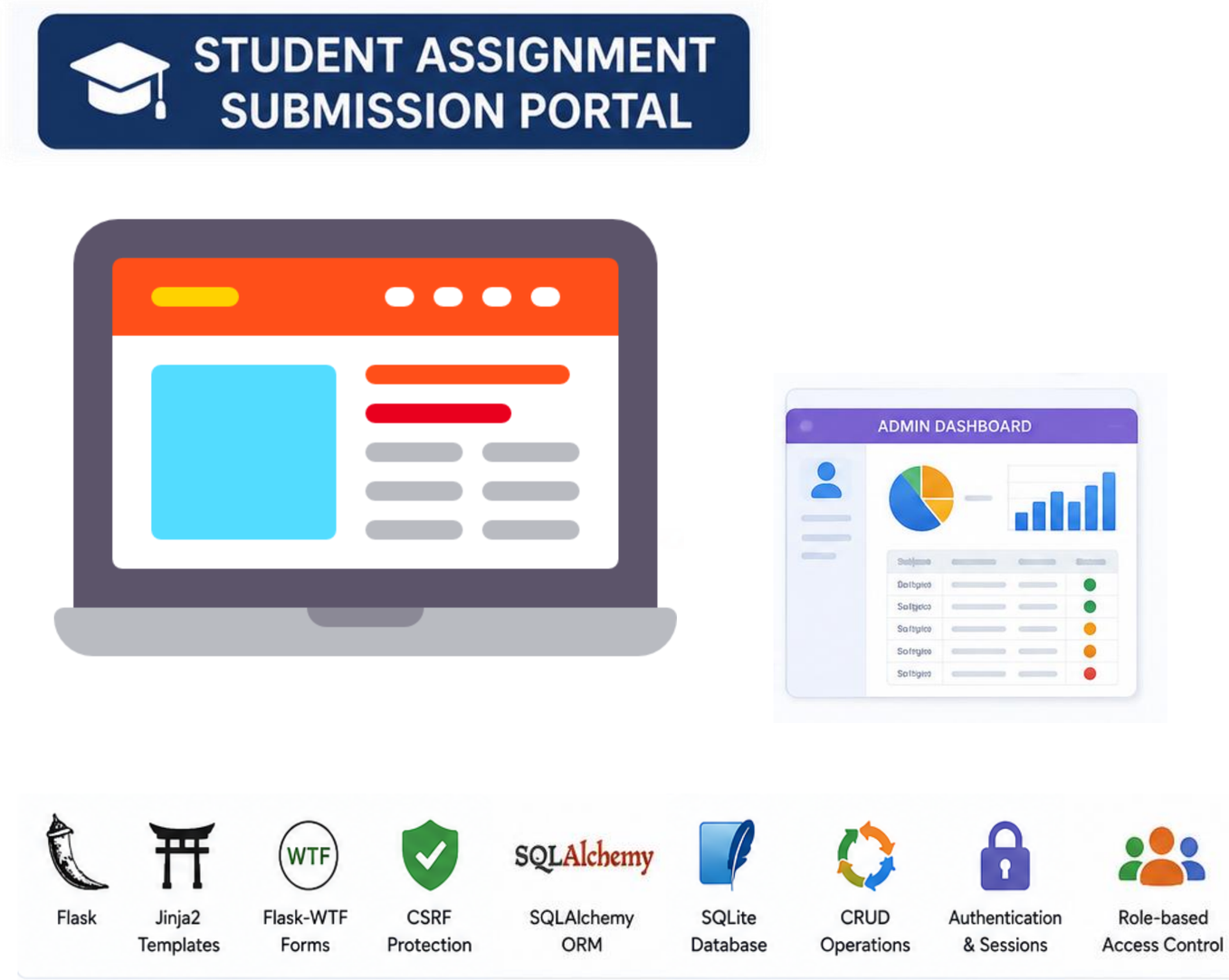
Demo | Use Case: Student Assignment Submission Portal

ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Student Assignment Submission Portal





Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

Student Assignment Submission Portal

```
├── app.py
```

- Main Flask application file
- Entry point of the application

- This file contains:

  - Flask app initialization
  - Application configuration
  - Route definitions
  - Authentication logic
  - Session handling
  - CRUD operations
  - Role-based access control

Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

Student Assignment Submission Portal

```
├── app.py
├── models.py
```

- Contains all database models
- Models define:
  - Database tables
  - Structure using SQLAlchemy
- User model stores user information
- Assignment model stores assignment details

ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Student Assignment Submission Portal

```
├── app.py
├── models.py
├── forms.py
```

- Contains all Flask-WTF form classes
- It handles:
  - Form creation
  - Adds validation
  - Enables CSRF protection

Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

Student Assignment Submission Portal

```
├── app.py
├── models.py
├── forms.py
├── extensions.py
```

- Initializes Flask extensions separately from main application
- It:
  - Avoids circular imports
  - Makes the application modular
  - Improves scalability

ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Student Assignment Submission Portal

```
├── app.py
├── models.py
├── forms.py
├── extensions.py
├── requirements.txt
│
├── templates/
```

- Stores all HTML templates used by Flask
- Flask-WTF Forms:
  - Login
  - Register
  - Add assignment
- CSRF token

ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Student Assignment Submission Portal

```
├── app.py
├── models.py
├── forms.py
├── extensions.py
├── requirements.txt
│
├── templates/
│   ├── base.html
│   ├── index.html
│   ├── register.html
│   ├── login.html
│   ├── dashboard.html
│   ├── add_assignment.html
│   └── admin.html
```

- base.html:
  - nav bar
  - Extended over every other file using jinja2 template
- index.html:
  - Landing page
- register.html:
  - Register
- login.html:
  - Login
- dashboard.html:
  - All assignments listed
- add_assignment.html:
  - User can add assignments
- admin.html:
  - Only for admin access

ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Student Assignment Submission Portal

```
├── app.py
├── models.py
├── forms.py
├── extensions.py
├── requirements.txt
│
├── templates/
│   ├── base.html
│   ├── index.html
│   ├── register.html
│   ├── login.html
│   ├── dashboard.html
│   ├── add_assignment.html
│   └── admin.html
│
├── static/
│   └── style.css
```

- static:
  - Contains css files

Copyright © Thinknyx Technologies LLP

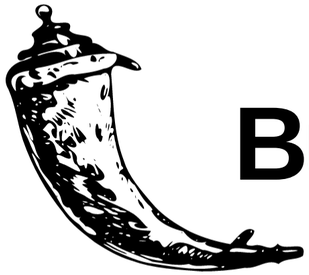
ThinkRook
By Thinknyx Technologies LLP

Thank you



ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP



Building REST APIs with Flask

Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP



Building REST APIs with Flask

Section Overview

- Understanding REST Architecture
- Introduction to Flask-RESTful
- Demo: Flask-RESTful in Action

ThinkRook
By Thinknyx Technologies LLP

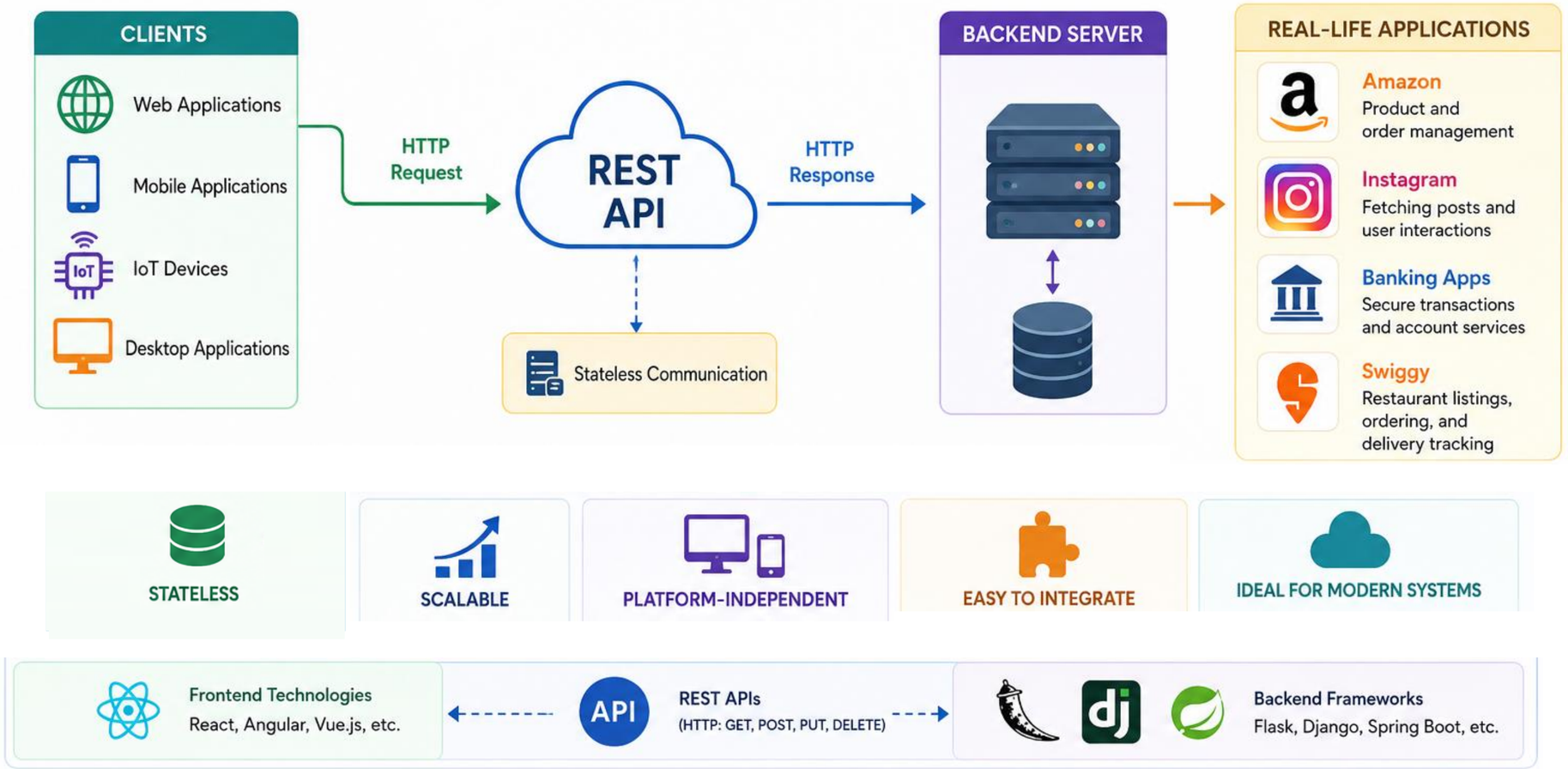


REST Architecture

Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

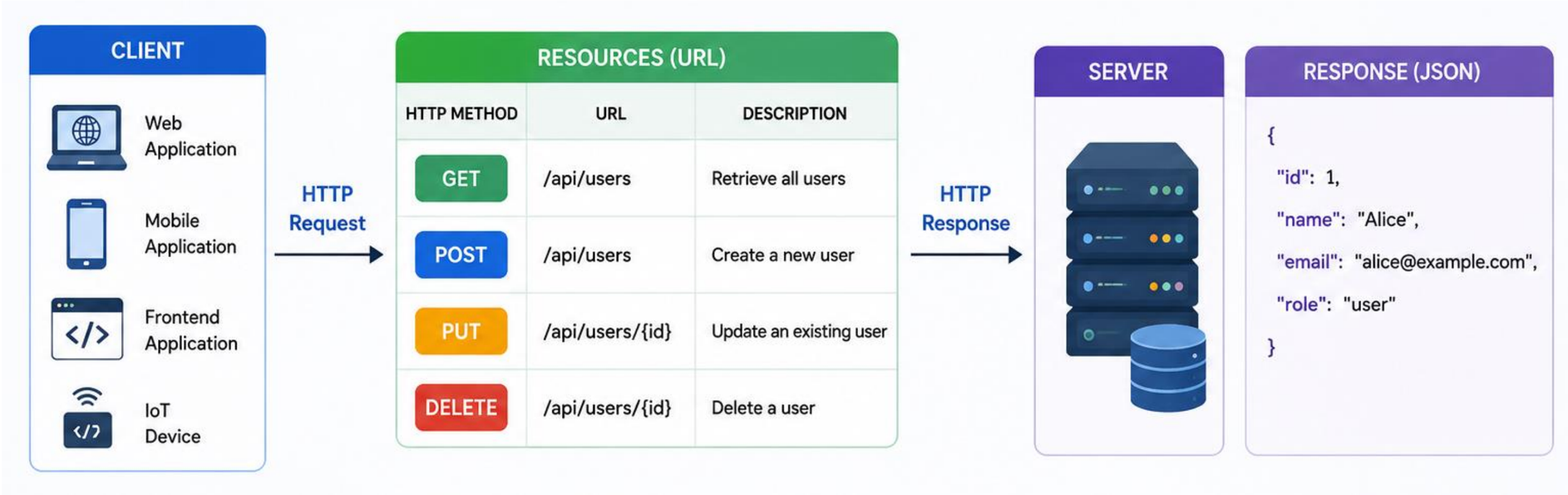
REST Architecture



ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

REST Architecture



Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

REST Architecture



- It does not enforce any **strict architecture**

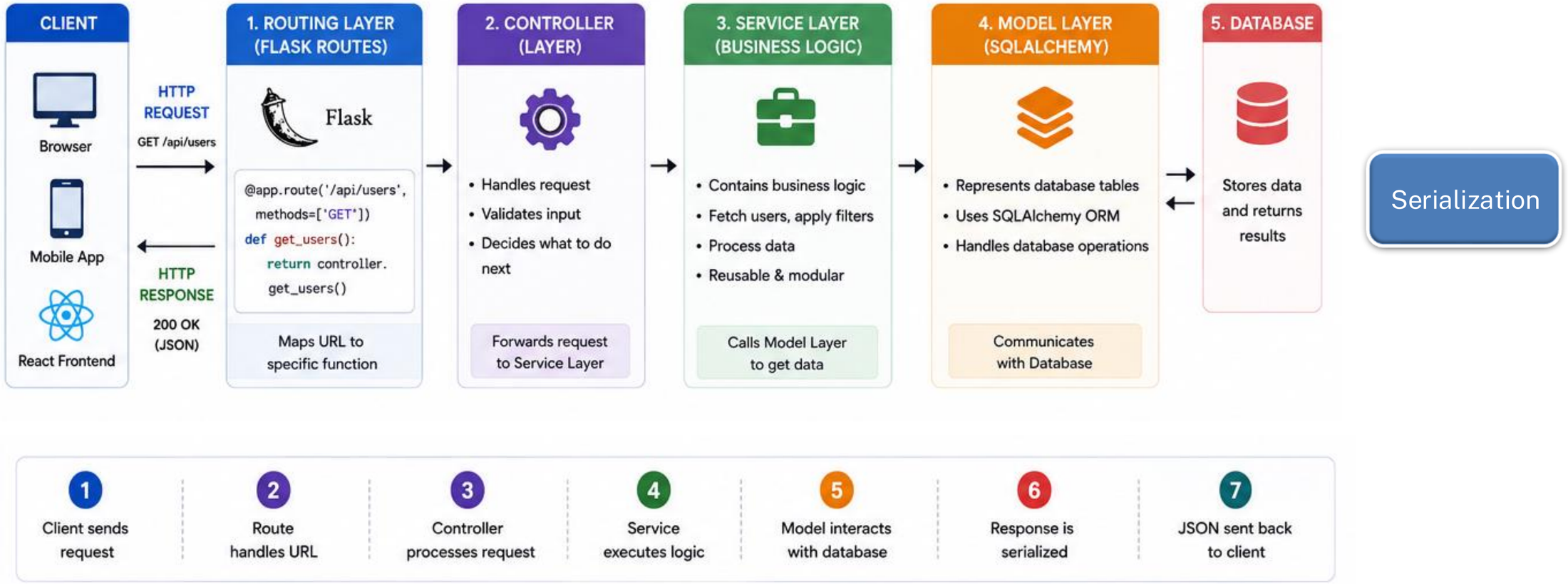
Design a layered structure to keep our application

clean and **scalable**

ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

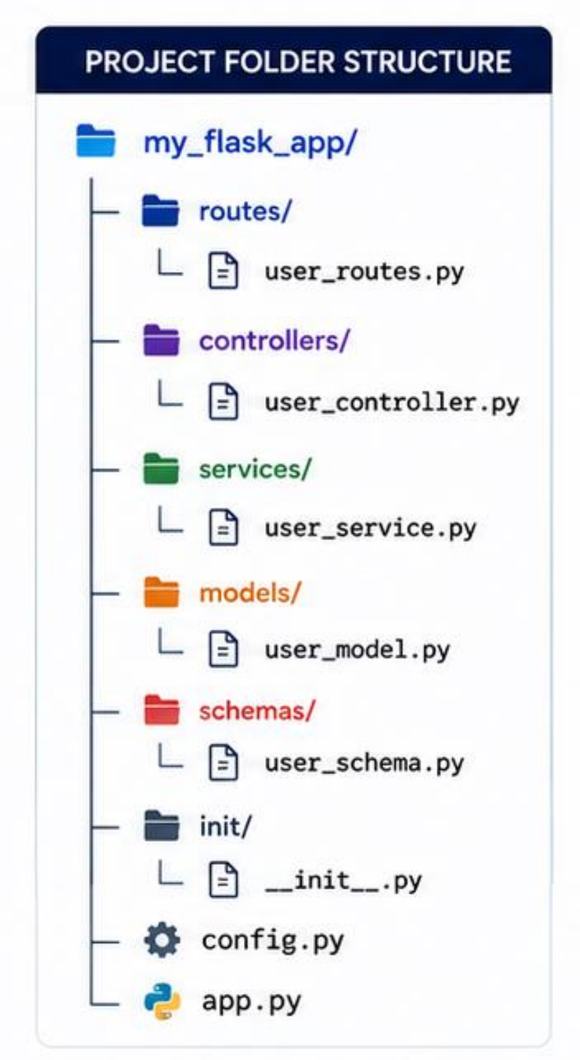
REST Architecture



ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

REST Architecture



Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

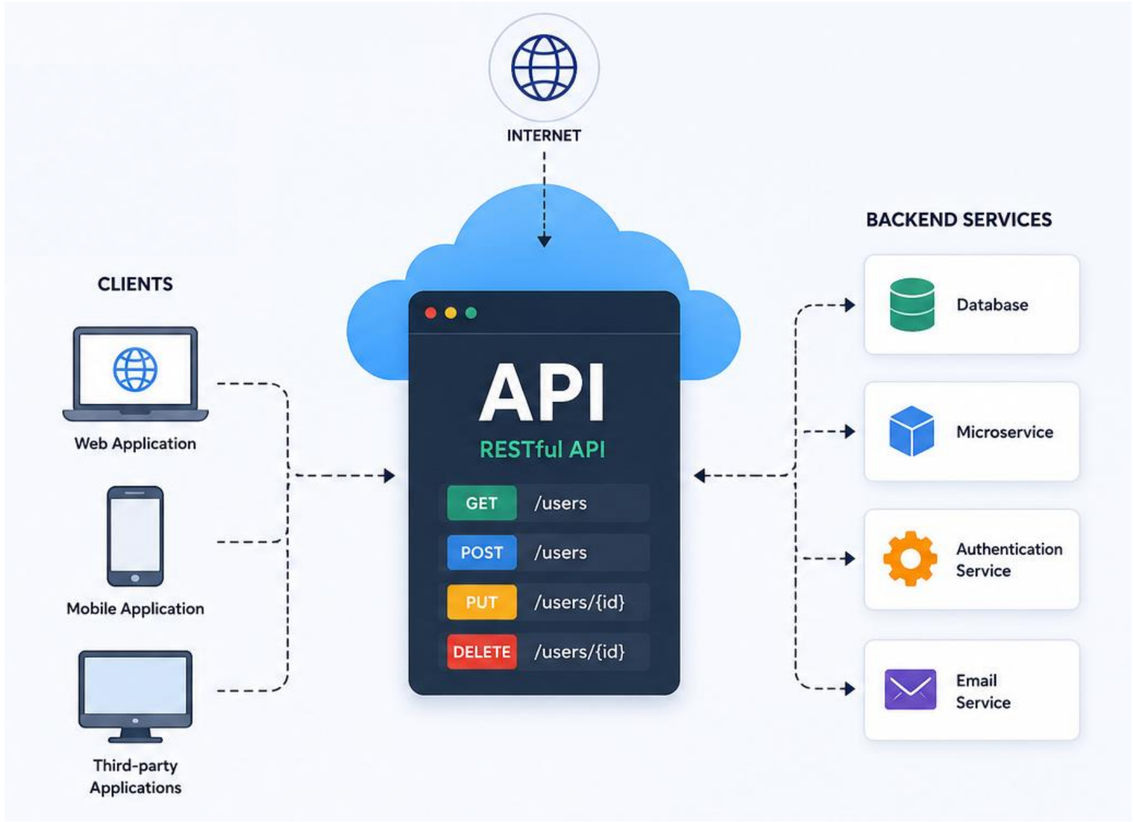


Introduction to Flask-RESTful

Copyright © Thinknyx Technologies LLP

Introduction to Flask-RESTful

But in modern web development, building **scalable** and **maintainable** APIs is a fundamental requirement

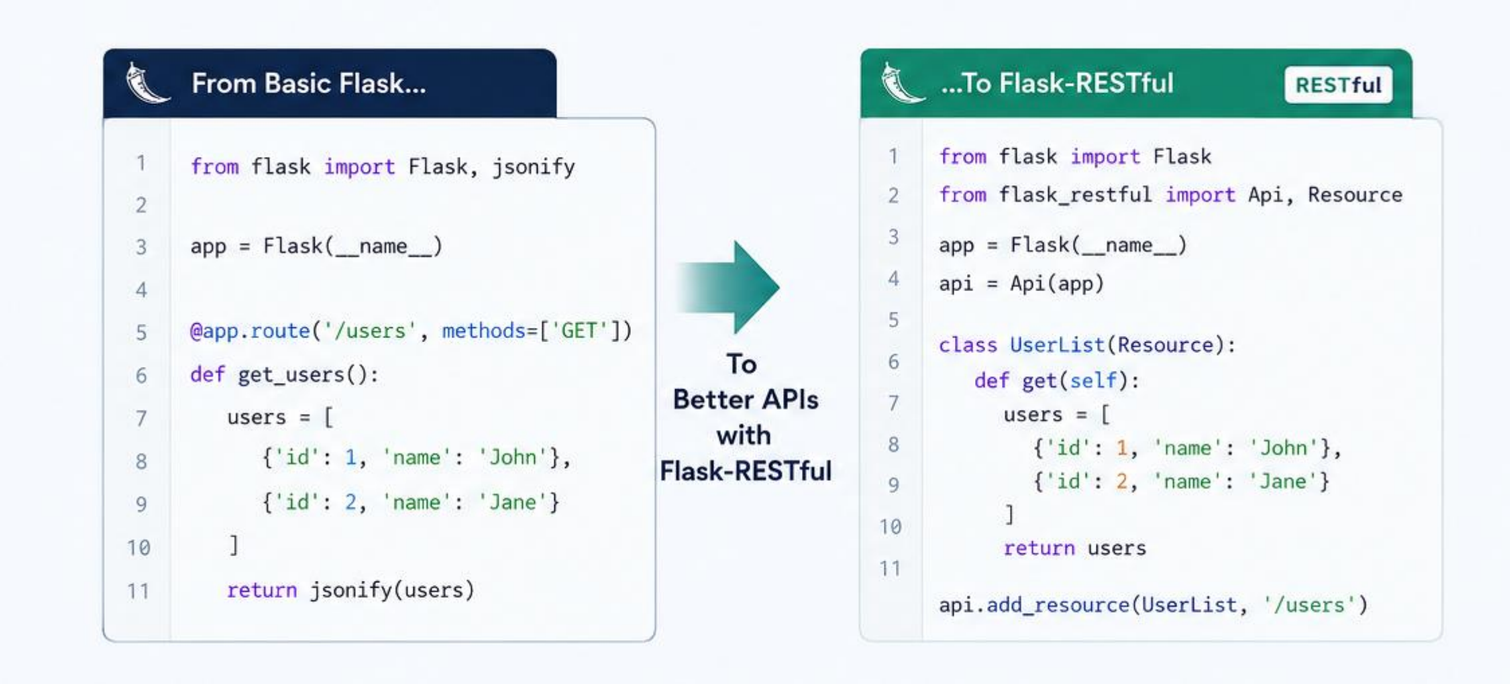


Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

Introduction to Flask-RESTful

Flask, is **a lightweight** and **flexible** Python web framework

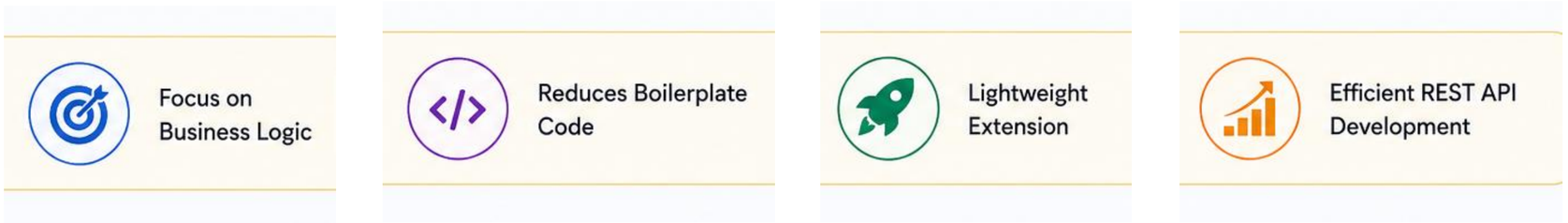


ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Introduction to Flask-RESTful

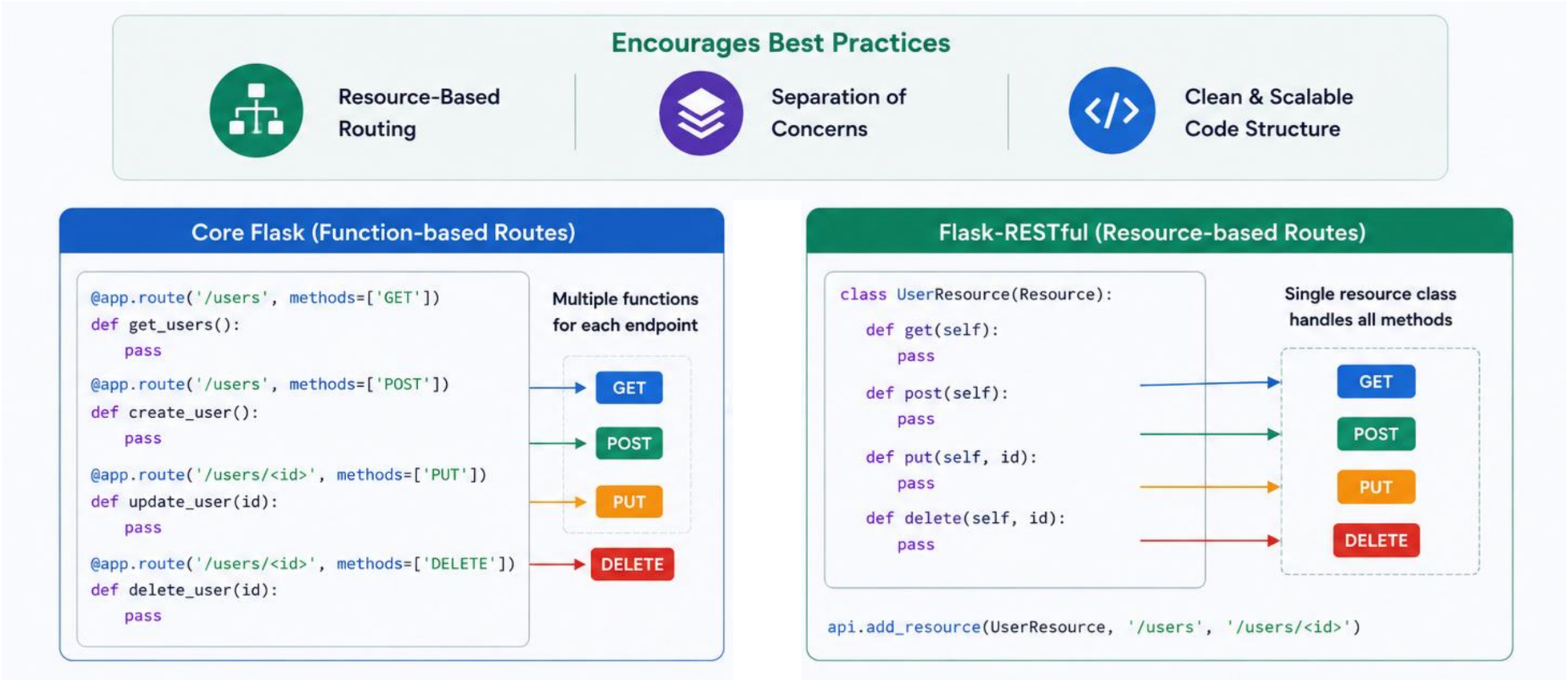




ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Introduction to Flask-RESTful



ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Introduction to Flask-RESTful



Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

Introduction to Flask-RESTful

- Create an Api object and pass our Flask app

```
api = Api(app)
```

- Create classes called Resources

```
class Item(Resource):
```

- Define behavior using **get()** and **post()** methods

```
def get(self):
def post(self):
```

- Connect our resource to a URL

```
api.add_resource(Item, '/items')
```

ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

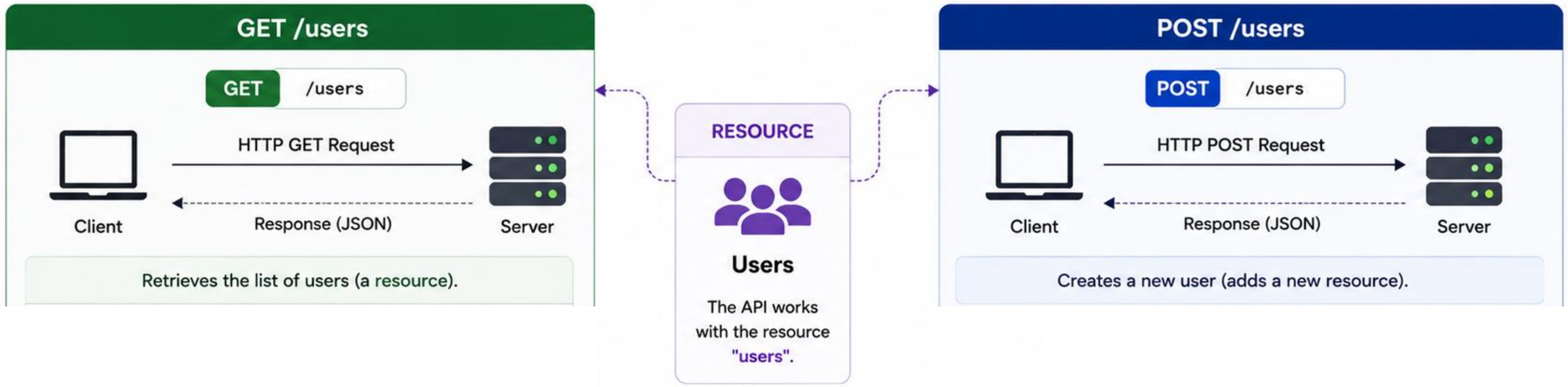


Demo | Flask-RESTful in Action

Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

Flask-RESTful in Action



ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Thank you



ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP



Error Handling and Debugging

Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP



Error Handling and Debugging

Section Overview

- Understanding Debug Mode in Flask
- Implementing Custom Error Pages
- Logging in Flask Applications

ThinkRook
By Thinknyx Technologies LLP



Demo | Debug Mode

Copyright © Thinknyx Technologies LLP

Debug Mode

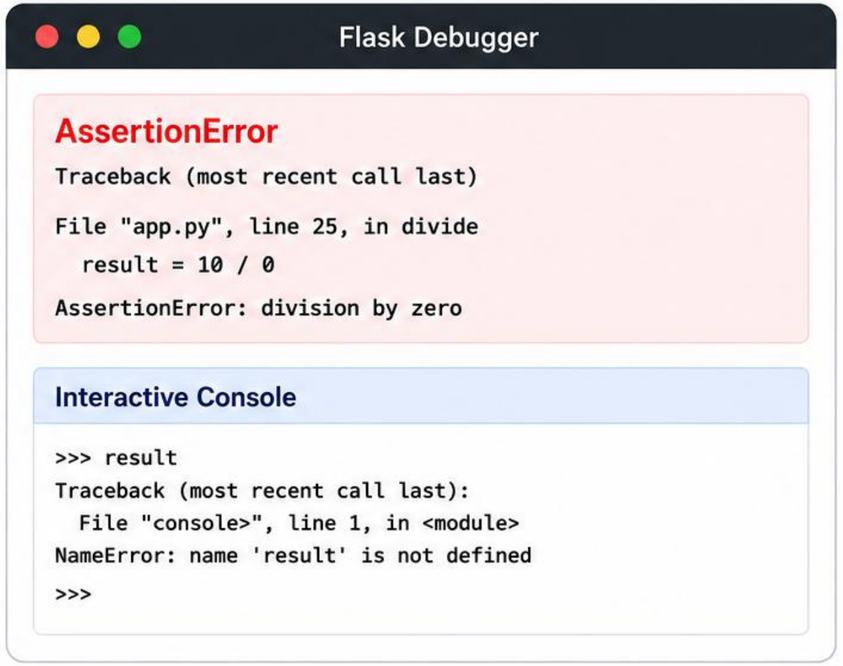
Debug mode is a special development setting in Flask that helps developers debug applications more efficiently.

1. Error Messages

2. Interactive Debugger

3. Automatic Application Reload

4. Easier Troubleshooting

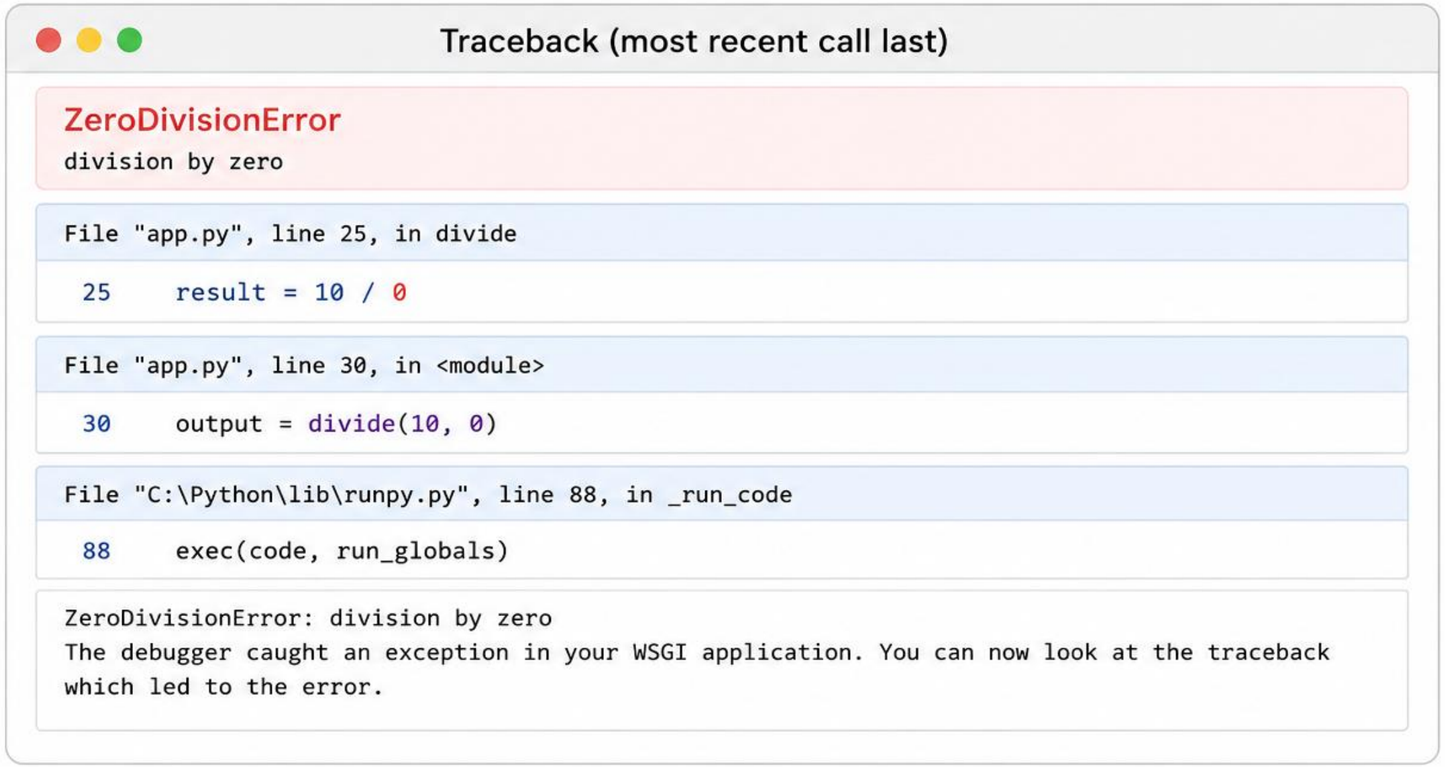


Debug mode should only be used in development and never in production environments.

ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

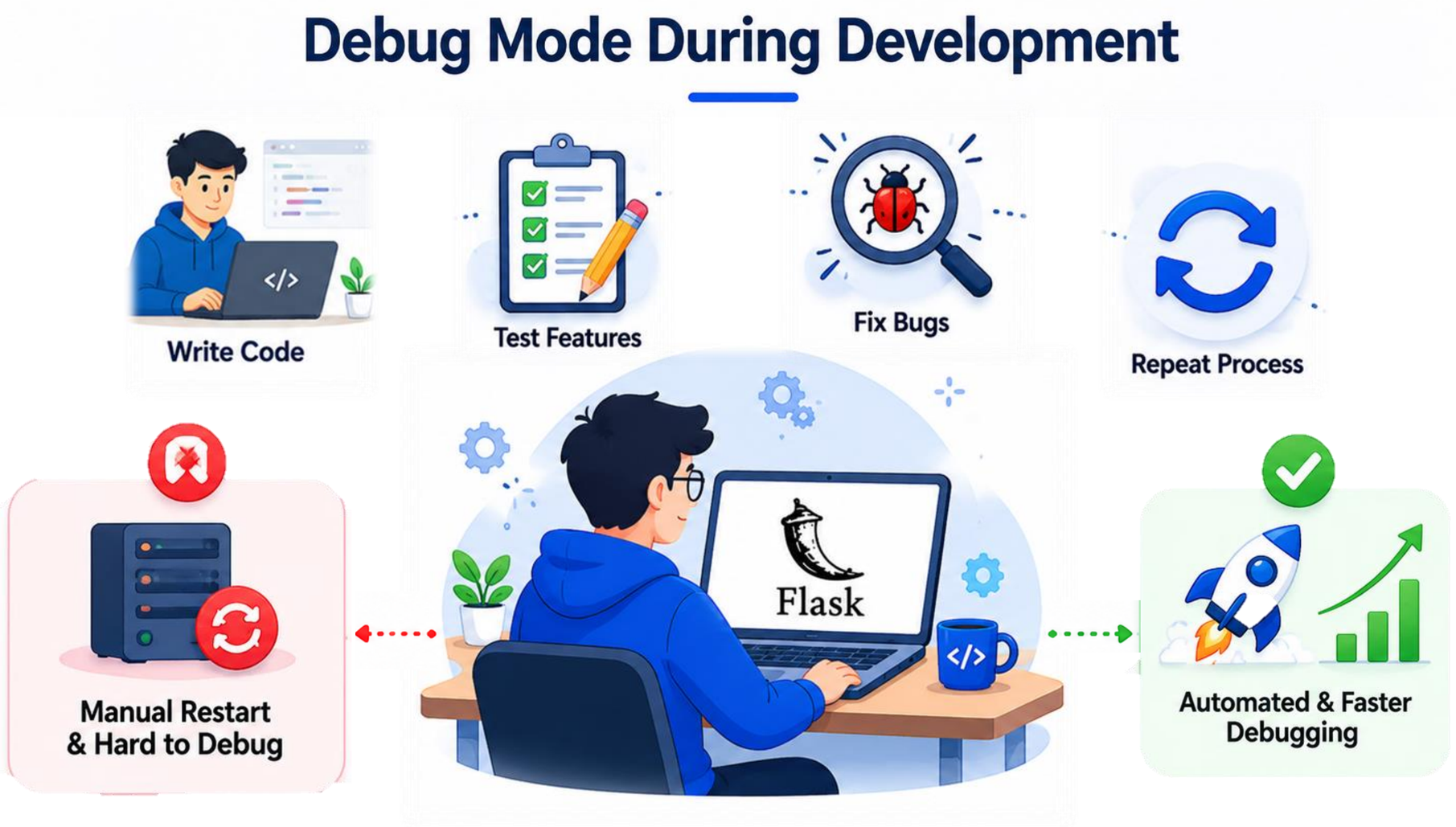
Debug Mode



ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

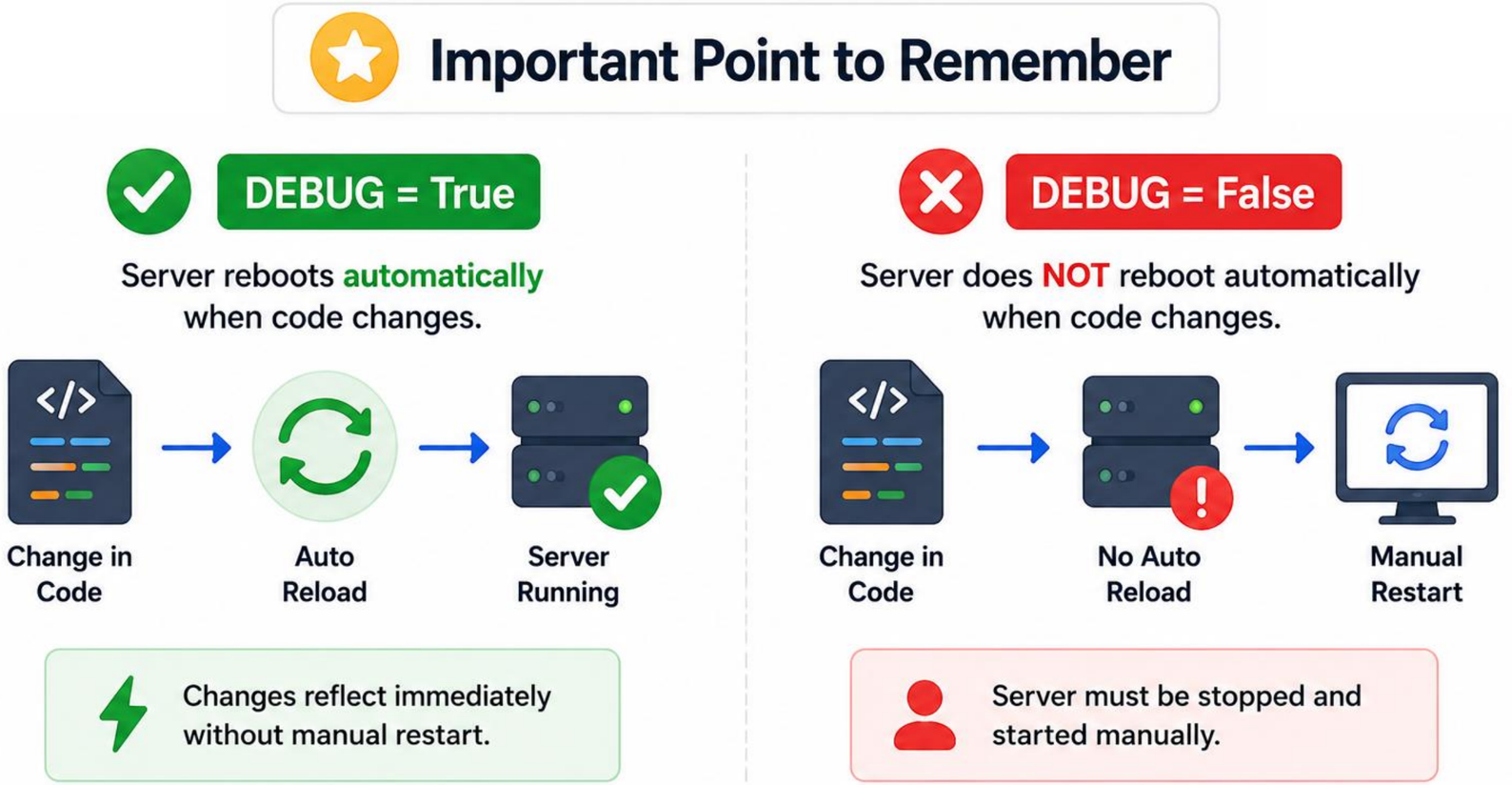
Debug Mode



Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

Debug Mode



ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Debug Mode

- Multiple ways to enable debug mode in Flask

Enabling debug mode directly in the application

```
app.run(debug=True)
```

Terminal Commands

- Windows

```
set FLASK_DEBUG=1
flask run
```

- Linux or Mac

```
export FLASK_DEBUG=1
flask run
```

ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

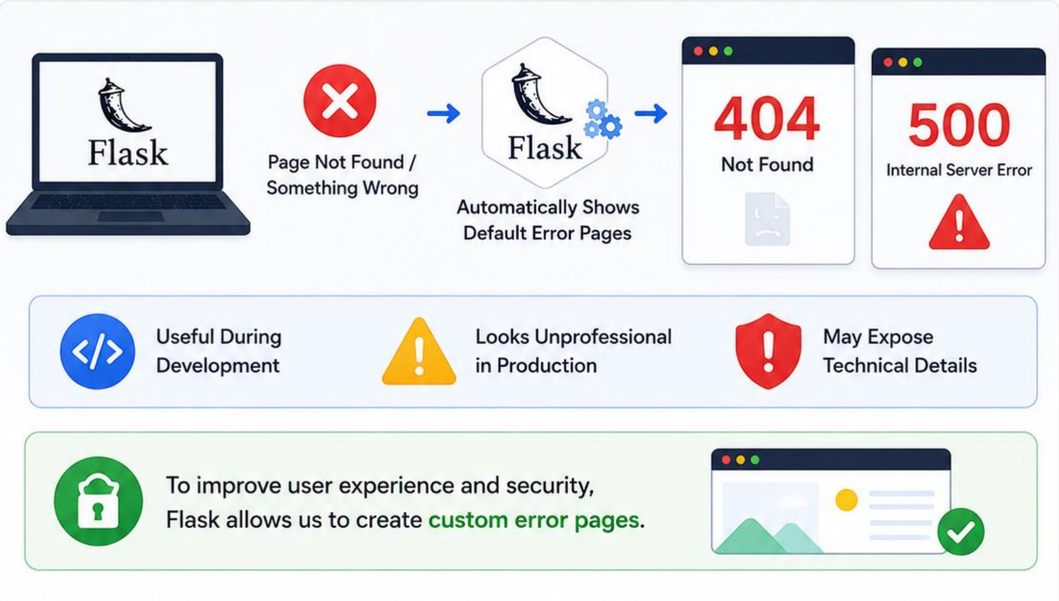


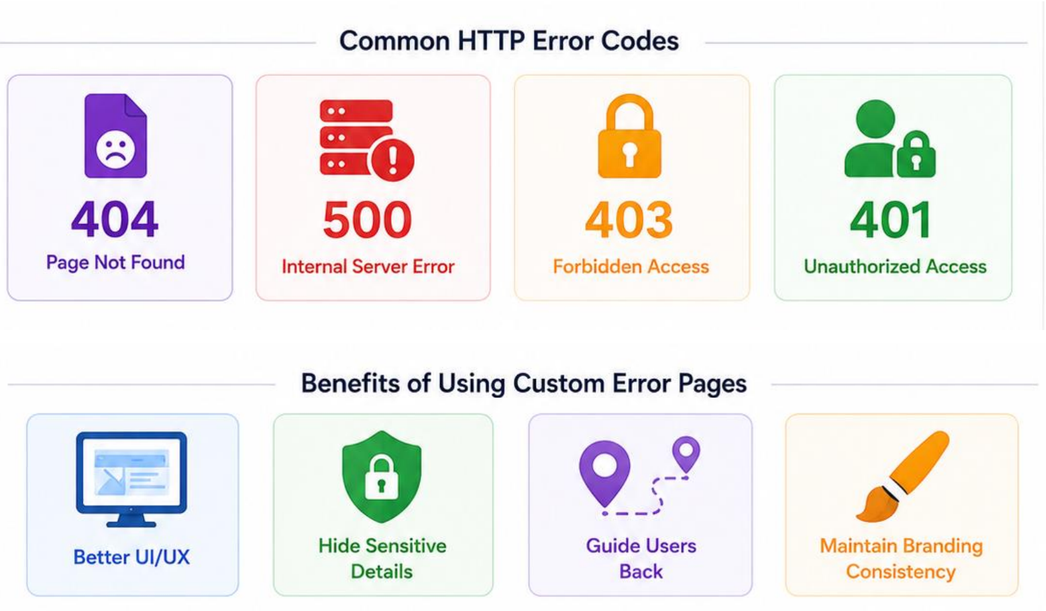
Demo | Custom Error Pages

Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

Custom Error Pages





ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Custom Error Pages

Flask uses the `@app.errorhandler()` decorator to handle specific errors.

Syntax

```
1  @app.errorhandler(error_code)
2  def function_name(error):
4      return render_template("page.html"), error_code
```

- `error_code` is HTTP error code like 404 or 500
- `error` contains information about the exception
- `render_template()` loads the custom HTML page
- The second value `error_code` sets the correct HTTP status code

Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

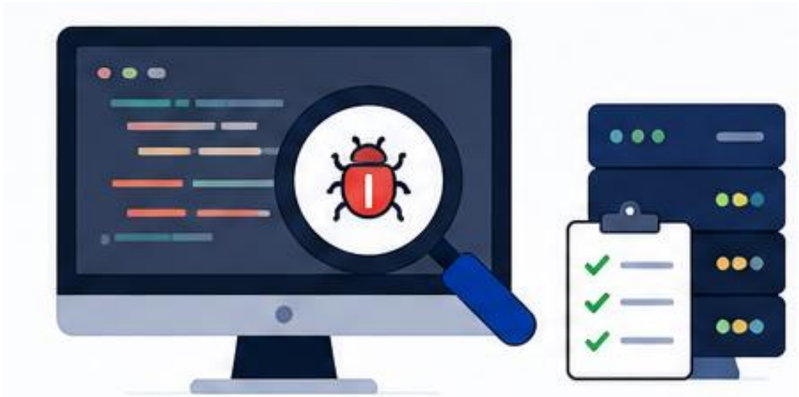


Demo | Logging

Copyright © Thinknyx Technologies LLP

Logging

- Recording Events, Errors & Warnings
- Monitoring Application Activity
- Debugging Issues Efficiently
- Tracking System Information
- Uses Python Built-in **Logging** Module
- Flask Creates **Logger Object** Automatically
- Access Logger Using **app.logger**



ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Why logging is important?





Using print()

- Unstructured output
- No severity level
- Hard to filter and search
- Not suitable for production

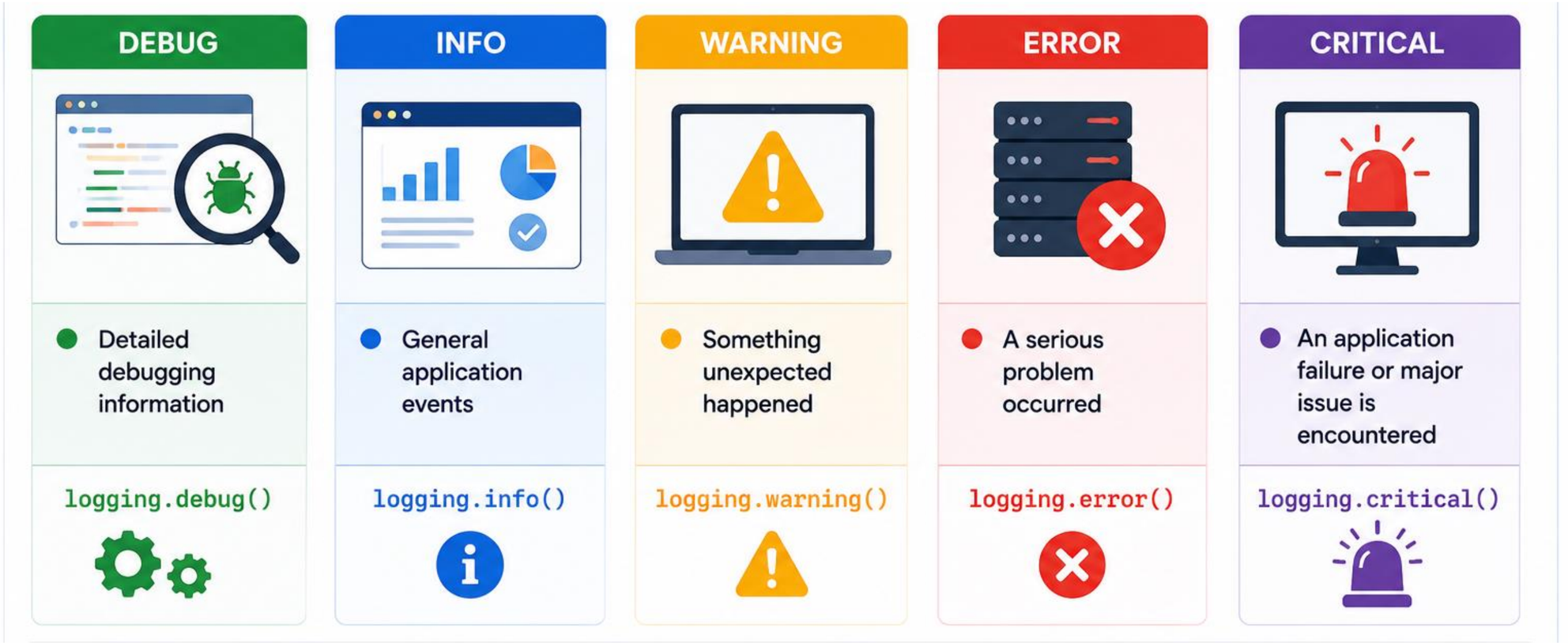
```
print("An error occurred")
print("User logged in")
print("Warning: Invalid data")
```

Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

Logging

- **Python logging provides multiple severity levels**



ThinkRook
By Thinknyx Technologies LLP

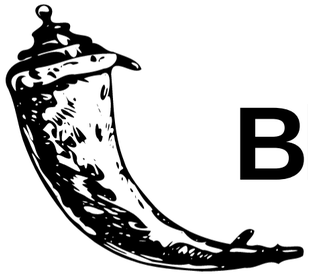
Copyright © Thinknyx Technologies LLP

Thank you



ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP



Blueprints and Application Factory Pattern

Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

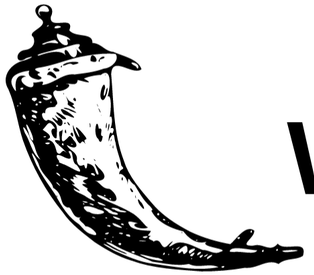


Blueprints and Application Factory Pattern

Section Overview

- Understand Flask Blueprints and their purpose
- Create a modular application using Blueprints
- Discuss Application Factory Pattern

ThinkRook
By Thinknyx Technologies LLP

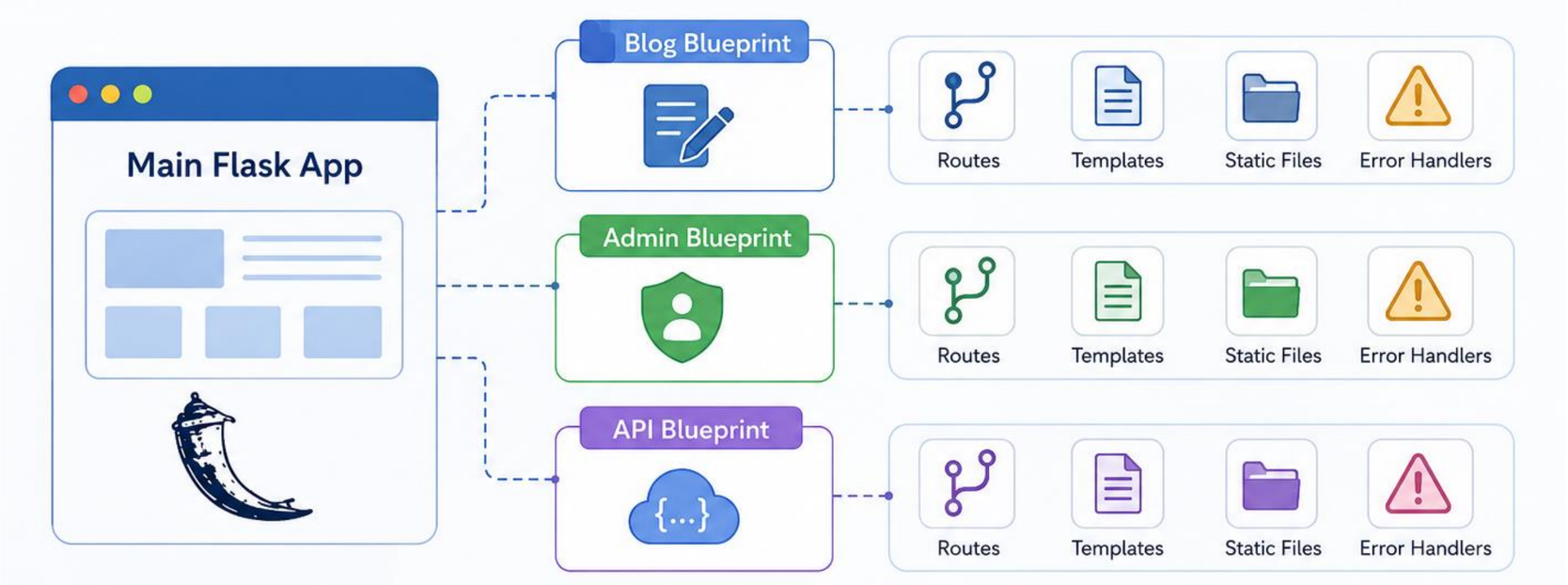


What are Blueprints

Copyright © Thinknyx Technologies LLP

What are Blueprints

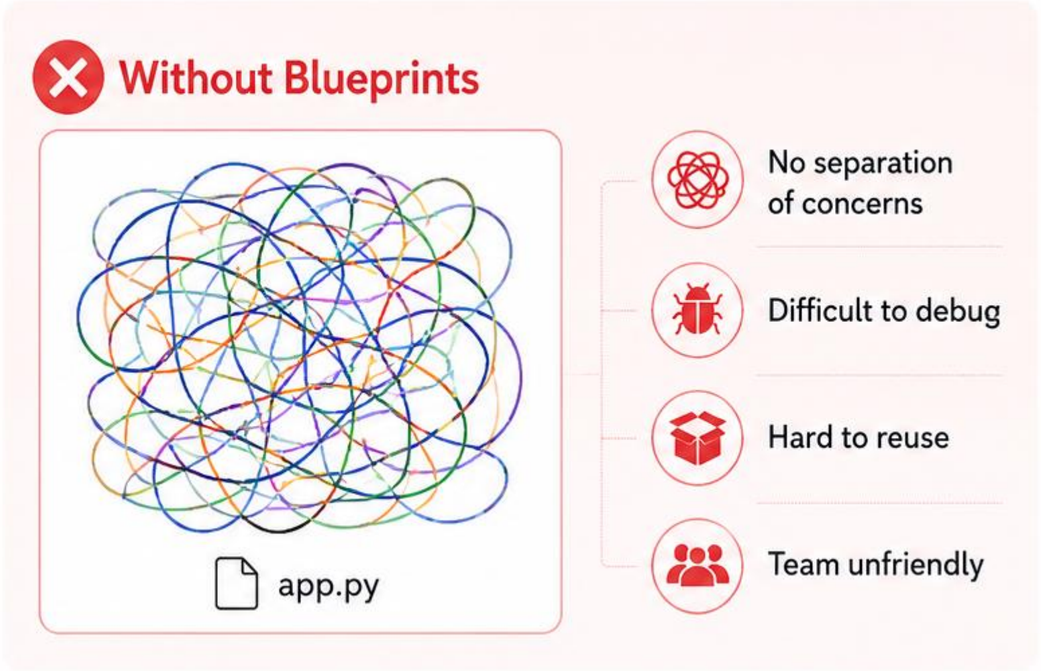
Flask Blueprints are a powerful way to organize your application into modular, reusable components



ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

What are Blueprints



Blueprints solve this by letting you split routes and logic into separate files or folders

ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

What are Blueprints



Blueprints are essential beyond a simple Flask script

Copyright © Thinknyx Technologies LLP

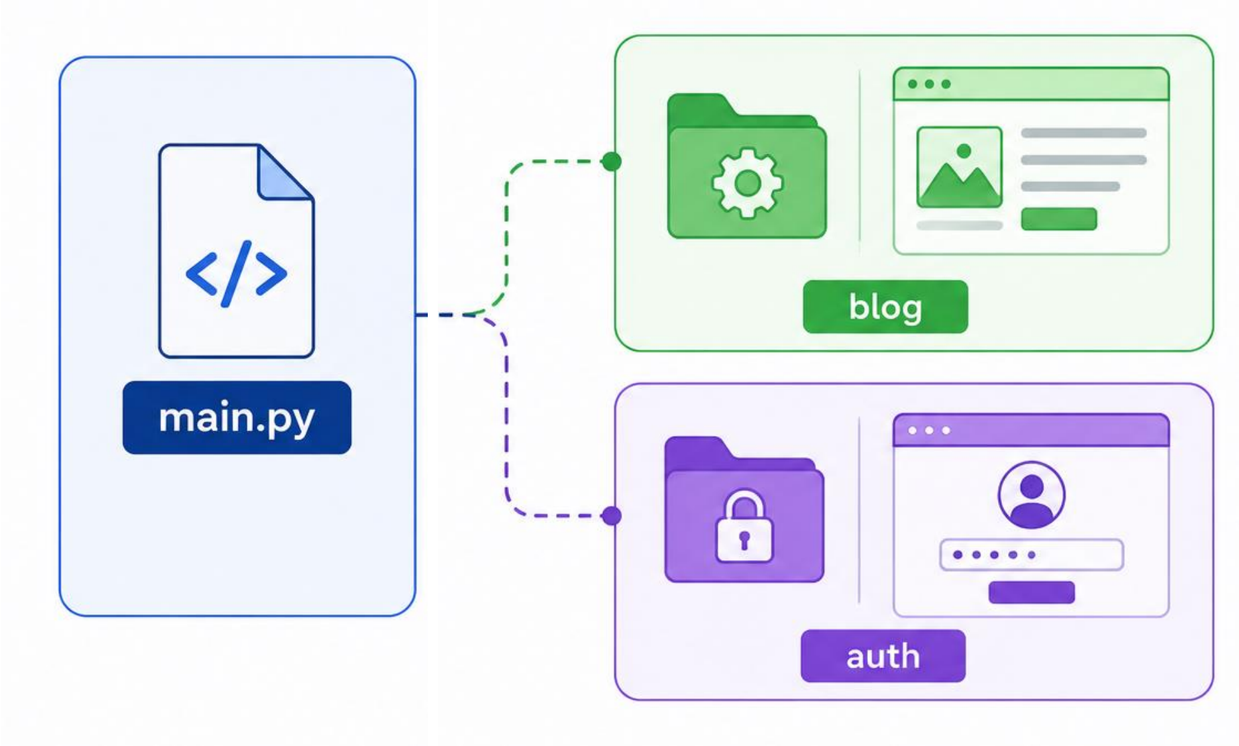
ThinkRook
By Thinknyx Technologies LLP



Demo | Creating Modular Application

Copyright © Thinknyx Technologies LLP

Creating Modular Application



Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP



Application Factory Pattern

Copyright © Thinknyx Technologies LLP

Application Factory Pattern

Design pattern used to create Flask applications in a flexible and scalable way

Flask app is created inside a function called the Application Factory instead of a global variable



1. Large and modular applications



2. Using multiple configurations



3. Testing applications easily



4. Avoiding circular imports



5. Initializing extensions cleanly

Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

Application Factory Pattern

- Beginners create Flask apps

```
from flask import Flask
app = Flask(__name__)
@app.route("/")
def home():
   return "Hello"
```

- Application Factory Pattern

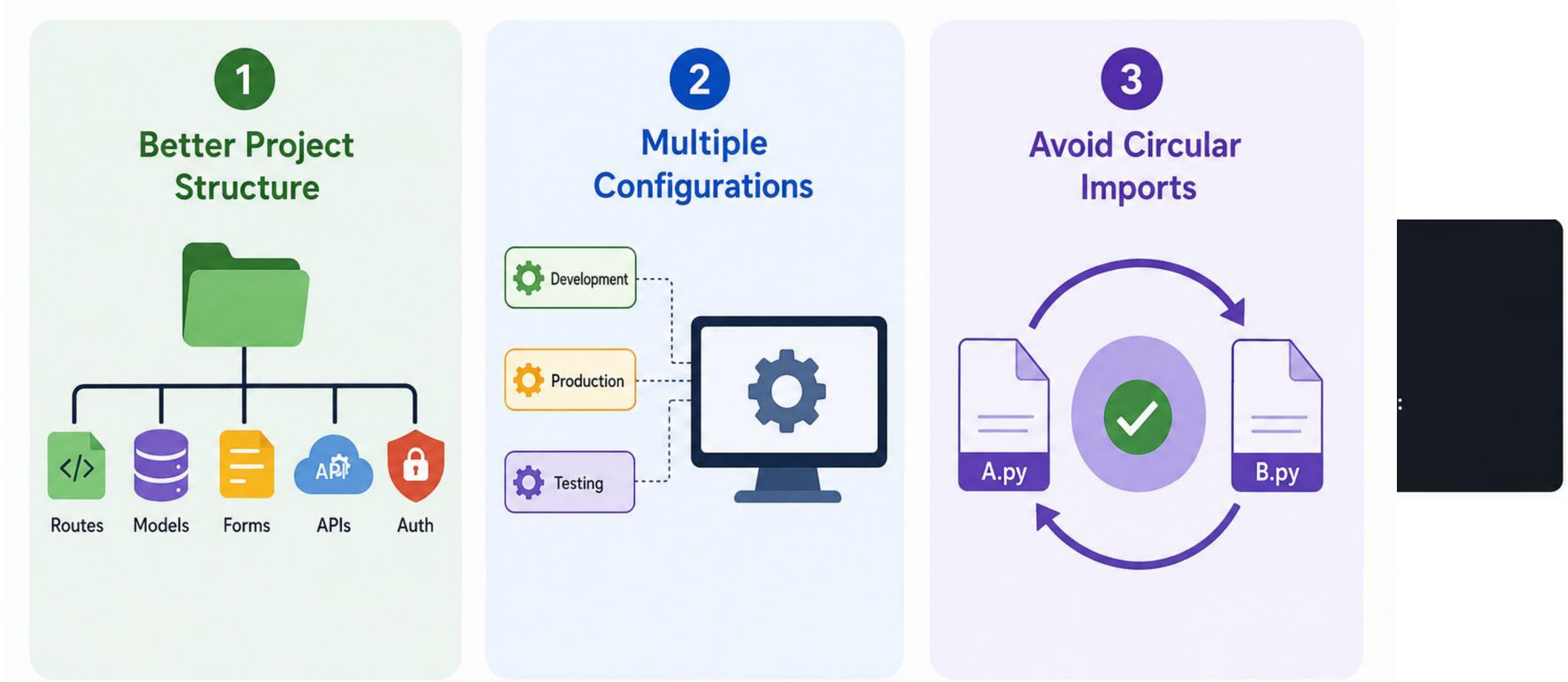
```
from flask import Flask
def create_app():
    app = Flask(__name__)
    @app.route("/")
    def home():
        return "Hello"
    return app
```

Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

Why to use Application Factory Pattern?

- **3 major reasons for this:**



Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

Application Factory Pattern



Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

Thank you



ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP



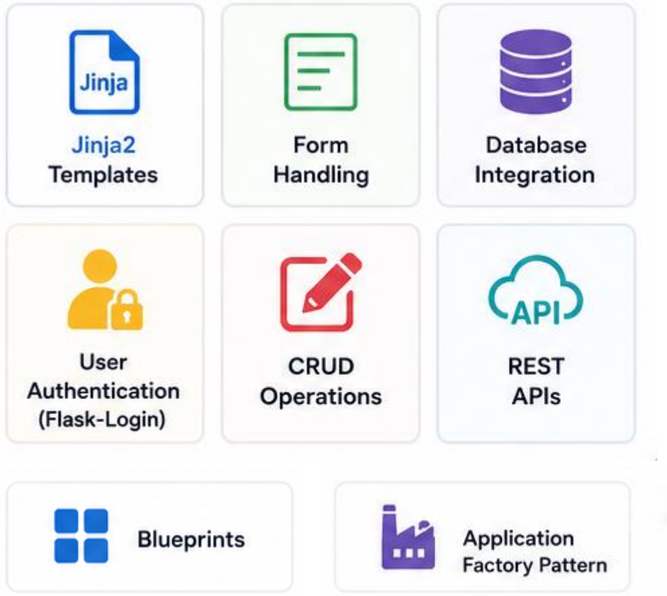
Demo | Use case: Task Management

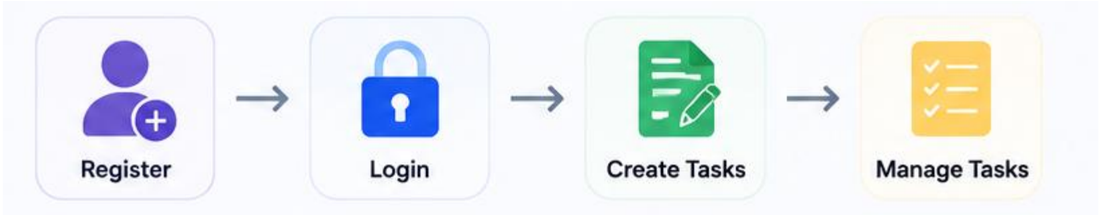
Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

Task Management



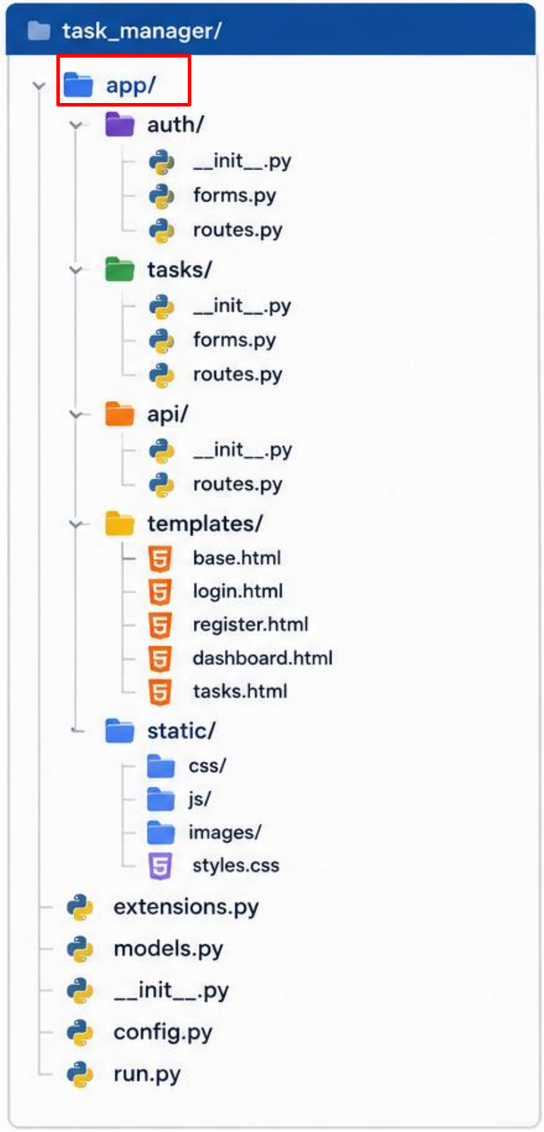




Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

Task Management

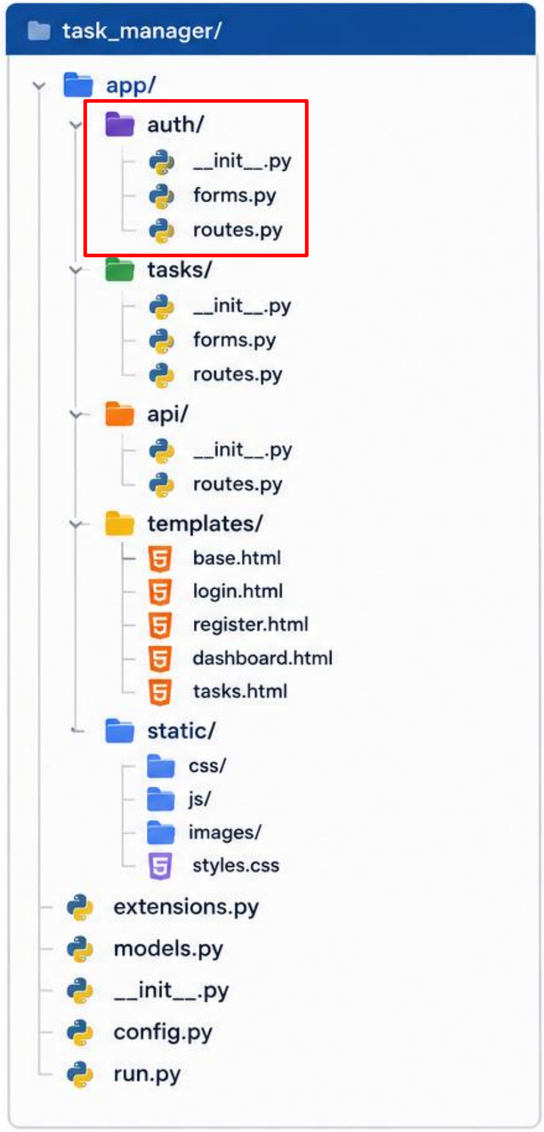


- Core application package

ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Task Management

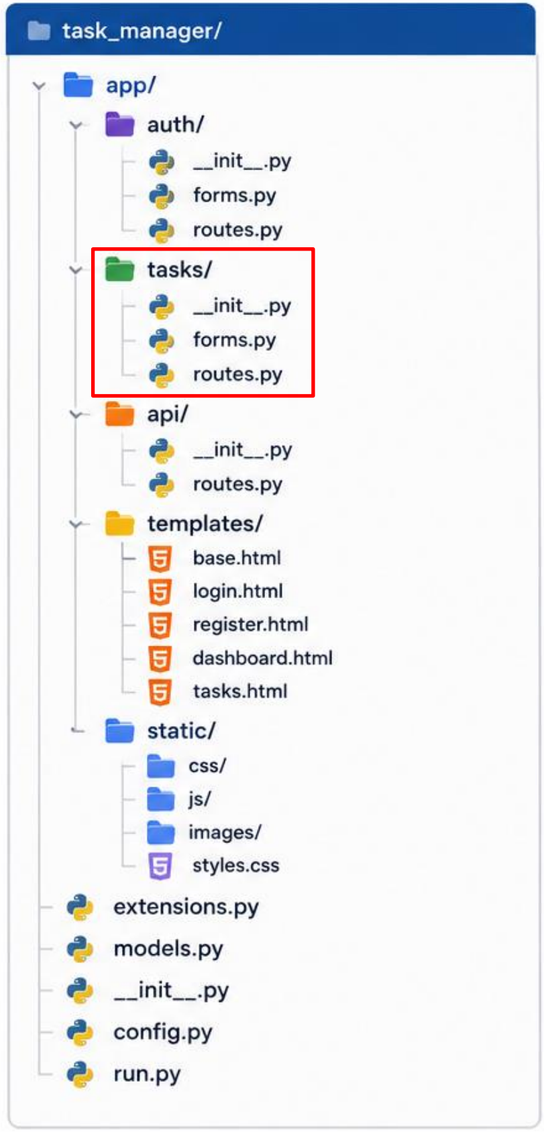


- Handles user authentication features
- forms.py → Registration, Login, and Logout
- routes.py → Authentication, routes, and session handling

ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Task Management

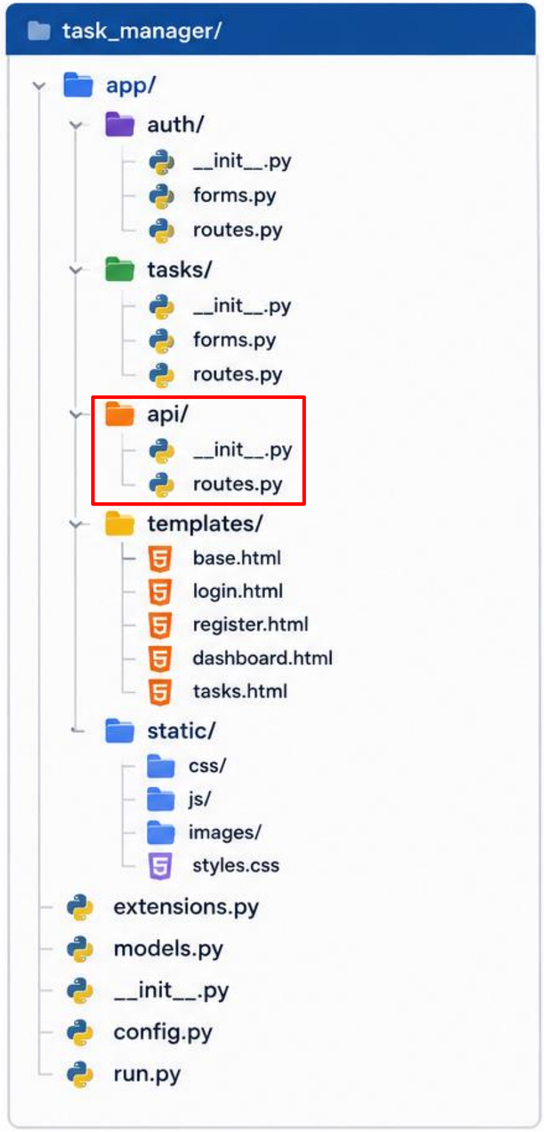


- Manages task-related operations – CRUD operations on tasks

ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Task Management

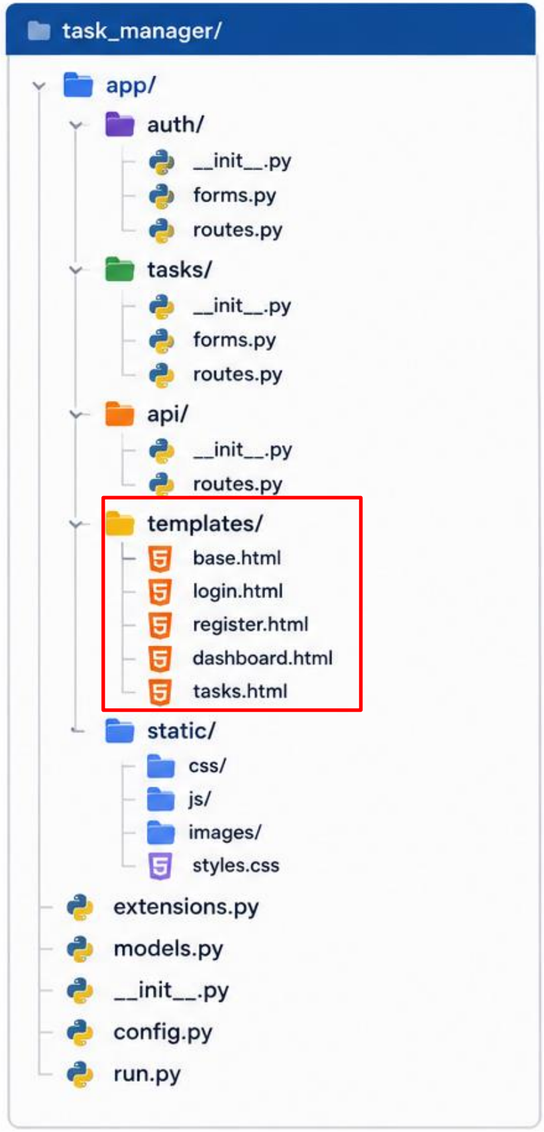


- REST API development and JSON responses
- Endpoints : /api/tasks

ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Task Management

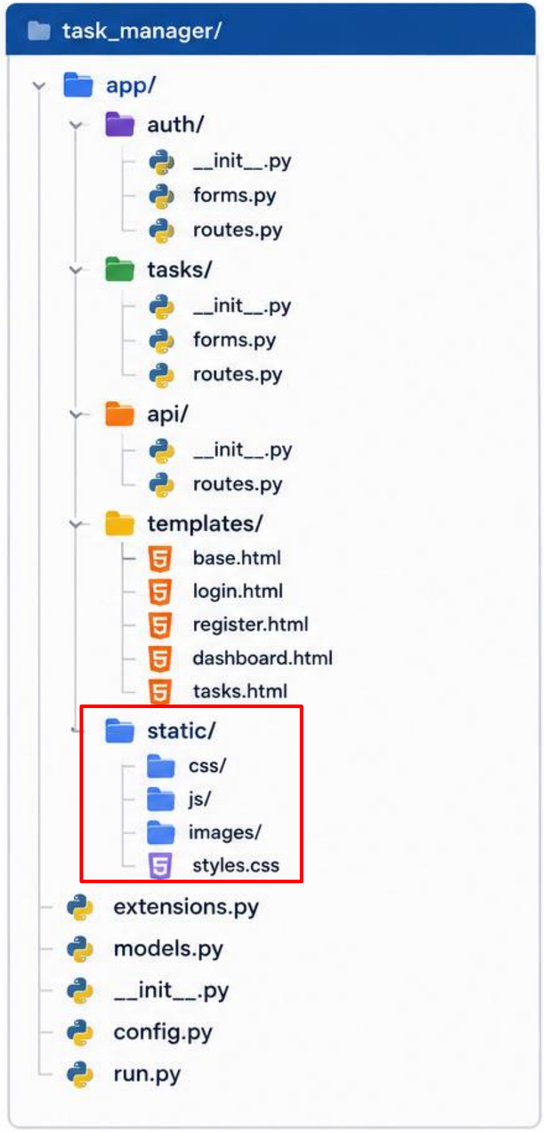


- Stores Jinja2 HTML templates

ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Task Management

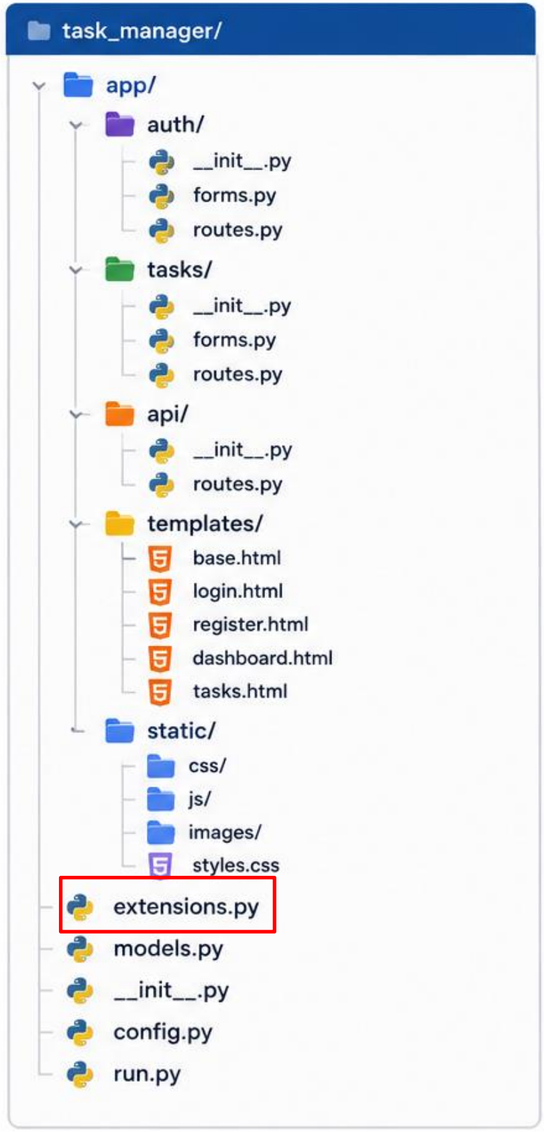


- Contains static assets

ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Task Management

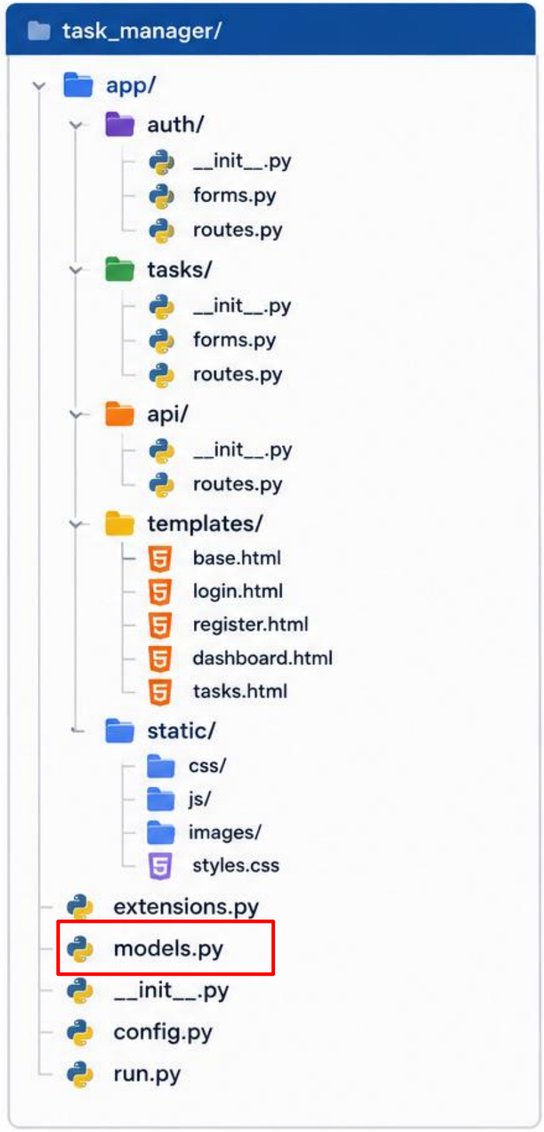


- Initializes Flask extensions like SQLAlchemy, Flask-Login, and Flask-Migrate

ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Task Management

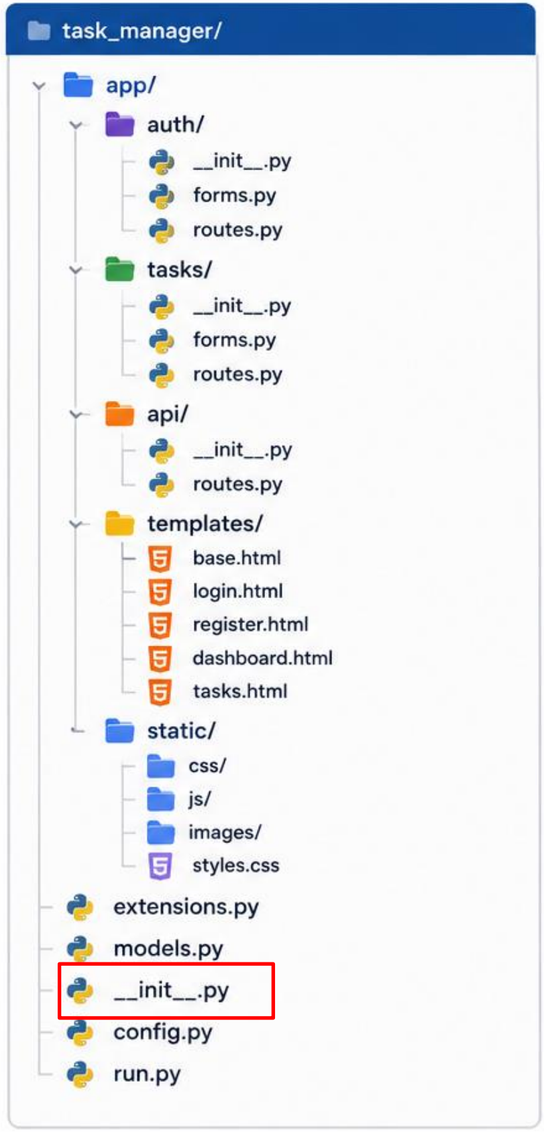


- Defines database models :
  - User and Task
  - Including columns, relationships, and helper methods like password hashing and verification

ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Task Management

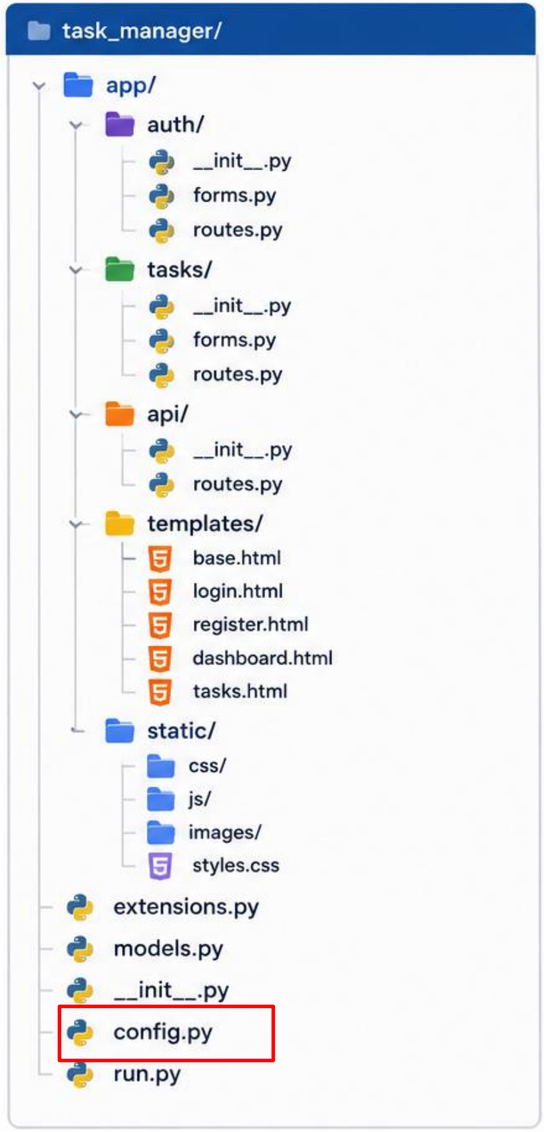


- Creates and Configures Flask application using Application Factory Pattern
- Initializes Extensions
- Loads Configurations
- Registers Blueprints

ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Task Management

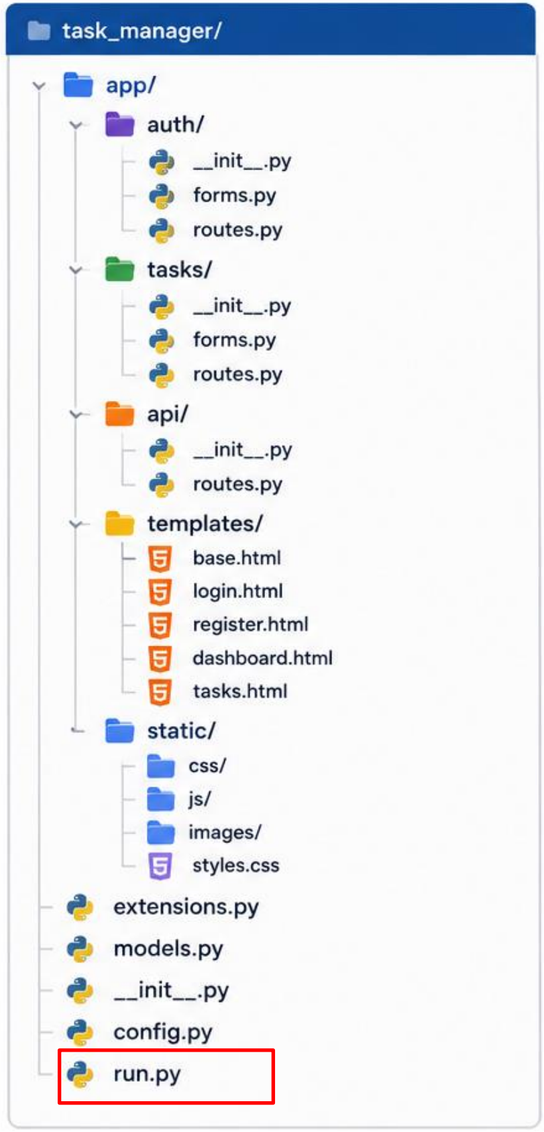


- Stores application configuration settings:
  - Secret keys
  - Database URI
  - Flask environment settings

ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Task Management



- Main entry point of the project

ThinkRook
By Thinknyx Technologies LLP

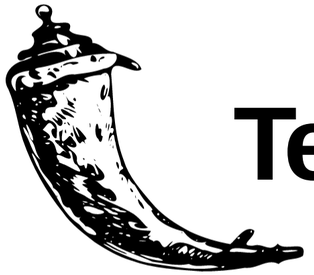
Copyright © Thinknyx Technologies LLP

Thank you



ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP



Testing Flask Application

ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP



Testing Flask Application

Section Overview

- Securing cookies
- Preventing XSS attacks
- Defending against SQL Injection attacks
- Environment variables and secret management
- Summary of essential security concepts covered earlier in the course
- Security best practices and recommendations

ThinkRook
By Thinknyx Technologies LLP



Securing Cookies

Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

Securing Cookies

Cookies store session data, login states, and user preferences in web applications

Proper cookie security prevents session theft and unauthorized account access

- SECRET_KEY

```
app.config["SECRET_KEY"] = "mysecretkey"
```

  - SECRET_KEY signs Flask session cookies using a cryptographic signature
  - Flask verifies the cookie signature on every request
  - Modified or tampered cookies are automatically rejected by Flask

Never hardcode the secret key directly inside the source code

```
import os
app.config["SECRET_KEY"] = os.getenv("SECRET_KEY")
```

ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Securing Cookies

- SESSION_COOKIE_HTTPONLY

```
app.config["SESSION_COOKIE_HTTPONLY"] = True
```

  - **SESSION_COOKIE_HTTPONLY=True** prevents JavaScript from accessing session cookies
  - Blocks cookie access through **document.cookie**
  - Helps protect against cookie theft during XSS attacks
  - Enhances overall session security

ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Securing Cookies

- SESSION_COOKIE_SECURE

```
app.config["SESSION_COOKIE_SECURE"] = True
```

  - Ensures cookies are sent only over HTTPS connections
  - Prevents cookies from being transmitted through normal HTTP
  - Uses encrypted HTTPS communication for cookie transmission

Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

Securing Cookies

- SESSION_COOKIE_SAMESITE

```
app.config["SESSION_COOKIE_SAMESITE"] = "Lax"
```

  - Helps protect against CSRF attacks
  - Controls whether cookies are sent with cross-site requests
  - **Strict** → Cookies are sent only from the same site
  - **Lax** → Provides balanced protection and is commonly recommended
  - **None** → Allows cross-site cookies and requires HTTPS

Lax is a good default option for most Flask applications

ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Securing Cookies

- Session expiration

```
from datetime import timedelta
app.config["PERMANENT_SESSION_LIFETIME"] = timedelta(minutes=30)
```

  - Expires inactive sessions after 30 minutes
  - Expired sessions reduce the risk of stolen session cookies being misused
  - Sensitive data should never be stored directly in cookies
  - Avoid storing passwords, API keys, banking details, and confidential personal data in cookies
  - Flask signs cookies to prevent modification, not visibility
  - Cookie contents remain visible to users

ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Securing Cookies

- How Flask Sessions Work Internally.

```
from flask import session
session["username"] = "admin"
```

  - Flask serializes session data and stores it in a signed cookie
  - Cookies are validated using the SECRET_KEY when sent back by the browser
  - Flask automatically rejects tampered cookies with invalid signatures

ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Securing Cookies

- Attack Scenarios

  Suppose **HttpOnly is disabled**

```
alert(document.cookie)
```

  - This exposes session cookies to attackers. But with **SESSION_COOKIE_HTTPONLY=True**, JavaScript cannot access the cookie

  - **HTTPS is not enabled**, cookies may be intercepted over public WiFi networks
  - **SESSION_COOKIE_SECURE=True** is extremely important in production environments

ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Securing Cookies

- Production Configuration

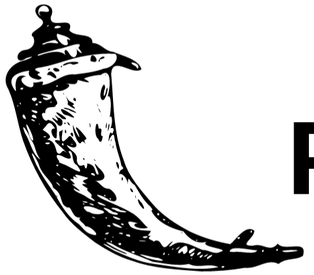
```
app.config.update(
    SECRET_KEY=os.getenv("SECRET_KEY"),
    SESSION_COOKIE_HTTPONLY=True,
    SESSION_COOKIE_SECURE=True,
    SESSION_COOKIE_SAMESITE="Lax"
)
```

  - Use a strong **SECRET_KEY** for cookie signing
  - Enable **HttpOnly** to prevent JavaScript access to cookies
  - Enable Secure to send cookies only over HTTPS
  - Configure SameSite to protect against CSRF attacks
  - Use HTTPS for secure communication
  - Set proper session expiration
  - Helps protect against session hijacking, XSS attacks, CSRF attacks, and cookie theft



ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP



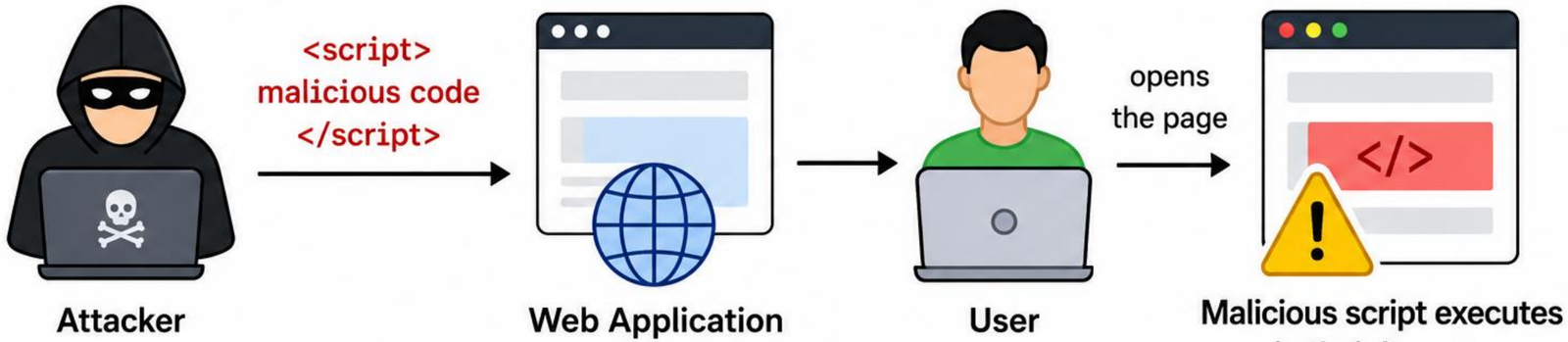
Preventing XSS

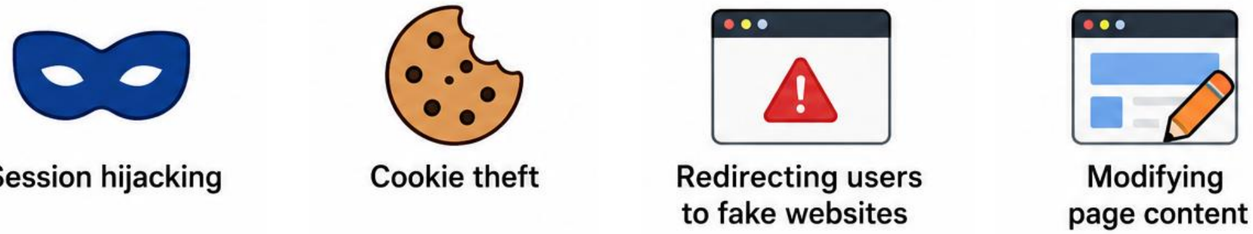
Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

Preventing XSS

Cross-Site Scripting, or XSS, is a **security vulnerability**

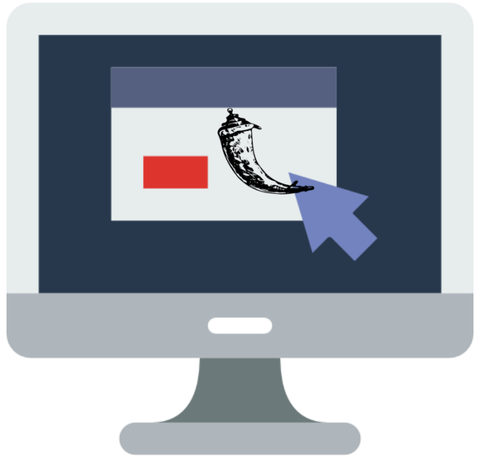




ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Preventing XSS



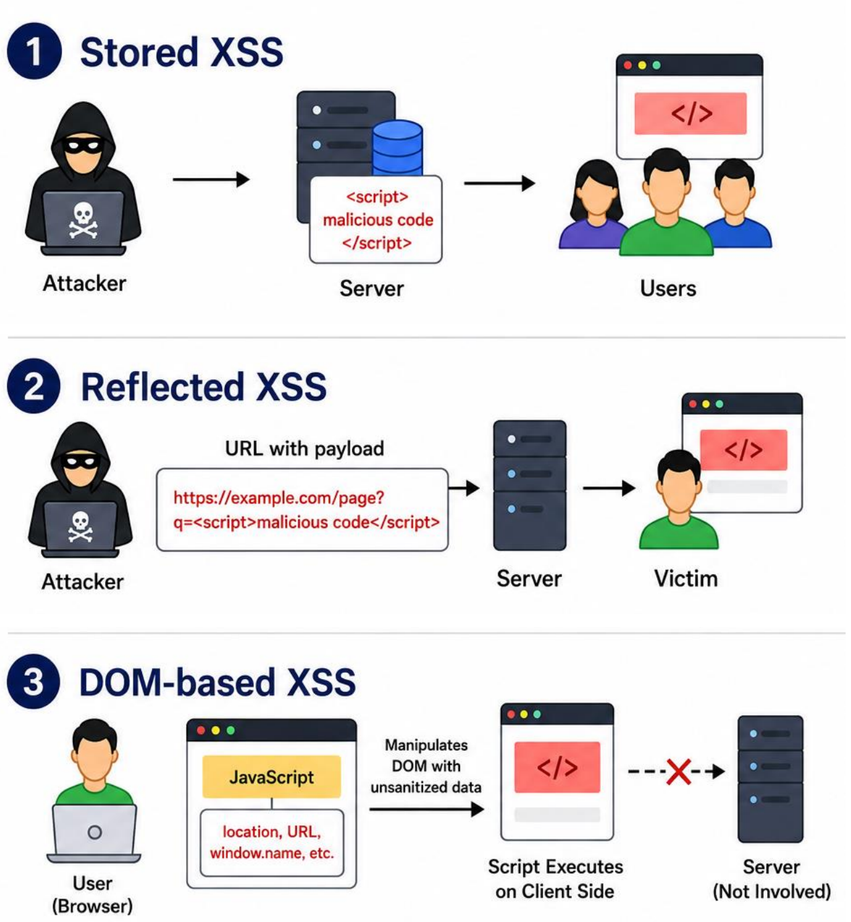
XSS usually happens when user input is displayed on a webpage without **proper escaping** or **sanitization**

Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

Preventing XSS

➢ Types of XSS



- Script stored on server
- Stored in database/comment/forum
- Executes for every user
- Affects multiple users

- Script in URL/HTTP request
- Server reflects it back
- Victim clicks crafted link
- Not stored on server

- Vulnerability in client-side JavaScript
- Uses unsanitized data
- Server may be innocent
- Executed purely on client side

ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Preventing XSS

Never directly render untrusted user input using **“|safe”**

- **Example**

```
<p>{{ user_input | safe }}</p>
```

Safe filter disables escaping and allows **HTML** or **JavaScript** execution

```
<script>alert("Hacked")</script>
```

ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Preventing XSS

Jinja2 Auto Escaping: Always render data using templates instead of manually building HTML strings

- **Example**

```
Good -> <p>{{ comment }}</p>
```

```
Bad -> return f"<p>{comment}</p>"
```

Validate input user: Restrict what users can submit

- **Example**

```
import re
username = request.form.get("username")
if not re.match(r"^[a-zA-Z0-9_]+$", username):
return "Invalid Username"
```

Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

Preventing XSS

Sanitize HTML Content: If your application allows formatted text, sanitize it before rendering

- **Example**

```
import bleach
clean_text = bleach.clean(user_input)
```

Use Content Security Policy (CSP): CSP restricts which scripts can execute in the browser

- **Example**

```
@app.after_request
def add_security_headers(response):
    response.headers["Content-Security-
Policy"] = "default-src 'self'"
    return response
```

Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

Preventing XSS

Another critical security consideration is **unquoted attributes**

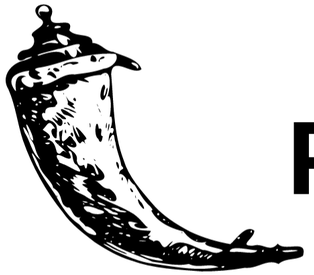




Link: https://flask.palletsprojects.com/en/stable/web-security/#cross-site-scripting-xss

ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP



Preventing SQL Injection

Copyright © Thinknyx Technologies LLP

Preventing SQL Injection

SQL Injection is a security vulnerability where an attacker inserts malicious SQL queries into user input fields to manipulate the database



- Occurs when user input is directly concatenated into SQL queries
- Prevent using parameterized queries instead of manual SQL statements
- Use SQLAlchemy ORM for secure database interactions

ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Preventing SQL Injection

- **Example**
  - Instead of writing a query like this:

```
query = f"SELECT * FROM users WHERE username = '{username}'"
```

  - Use **parameterized queries** like this:

```
cursor.execute(
    "SELECT * FROM users WHERE username = ?",
    (username,)
)
```

  - Use **SQLAlchemy** like this:

```
user = User.query.filter_by(username=username).first()
```

Database treats user input as **data** rather than executable SQL code, which blocks malicious injections such as ' **OR 1=1 --**

ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Preventing SQL Injection

- Validating user input
- Limiting database permissions
- Using Flask-WTF forms for safer form handling
- Never exposing raw database errors in production mode

ThinkRook
By Thinknyx Technologies LLP

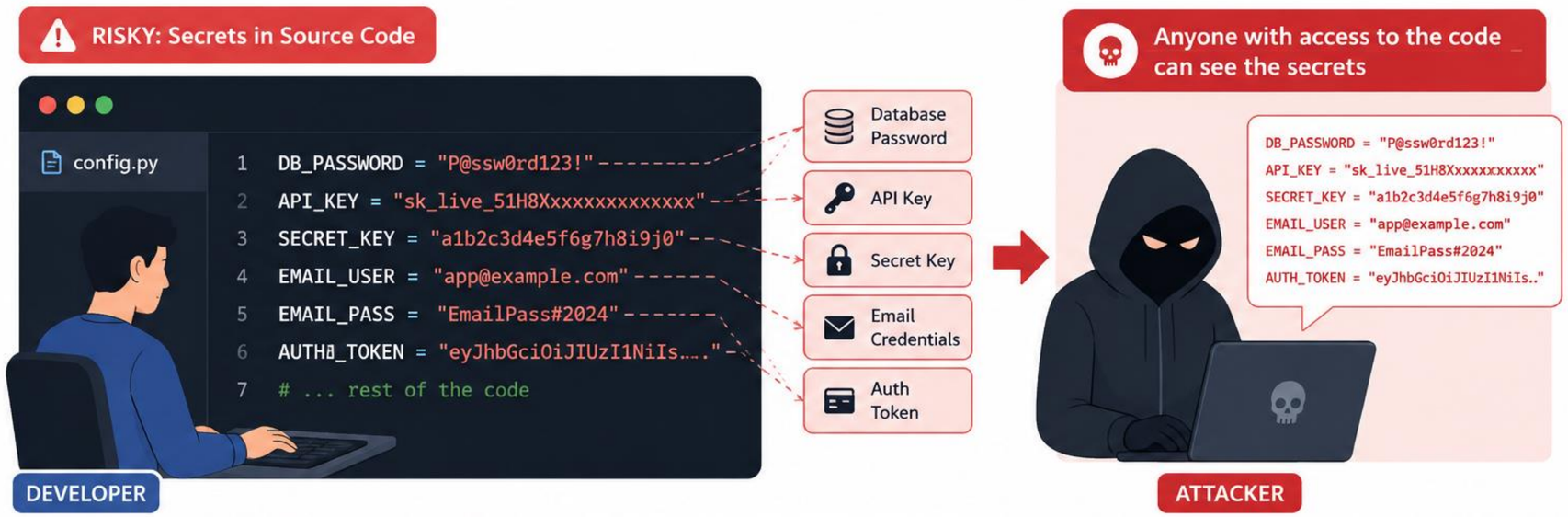
Copyright © Thinknyx Technologies LLP



Environment Variables & Secret Management

Copyright © Thinknyx Technologies LLP

Environment Variables & Secret Management



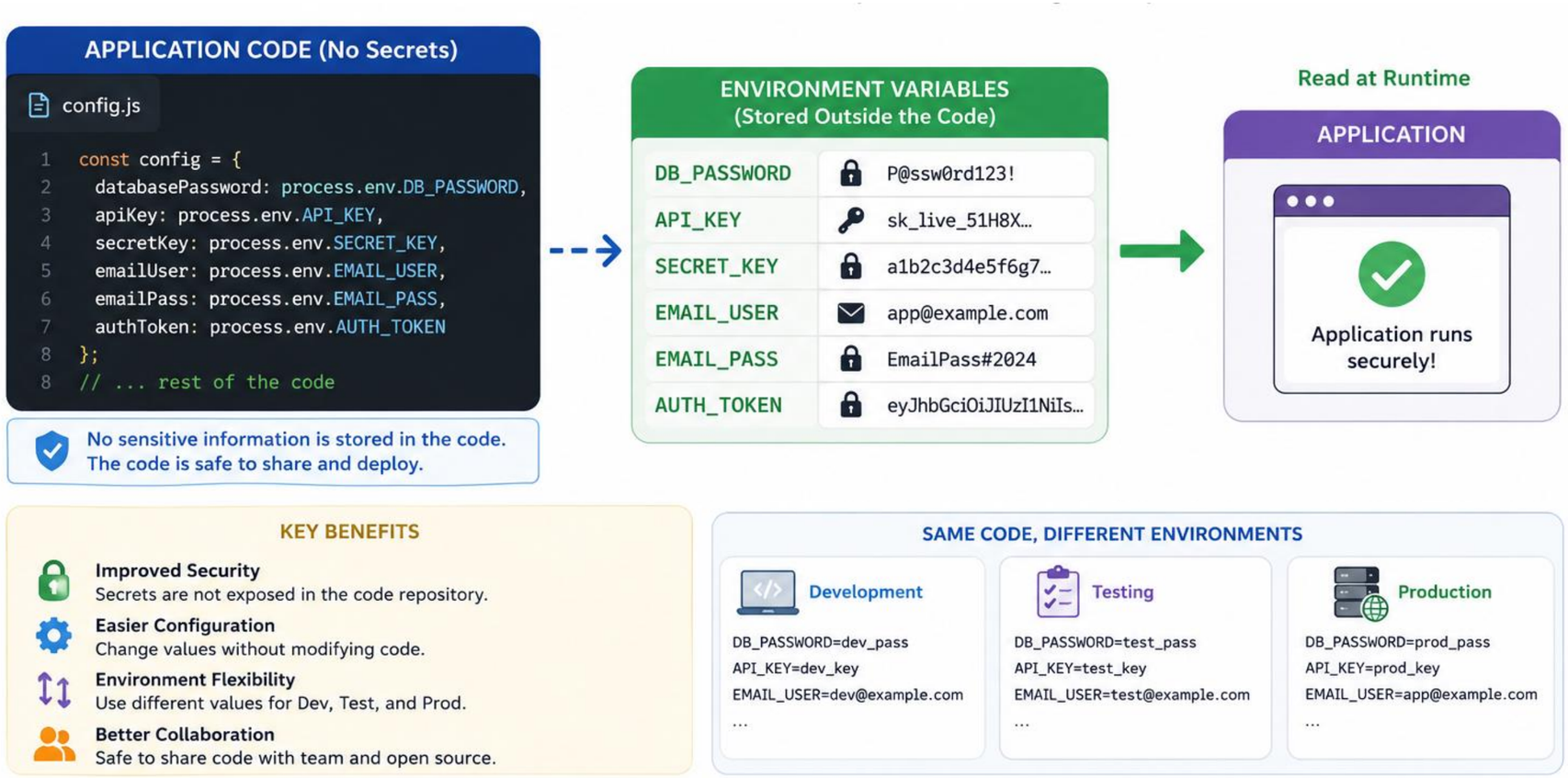


ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Environment Variables & Secret Management

➢ Environment Variables



ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Environment Variables & Secret Management

- **Example**
  - Instead of writing this directly in the code

```
app.config["SECRET_KEY"] = "mysecretpassword"
```

  - Environment variable

```
SECRET_KEY=mysecretpassword
```

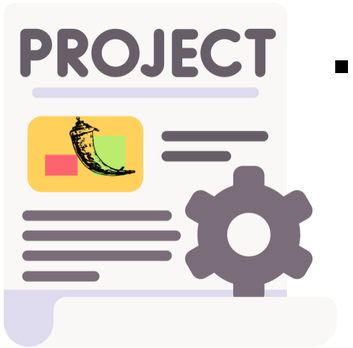
  - Inside the Flask application

```
import os
from flask import Flask
app = Flask(__name__)
app.config["SECRET_KEY"] = os.getenv("SECRET_KEY")
```

ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Environment Variables & Secret Management



- Environment variables are commonly stored inside a .env file during development
  - A **.env** file may look like this:

```
SECRET_KEY=mysecretpassword
DATABASE_URL=sqlite:///users.db
MAIL_PASSWORD=examplepassword
```

    - Used **python-dotenv** to load environment variables automatically
    - Flask securely reads values from the **.env** file
    - Avoid storing sensitive information in the source code
    - Never upload the **.env** file to GitHub
    - Developers usually add the **.env** file to **.gitignore**
    - Prevents sensitive information from being exposed publicly

ThinkRook
By Thinknyx Technologies LLP

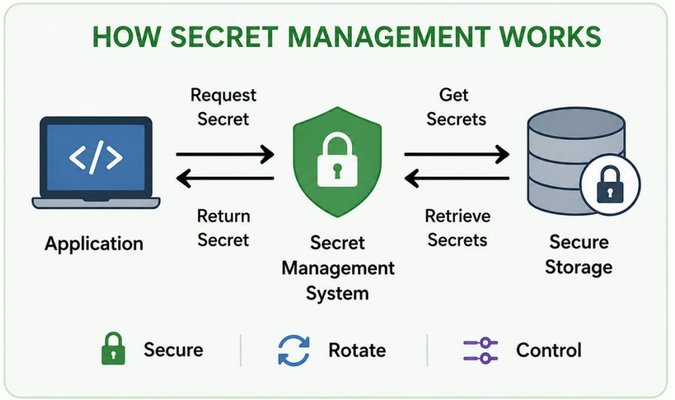
Copyright © Thinknyx Technologies LLP

Environment Variables & Secret Management

➢ Secret Management

Securely storing, **accessing**, **rotating**, and **controlling sensitive credentials** used by applications

In small projects, `.env` files are usually enough.



But in large production systems, organizations use dedicated secret management tools.

HashiCorp Vault

Amazon Web Services Secrets Manager

Google Cloud Secret Manager

Microsoft Azure Key Vault



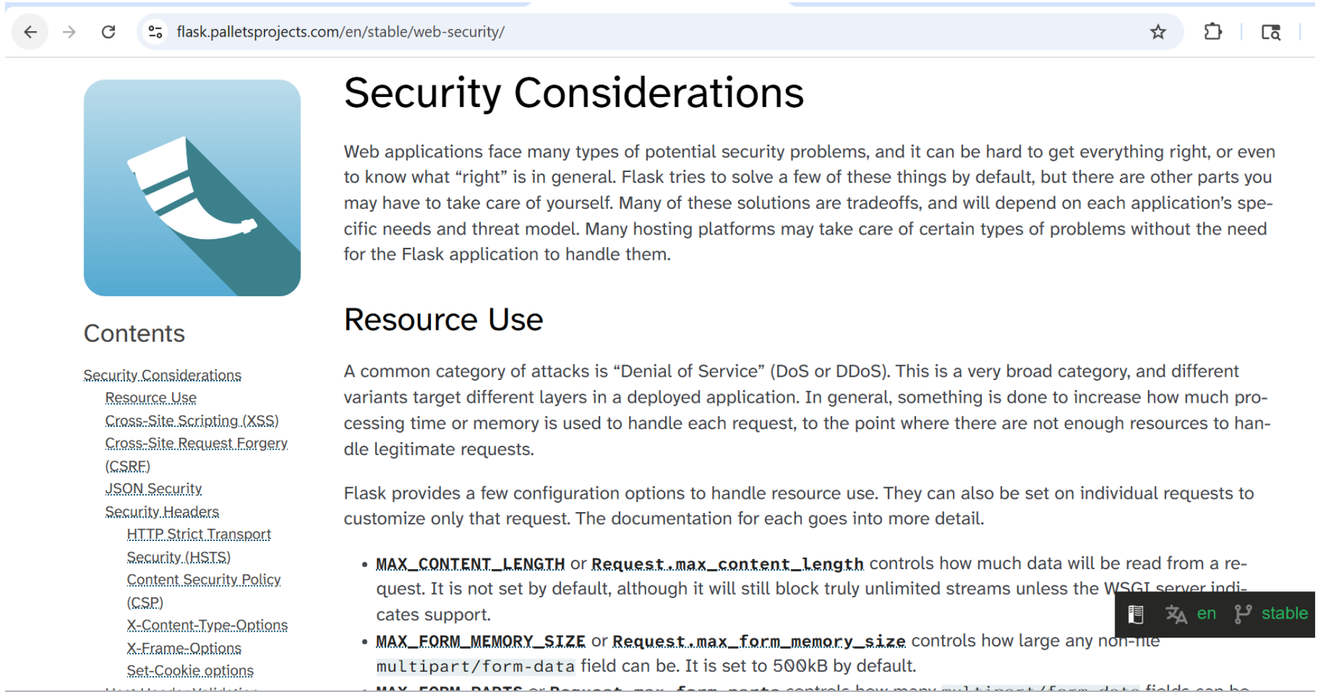
Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP

Environment Variables & Secret Management

Environment variables and **secret management** help us build more secure and production-ready Flask applications

URL: https://flask.palletsprojects.com/en/stable/web-security/



Copyright © Thinknyx Technologies LLP

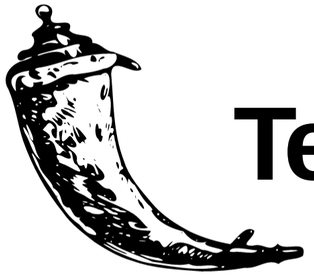
ThinkRook
By Thinknyx Technologies LLP

Thank you



ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP



Testing Flask Applications

Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP



Testing Flask Applications

Section Overview

- Understanding Unit Testing in Flask
- Introduction to Flask Testing Client
- Working with Pytest in Flask
- Mocking Database Operations

ThinkRook
By Thinknyx Technologies LLP



Demo | Unit Testing

Copyright © Thinknyx Technologies LLP

Unit Testing

Unit testing in Flask is the process of testing individual parts of a Flask application



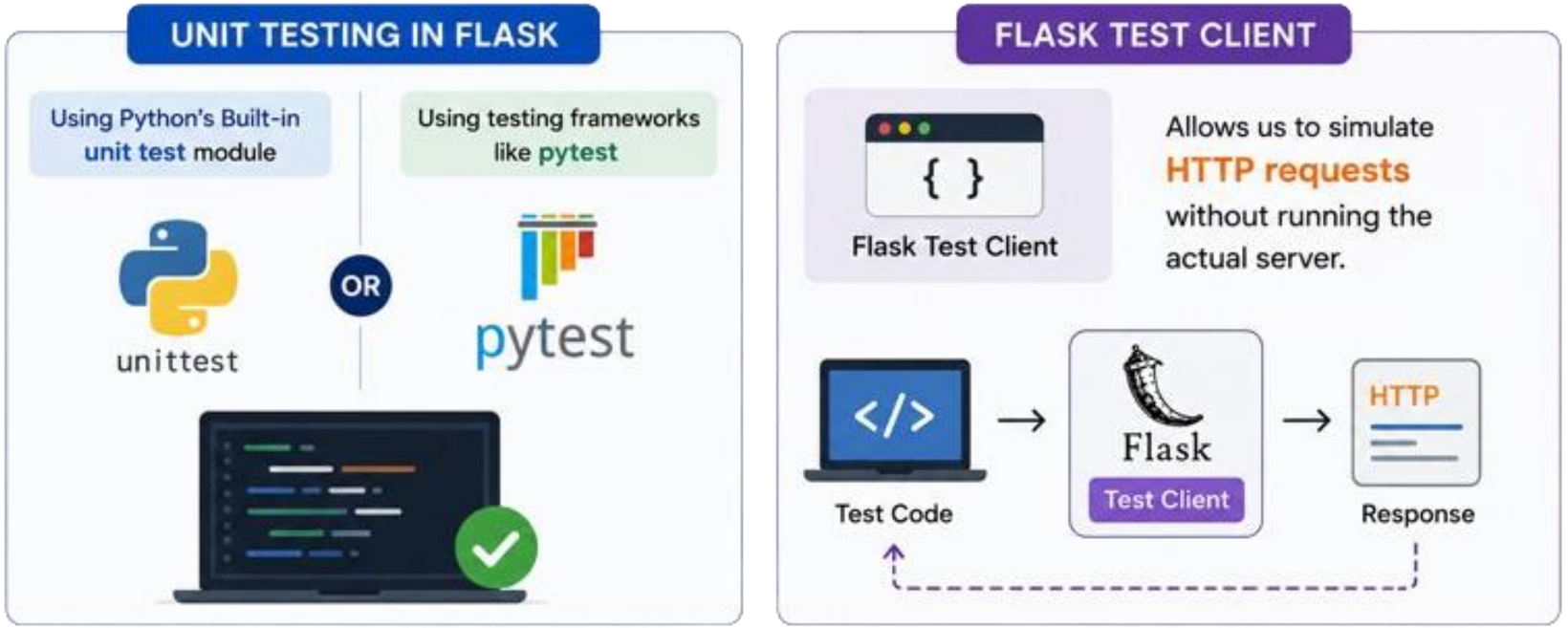


ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Unit Testing





ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Why Unit Testing is Important?

- It ensures application features work correctly
- Helps catch bugs before deployment
- Makes code easier to maintain and refactor
- Improves application stability
- Saves debugging time in large projects



ThinkRook
By Thinknyx Technologies LLP

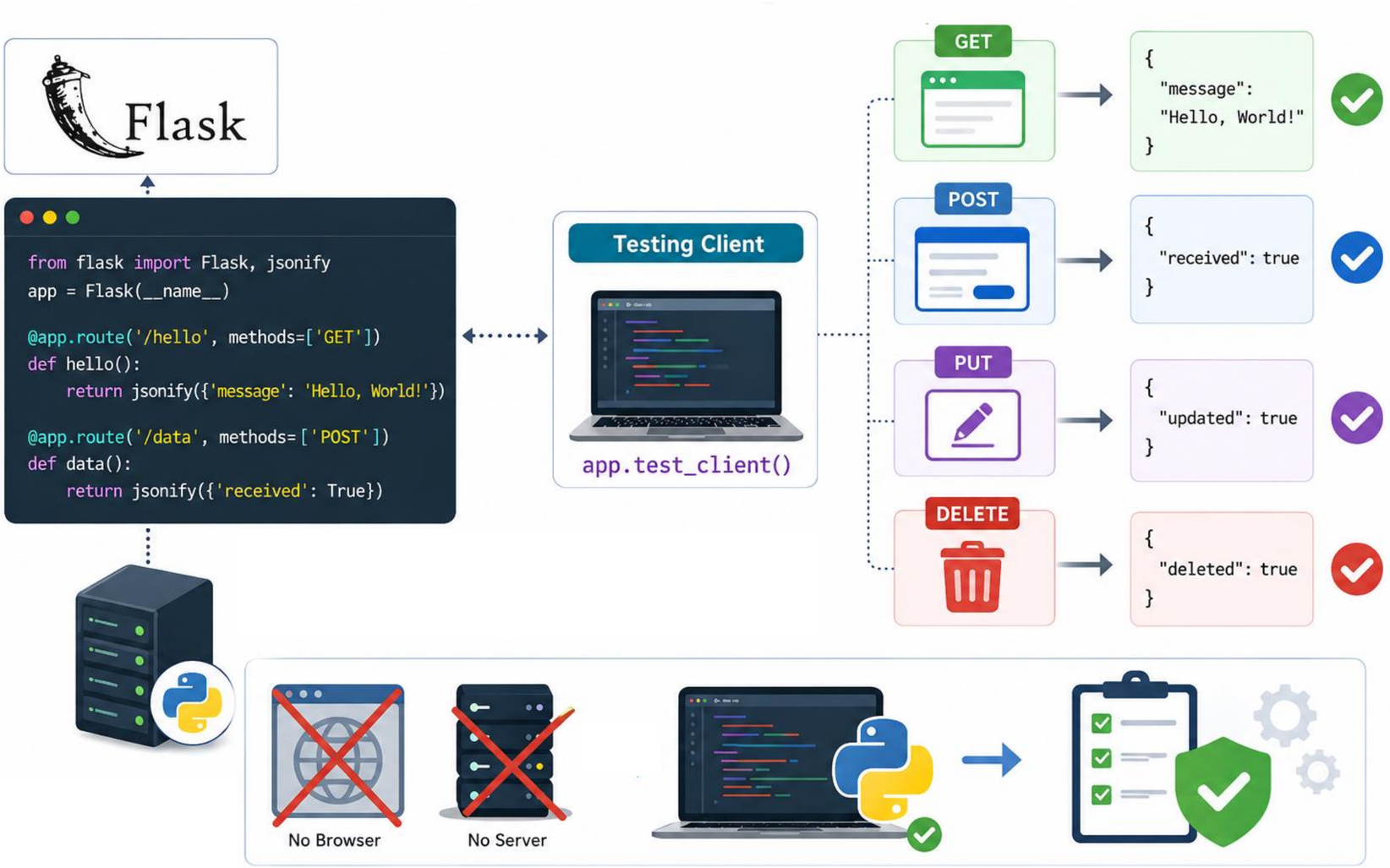
Copyright © Thinknyx Technologies LLP



Demo | Flask Testing Client

Copyright © Thinknyx Technologies LLP

Flask Testing Client



Copyright © Thinknyx Technologies LLP

ThinkRook
By Thinknyx Technologies LLP



Demo | Pytest with Flask

Copyright © Thinknyx Technologies LLP

Pytest with Flask

Pytest is a powerful and popular Python testing framework used for writing and running tests in Flask applications

It provides a **simpler** and **cleaner syntax** compared to Python's built-in **unittest** module.




```
pip install pytest
```

Pytest works very well with **Flask's built-in testing client** and helps **automate application testing efficiently**.



ThinkRook
By Thinknyx Technologies LLP

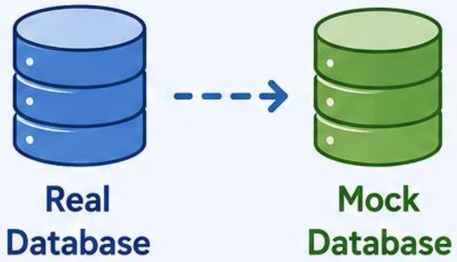
Copyright © Thinknyx Technologies LLP



Demo | Mocking Database

Copyright © Thinknyx Technologies LLP

Mocking Database



Replacing the real database with a fake or simulated version during testing



Testing application logic without affecting actual data



Faster testing



Isolated testing environment



More reliable tests



No dependency on a real database connection

ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP

Thank you



ThinkRook
By Thinknyx Technologies LLP

Copyright © Thinknyx Technologies LLP



Flask Web Development Bootcamp | Hands-On Learning

- By Thinknyx Technologies

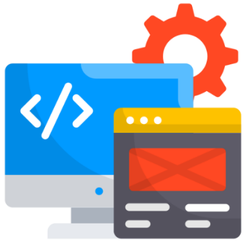


- Built Web Applications using Flask
- Worked with Routing, Requests & Responses
- Developed dynamic pages with Jinja2 Templates
- Integrated Forms, Databases & SQLAlchemy
- Implemented Authentication and RESTful APIs
- Applied Blueprints for Modular Architecture
- Used Logging, Error Handling & Security Practices
- Built & Tested complete Flask Applications

ThinkRook
By Thinknyx Technologies LLP



Design



Develop



Secure



Test



Deploy

Real-world Environments



Web Applications

REST APIS

ThinkRook
By Thinknyx Technologies LLP



AI Ecosystem for the Absolute Beginners - Hands-On



Generative AI Essentials - Practical use Cases

ThinkRook
By Thinknyx Technologies LLP



Follow us on:

@thinknyx

@thinknyx

@thinknyx-technologies

@thinknyx

@thinknyx-technologies

ThinkRook
By Thinknyx Technologies LLP